TW128: A Tree-Structured Duplex Authenticated-Encryption Mode

Coda Hale

Hale Institute for Bicycle Appreciation

Abstract. We present TW128, a nonce-based authenticated-encryption mode with associated data built as a two-level tree of keyed duplexes over the KECCAK-p[1600, 12] permutation. A serial root duplex binds the key, nonce, and associated data, encrypts the first message chunk, and aggregates a tag from every remaining chunk into a single authentication tag; the remaining chunks are handled by independent leaf duplexes that share no state once the key and nonce are fixed and can therefore be evaluated in parallel across SIMD lanes.

We analyze TW128 in the public-permutation model, proving each bound first against an intermediate random oracle and then transferring it to the real permutation by the flat-sponge lift. We obtain concrete bounds for all-in-one authenticated encryption, multi-user indistinguishability, and both facets of commitment security: the collision facet—s-way committing security in the strongest full-input sense (CMTDs-4) and its tag-projected strengthening, in which the root tag alone is the compact commitment—and the preimage facet, context discoverability against that same root tag. At the implemented parameters these notions meet a 128-bit security target. Vectorized `amd64` and `arm64` implementations realize the tree's leaf parallelism, reaching multi-gigabyte-per-second throughput within a small factor of hardware-accelerated AES-128-GCM.

Keywords: authenticated encryption · committing security · sponge constructions · duplex constructions · Keccak

1 Introduction

Authenticated encryption with associated data is the workhorse of applied cryptography, and the sponge and duplex constructions over a single public permutation offer an attractive route to it. The duplex of [BDPVA11] turns one permutation—here KECCAK-p[1600, 12], the round-reduced permutation underlying the [Nat15] standard—into a single-pass authenticated cipher with a small, constant memory footprint and no separate block cipher or universal-hash key schedule. Its security rests on the indifferentiability of the sponge from a random oracle [BDPVA08], which reduces analysis of the full construction to a random-oracle argument plus one capacity-birthday loss.

This compactness comes at a structural cost. A duplex is inherently *sequential*: each call absorbs its input into a state produced by the previous call, so a single duplex cannot exploit the wide SIMD and multicore parallelism of modern processors, on which competing designs built from AES reach many bytes per cycle. A second, orthogonal gap is *commitment*. Widely deployed AEADs such as AES-GCM and ChaCha20-Poly1305 are not key-committing: an adversary can craft a single ciphertext that decrypts to valid plaintexts under two or more keys [DGRW18, ADG+22]. Far from a curiosity, this enables concrete abuses—the "invisible salamander" that defeats Facebook's attachment-franking scheme [DGRW18], ciphertexts that open to different well-formed files under different keys [ADG+22], and partitioning-oracle attacks that recover passwords from systems built on

non-committing AEAD [LGR21]. The committing-AE line that addresses these threats was initiated by [GLR17] and developed into formal committing notions with efficient generic transforms by [BH22]. Commitment has two facets: committing security is a collision requirement—no two contexts open one ciphertext—while its preimage counterpart, *context discoverability* [MLGR23], forbids an adversary from *finding* even one decrypting context for a target ciphertext. The latter's absence underlies the recent context-discovery attacks that break GCM, CCM, EAX, SIV, and OCB3 [MLGR23], all of which delegate authenticity to an algebraically invertible MAC. A mode that targets these notions natively, rather than through an add-on transform, avoids the transform's overhead and binds commitment to the same permutation it already uses for confidentiality and integrity.

TW128 (*TreeWrap*) addresses both gaps with a two-level tree of keyed duplexes. A serial *root* duplex, initialized with the key K and nonce N, absorbs the associated data, encrypts the first message chunk, and absorbs a tag from every other chunk before emitting the final authentication tag. The remaining chunks are processed by independent *leaf* duplexes, each keyed by (K, N, j) and operating on disjoint state and disjoint input. Because the leaves share nothing once (K, N) is fixed, they can be evaluated in parallel across SIMD lanes, recovering the throughput a flat duplex forgoes; and because the root binds the associated data, the first chunk, and every leaf tag into the single output tag, that tag is a sponge image of the entire (K, N, A, M) input. The structure ensures only that distinct (K, N, A) triples reach distinct root transcripts; the committing security itself rests on the indistinguishability of the sponge, which makes those transcripts' tags collision-resistant. In fact the root tag alone can serve as the compact commitment string: even if different openings use different ciphertext bodies, a successful full-input collision must either collide the root tag candidates or first collide leaf tags. The same sponge image is preimage-resistant, yielding context discoverability against that fixed tag (Section 8). Every duplex still operates on a bounded 1600-bit state, so the memory footprint remains constant in the message length.

We analyze TW128 in the public-permutation model and obtain concrete bounds for the standard and tag-projected notions: all-in-one authenticated encryption in the sense of [RS06], its multi-user indistinguishability lift in the sense of [BT16], and both facets of commitment: the collision facet—s-way committing security in the strongest full-input sense (CMTDs-4) of [BH22] and its tag-projected compact strengthening—and the preimage facet, context discoverability in the sense of [MLGR23]. Each bound is proved first against an intermediate random oracle and then transferred to the real permutation by the flat-sponge lift of [BDPVA08]. At the implemented parameters ($k = c = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$), these guarantees meet a 128-bit security target for adversaries running up to $N_{\mathsf{tot}} \leq 2^{63}$ KECCAK-p[1600, 12] calls—on the order of 2^{71} bytes of processed data, beyond any realistic deployment lifetime. Optimized `amd64` and `arm64` implementations turn the tree's leaf parallelism into measured throughput within a small factor of hardware-accelerated AES-128-GCM (Section 9).

1.1 Our Contributions

- **The TW128 mode.** We specify TW128, a nonce-based AEAD with associated data built as a two-level tree of strict [BDPVA11] duplexes over KECCAK-p[1600, 12] (Section 3): a serial root duplex that binds (K, N, A), the first chunk, and the leaf tags into a single authentication tag, over a set of parallel, independently keyed leaf duplexes. All phases are domain-separated within one permutation, and the state is constant-size regardless of message length.

- **All-in-one AE security.** We prove a concrete bound on the all-in-one AE advantage [RS06] in the public-permutation model (Theorem 1), obtained by composing random-oracle privacy and INT-CTXT bounds through the generic-composition argument of

[BN00] and lifting the result to the real permutation.

- **Multi-user security.** We give a concrete multi-user indistinguishability bound [BT16] for D users (Theorem 2), obtained by a key-collision-stopped hybrid over the users that reuses the single-user privacy and INT-CTXT envelopes.

- **Committing security.** We prove that TW128 achieves s-way CMTDs-4 committing security—commitment to the full (K, N, A, M) input, for any number $s \geq 2$ of claimed openings—with a concrete root-collision bound (Corollary 1), and that it satisfies the stronger tag-projected notion, in which the 256-bit root tag alone is a binding commitment to that input even when the claimed openings use distinct ciphertext bodies (Theorem 3), matching the compactly committing AEAD goal of [GLR17]. We derive the remaining [BH22] committing notions (CMT and CMTD for $\ell \in \{1, 3, 4\}$) as direct bounds (Section C). Committing security is native to the mode, with no generic transform.

- **Context discoverability.** We prove that TW128 is context-discovery resistant in the sense of [MLGR23]: given a target tag fixed in advance, no adversary can find a context and ciphertext body that decrypt successfully under it, with a concrete root-preimage bound (Theorem 4). This is the preimage counterpart of committing security; because the fixed target is only the 256-bit root tag rather than a full ciphertext, the result already concerns that compact tag. Taken with the collision facet of tag-CMTDs-4, it makes TW128's 256-bit root tag both collision- and preimage-resistant as a commitment to the full encryption input—in one tag, as strong a commitment as a cryptographic hash. Because TW128's authenticity is not delegated to an invertible MAC, the context-discovery attacks of [MLGR23] against GCM, CCM, EAX, SIV, and OCB3 do not apply. The unrestricted notion we prove dominates the restricted per-component variants of [MLGR23].

- **Proof methodology.** We carry out the analysis in a dedicated TW128 random-oracle model and transfer every bound to the public-permutation setting through the [BDPVA08] flat-sponge lift (Section B). A derived-adversary argument shows that each direct primitive query induces at most one random-oracle query, so the random-oracle envelopes are instantiated at the direct-query count N_{prim} rather than the total work N_{tot}, separating the single capacity-birthday loss $\mathsf{Lift}_c(N_{\mathsf{tot}})$ from the scheme-specific terms.

- **Optimized implementations.** We describe and evaluate vectorized `amd64` (AVX-512 and AVX2) and `arm64` (NEON with the SHA-3 extension) implementations whose batched leaf cores realize the tree's parallelism in practice, reaching multi-gigabyte-per-second throughput on contemporary hardware (Section 9).

1.2 Related Work

Sponge and duplex AEADs. Permutation-based authenticated encryption built on the sponge and duplex constructions of [BDPVA08, BDPVA11] is well established. Keyak [BDP+15] is a CAESAR candidate built on the duplex over round-reduced Keccak-p, whose Motorist mode already admits parallel duplex instances. Ascon [DEMS21], the lightweight winner of the CAESAR competition and the basis of the NIST lightweight-cryptography standard, and Xoodyak [DHP+20], a finalist in that process whose Cyclist mode is a deliberately serial simplification of Motorist, are duplex AEADs over the Ascon and Xoodoo permutations. TW128 shares this single-permutation, constant-state duplex foundation but differs in structure: rather than one duplex session, it is a two-level tree whose leaves are *independently keyed* duplexes under domain-separated chunk identifiers. Like Keyak's Motorist mode, the tree admits data-parallel evaluation across SIMD lanes;

the designs differ in where the degree of parallelism lives. Keyak fixes it as a mode parameter—each named instance sets the number Π of parallel duplex instances across which a message is spread, so Π is part of the wire format and must be agreed by the encryptor and decryptor in advance. In TW128 the leaf decomposition is fixed by the mode, but the number of leaves evaluated simultaneously is not part of the transcript: an implementation chooses it at run time to match the host's vector width, and the ciphertext and tag are identical however it is chosen, so a wide-SIMD encryptor and a scalar decryptor interoperate.

Tree hashing. The two-level shape is inspired by tree hashing over the same permutation, in particular KangarooTwelve [BDP+16], which splits its input into fixed-size chunks hashed independently under KECCAK-p[1600, 12] and absorbs their chaining values into a root node. TW128 reuses the round count and the final-node/leaf split of that design and adapts it to authenticated encryption: each leaf chunk is encrypted and tagged independently, and the leaf tags are absorbed by the root in place of chaining values. It also inherits the short-message behavior of KangarooTwelve's *kangaroo hopping*, which merges the first chunk with the hashing of the remaining chunks' chaining values so that an input fitting in a single chunk incurs no tree overhead: in TW128 a single-chunk message engages no leaf duplexes and no aggregation phase, the root processing it directly and emitting the tag as a plain keyed duplex would. Short messages therefore pay none of the tree's cost and retain the latency profile of an unstructured duplex (Section 9.4); the leaf layer, and its parallelism, engage only once a message spans more than one chunk.

Committing AEAD. The study of committing authenticated encryption was initiated by [GLR17], who introduced compactly committing AEAD for message franking, and by [DGRW18], who built it from a dedicated primitive (encryptment); [BH22] subsequently organized the notions into the CMT-1/3/4 and CMTD hierarchy and gave efficient generic transforms (UtC, RtC, HtE) that add commitment to a base scheme. A complementary line shows that widely used and dedicated AEADs fail these notions: the invisible-salamander [DGRW18] and partitioning-oracle [LGR21] attacks, multi-format ciphertexts [ADG+22], the context-discovery and commitment attacks that break CCM, EAX, SIV, GCM, and OCB3 [MLGR23], and an $O(1)$ key-committing attack against all variants of AEGIS [WP14] that refutes the prior belief that AEGIS was committing [IR23]. [MLGR23] also introduce context discoverability, the preimage facet of commitment, and show that their discovery attacks exploit schemes whose authenticity is delegated to a non-preimage-resistant MAC such as GHASH or CMAC. TW128 instead targets both facets directly: its committing and discoverability security follow from the indistinguishability of the sponge, which makes the root tag collision- and preimage-resistant, rather than from a generic transform applied to a non-committing base scheme. Because that tag is a sponge image and not an invertible MAC, the discovery attacks of [MLGR23] have no analogue against TW128.

Closest to TW128 is the recent work of [DHM+24], which also builds nonce-based AE natively on the SHA-3 functions SHAKE and TurboSHAKE—the latter over the same round-reduced KECCAK-p permutation as TW128—obtaining CMT-4 from the collision resistance of the underlying function rather than from a transform. Its schemes target session-based communication and process each message through a sequential duplex chain, whereas TW128, on the same native-committing single-permutation foundation, is structured as a two-level tree whose independent leaves are evaluated in parallel across SIMD lanes. TW128's principal advantage is throughput on large messages, where this leaf parallelism fills vector units that a sequential duplex leaves idle; the cost is the session support that [DHM+24] provides and TW128 does not (Sections 9 and 10.3).

1.3 Paper Organization

Section 2 recalls the notation, sponge and duplex constructions over KECCAK-p, the authenticated-encryption, committing-security [BH22], and context-discoverability [MLGR23] notions used throughout, and the multi-user indistinguishability notion of [BT16]. Section 3 specifies the TW128 construction and its parameters. Section 4 fixes the public-permutation games, the intermediate random-oracle model, and the resource accounting that all later bounds reference. Section 5 states the headline AE, mu-ind, CMTDs-4, tag-CMTDs-4, and CDY theorems and their concrete TW128-parameter forms; Sections 6 and 7 prove the random-oracle privacy and INT-CTXT bounds that feed those theorems, and Section 8 proves the random-oracle CMTDs-3, tag-CMTDs-4, and CDY bounds. Section 9 reports the performance of the reference and optimized implementations. Section 10 compares TW128 to prior sponge AEADs and committing constructions and discusses limitations. Section 11 concludes. Section A collects the structural decoding and accounting lemmas that form the proof core; Section B carries out the [BDPVA08] flat-sponge lift that converts each random-oracle bound to its public-permutation form; Section C gives the remaining [BH22] committing-security bounds; and Section E lists TW128 test vectors.

2 Preliminaries

2.1 Notation

Strings. Write $\{0,1\}$ for the binary alphabet. For an integer $n \geq 0$, $\{0,1\}^n$ denotes the set of n-bit strings, $\{0,1\}^* = \bigcup_{n\geq 0}\{0,1\}^n$ the set of all finite-length bit strings, and ε the empty string. For $x \in \{0,1\}^*$, $|x|$ is the bit length and $x \parallel y$ is the concatenation; the bitwise XOR of two equal-length strings $x, y \in \{0,1\}^n$ is $x \oplus y$. A *byte string* is an element of $(\{0,1\}^8)^*$; its byte length is denoted $\|x\| = |x|/8$. For an integer $0 \leq n < 2^{64}$, $\langle n\rangle_8 \in \{0,1\}^{64}$ is its eight-byte little-endian encoding. For a finite or infinite bit string x and $0 \leq \tau \leq |x|$ when x is finite, $\lfloor x\rfloor_\tau \in \{0,1\}^\tau$ denotes the leading τ bits of x.

Sampling and probability. We write $x \leftarrow\!\$\ S$ for sampling x uniformly at random from a finite set S, with implicit coins independent across samples. For an event E in a probability experiment, $\Pr[E]$ denotes its probability.

Sets of maps. For a set S, $\mathsf{Perm}(S)$ denotes the set of permutations on S; for a positive integer b, $\mathsf{Perm}(\{0,1\}^b)$ is the set of b-bit permutations.

Games and advantages. We write $\mathcal{A}^{O_1,\ldots,O_k}$ for a (possibly randomized) adversary with oracle access to $O_1, \ldots, O_k$, and $\Pr[\mathsf{G}^{\mathcal{A}} \Rightarrow 1]$ for the probability that a game G—an experiment that runs $\mathcal{A}$ against specified oracles and returns a Boolean outcome—returns 1. For a distinguishing experiment between two games $\mathsf{G}_0, \mathsf{G}_1$, the advantage of $\mathcal{A}$ against a scheme Π in notion X is the absolute difference

$$\mathbf{Adv}^{\mathsf{X}}_{\Pi}(\mathcal{A}) = \left|\Pr[\mathsf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathsf{G}_1^{\mathcal{A}} \Rightarrow 1]\right|.$$

For a win-event experiment without a hidden challenge bit it is instead $\Pr[\mathsf{G}^{\mathcal{A}}\ \text{wins}]$, the probability the game returns 1; a bit-guessing experiment with a hidden challenge bit uses the rescaled form $\left|2\Pr[\mathsf{G}^{\mathcal{A}}\ \text{wins}] - 1\right|$, as in the multi-user notion of Section 2.4.

Concrete security. All bounds in this paper are concrete: each advantage is an explicit function of the resource caps introduced in Section 4.3. We make no use of asymptotic or negligibility conventions.

Reference summary. Table 1 summarizes the resource caps and loss terms used throughout. The caps are defined formally in Section 4.3, the loss terms in Section 5.2.

Table 1: Resource caps and loss terms.

Symbol	Meaning	Defined
N	Construction-interface cost (KECCAK-p[1600, 12] calls)	§ 4.3
N_{prim}	Direct primitive-interface queries	§ 4.3
N_{tot}	$N + N_{\mathsf{prim}}$ (total query-sequence cost)	§ 4.3
q_{v}, Q_{v}	Valid verification queries / cap (INT-CTXT)	§ 4.3
q_{RO}, Q_{RO}	Direct random-oracle queries / cap (RO-model)	§ 4.3
n_{leaf}, N_{leaf}	Distinct construction-generated final leaf transcripts / cap	§ 4.3
$\mathsf{Lift}_c(N_{\mathsf{tot}})$	$N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}$, [BDPVA08] flat-sponge loss	§ 5.2
$\mathsf{KeyGuess}_k(Q)$	$Q/2^k$, ideal-system key guess	§ 5.2
$\mathsf{RootColl}_\tau(Q,s)$	$\binom{Q+s}{s}/2^{\tau(s-1)}$, s-way root-collision	§ 5.2
$\mathsf{RootPre}_\tau(Q)$	$(Q+1)/2^\tau$, root-preimage	§ 5.2

2.2 Sponges and Duplexes over Keccak-*p*

The Keccak-*p* permutation. The KECCAK-p family of permutations, introduced as part of the Keccak design and standardized within FIPS 202 [Nat15], is parameterized by a state width b and a round count n_r; we write $\text{KECCAK}-p[b,n_r] \in \mathsf{Perm}(\{0,1\}^b)$ for the corresponding permutation. TW128 uses KECCAK-p[1600, 12], the same primitive as KangarooTwelve [BDP+16], fixed throughout. In the public-permutation security model of Section 4.1 and the flat-sponge lift of Section B, this permutation is modeled as a uniformly random element of $\mathsf{Perm}(\{0,1\}^{1600})$.

The sponge construction. Following [BDPVA08], fix a permutation $\pi \in \mathsf{Perm}(\{0,1\}^b)$, a *sponge-compliant* padding rule pad, and a rate $1 \le r < b$ with capacity $c = b - r$. A padding rule appends $\mathsf{pad}[r](|M|)$, a bit string determined by the input length and the rate, to an input M, extending it to a sequence of r-bit blocks; it is sponge-compliant [BDPVA11, Definition 1] if $M \,\|\, \mathsf{pad}[r](|M|)$ is never empty and, for all $M \neq M'$ and all $n \ge 0$, $M \,\|\, \mathsf{pad}[r](|M|) \neq M' \,\|\, \mathsf{pad}[r](|M'|) \,\|\, 0^{nr}$. The sponge function $\mathsf{Sponge}[\pi,\mathsf{pad},r] : \{0,1\}^* \times \mathbb{N} \to \{0,1\}^*$ maps an input M and an output length ℓ to an ℓ-bit string by absorbing $M \,\|\, \mathsf{pad}[r](|M|)$ into the all-zero b-bit state in r-bit blocks (each separated by an application of π) and squeezing ℓ bits in r-bit blocks from the resulting state. [BDPVA08, Theorem 2] shows that $\mathsf{Sponge}[\pi,\mathsf{pad10}^*,r]$, where the simple rule $\mathsf{pad10}^*$ appends a single 1 bit followed by the minimum number of 0 bits reaching a multiple of r, is indifferentiable from a random oracle when π is uniformly random. We bound its distinguishing advantage by $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}$, derived in Section 5.2 from the exact differentiating probabilities of [BDPVA08, Lemmas 4 and 5].

The duplex construction. The *duplex* construction of [BDPVA11] extends the sponge to a stateful interface. A duplex object $\mathsf{Duplex}[\pi,\mathsf{pad},r]$ is initialized to the all-zero state. Each call $\mathsf{duplexing}(\sigma,\ell)$, with $\ell \le r$, absorbs an input string σ of at most $\rho_{\max}(\mathsf{pad},r)$ bits via π and returns the first ℓ bits of the resulting state. Here $\rho_{\max}(\mathsf{pad},r) = \min\{x : x+|\mathsf{pad}[r](x)| > r\}-1$ is the largest input length for which the padded input fits in a single r-bit block; for the $\mathsf{pad10}^*1$ rule, $\rho_{\max}(\mathsf{pad10}^*1,r) = r-2$ [BDPVA11]. [BDPVA11, Lemma 3] establishes the duplex-to-sponge equality: for any call sequence $((\sigma_0,\ell_0),\ldots,(\sigma_i,\ell_i))$, the i-th output equals

$$\mathsf{Sponge}[\pi,\mathsf{pad},r]\big(\sigma_0 \,\|\, \mathsf{pad}_0 \,\|\, \cdots \,\|\, \sigma_i,\ \ell_i\big),$$

where $\mathsf{pad}_j = \mathsf{pad}[r](|\sigma_j|)$ is the padding the duplex appends in its j-th call; the flattened input carries these interior paddings explicitly, and the sponge appends only the final pad_i. The flattening map is injective by [BDPVA11, Lemma 4]: the duplex-call sequence is recoverable from the flattened sponge input.

TW128 instantiation. TW128 fixes the padding rule $\mathsf{pad10^*1}$ and the rate $r = 1344$, so the capacity is $c = 256$ bits. The largest input length per duplex call is therefore $\rho_{\max}(\mathsf{pad10^*1}, 1344) = 1342$ bits.

Bridge to the H-input domain. The indifferentiability theorem of [BDPVA08] is stated for the simpler $\mathsf{pad10^*}$ padding over its unpadded H-input domain, while TW128 uses $\mathsf{pad10^*1}$ internally. [BDPVA11, Theorem 3] supplies an injective preprocessing bridge $I[r, r_{\max}]$ between the two domains. For TW128, $r = r_{\max} = 1344$, so the bridge reduces to appending $1 \,\|\, 0^q$ with q determined by the input length modulo $r_{\max}$; on the bridge image, q is recovered as the trailing-zero count. Section 4.2 instantiates this bridge on typed TW128 duplex transcript prefixes as the map $\mathsf{flat}(\cdot)$ used by the random-oracle proofs of Sections C and 6 to 8.

2.3 Authenticated Encryption with Associated Data

Syntax. A *nonce-based authenticated-encryption scheme with associated data* (AEAD scheme) is a pair $\mathsf{SE} = (\mathsf{Enc}, \mathsf{Dec})$ of deterministic algorithms with associated key space $\mathcal{K} \subseteq (\{0,1\}^8)^*$, nonce space $\mathcal{N} \subseteq (\{0,1\}^8)^*$, associated-data space $\mathcal{A} \subseteq (\{0,1\}^8)^*$, message space $\mathcal{M} \subseteq (\{0,1\}^8)^*$, and ciphertext space $\mathcal{C} \subseteq (\{0,1\}^8)^*$. The encryption algorithm $\mathsf{Enc} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M} \to \mathcal{C}$ produces a ciphertext $C = \mathsf{Enc}_K(N, A, M)$. The decryption algorithm $\mathsf{Dec} : \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{C} \to \mathcal{M} \cup \{\perp\}$ returns either a plaintext or the rejection symbol $\perp$. The *tag length in bytes* is the constant $\|\mathsf{Enc}_K(N, A, M)\| - \|M\|$, fixed by the scheme. In this generic syntax, C denotes the full AEAD ciphertext, including any tag material. The TW128 algorithm section (Section 3) writes this same value as $C \,\|\, T$, with C the ciphertext body and T the root tag.

Correctness. For every $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$, $\mathsf{Dec}_K(N, A, \mathsf{Enc}_K(N, A, M)) = M$.

Privacy, integrity, and all-in-one AE. We use the standard nonce-based security notions, stated concretely for TW128 in Section 4.1. *Privacy* (IND-$-CPA) is the distinguishing advantage $\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{SE}}(\mathcal{A})$ between the real encryption oracle Enc_K and a random-output oracle Rand that returns a fresh uniform byte string of the ciphertext length on each query, over nonce-respecting adversaries. *Ciphertext integrity* (INT-CTXT) is the winning probability $\mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{SE}}(\mathcal{A})$ in the game that grants $\mathcal{A}$ the oracles Enc_K and a verification oracle Ver_K and declares a win when some verification query accepts a tuple (N, A, C) not produced by Enc_K; no nonce uniqueness is imposed. *All-in-one AE* [RS06], instantiated for nonce-based AE following [BT16], combines the two into a single experiment that grants either $(\mathsf{Enc}_K, \mathsf{Ver}_K)$ or the ideal pair $(\mathsf{Rand}, \mathsf{Replay})$, where Replay accepts exactly the tuples Rand produced, and defines $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{SE}}(\mathcal{A})$ as the distinguishing advantage between them over nonce-respecting adversaries.

Lemma 1 (Bellare–Namprempre Composition)**.** *For every adversary $\mathcal{A}$ against the all-in-one AE security of* SE*, there exist adversaries $\mathcal{A}_{priv}$ and $\mathcal{A}_{int\text{-}ctxt}$ with the same resources as $\mathcal{A}$ such that*

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{SE}}(\mathcal{A}) \le \mathbf{Adv}^{\mathsf{priv}}_{\mathsf{SE}}(\mathcal{A}_{priv}) + \mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{SE}}(\mathcal{A}_{int\text{-}ctxt}).$$

Lemma 1 is the all-in-one form of the privacy-plus-integrity composition theorem of [BN00]. The standard proof inserts the hybrid $(\mathsf{Enc}_K, \mathsf{Replay})$, where Replay accepts exactly the tuples recorded by the real encryption oracle. The hop from $(\mathsf{Enc}_K, \mathsf{Ver}_K)$ to $(\mathsf{Enc}_K, \mathsf{Replay})$ is bounded by INT-CTXT: the two interfaces differ only if verification accepts a tuple not previously recorded by encryption. The hop from $(\mathsf{Enc}_K, \mathsf{Replay})$ to $(\mathsf{Rand}, \mathsf{Replay})$ is bounded by privacy: $\mathcal{A}_{\mathrm{priv}}$ runs $\mathcal{A}$, forwards encryption queries to its own oracle, records the returned tuples, and answers verification queries by replay-table lookup. The lemma is applied in the random-oracle model in the proof of Theorem 1.

2.4 Multi-User Indistinguishability

The multi-user indistinguishability notion of [BT16, Fig. 3] is the multi-user lift of the single-user all-in-one notion of Section 2.3. The number of users u is adversary-determined: it is the number of New queries the adversary makes, i.e., the final value of the user counter v in the game below. A challenge bit $b \leftarrow\$ \{0,1\}$ selects between the real construction interface (real ciphertexts, real verification) and an ideal interface (uniform strings of matching length, verification that accepts only previously-issued encryption tuples). The adversary also accesses a public primitive interface: the real permutation (π, π^{-1}) in the real world and the simulator $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ in the ideal world. The adversary outputs a guess b' and wins if $b' = b$.

Game and advantage. The game $\mathsf{G}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A})$ of [BT16, Fig. 3] samples $b \leftarrow\$ \{0,1\}$ and maintains a user counter v, a set V of (d, N) pairs already used by encryption, and a set W of recorded encryption tuples; its New, Enc, and Vf oracles, together with the public primitive interface—the real permutation when $b = 1$ and the simulator when $b = 0$—are instantiated for $\mathsf{TW128}$ in Section 4.4. We write the user index as d and the associated-data input as A, where [BT16] writes i and H, and take the absolute value of their signed advantage so that

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A}) = \left|2\Pr[\mathsf{G}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}(\mathcal{A}) \text{ returns } 1] - 1\right| = \left|\Pr[\mathcal{A}^{\mathsf{Enc},\mathsf{Vf},b=1} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Enc},\mathsf{Vf},b=0} \Rightarrow 1]\right|,$$

the second equality holding because b is uniform. Thus $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{SE}}$ is the two-game distinguishing advantage between the real $(\mathsf{Enc}, \mathsf{Ver})$ pair and the ideal $(\mathsf{Rand}, \mathsf{Replay})$ pair instantiated per user. The single-user case $u = 1$ recovers the all-in-one AE notion of Section 2.3, augmented with the public primitive interface (the real permutation in the real world and the simulator in the ideal world).

Per-user nonce uniqueness. Nonce uniqueness is enforced per user via the set V: the same nonce may appear under distinct users but not twice under the same user. This is strictly weaker than the global nonce-uniqueness requirement that would force every (d, N) pair across all users to be unique.

2.5 Committing and Context-Discoverability Notions

[BH22] formalize a family of committing-security notions for nonce-based AEAD schemes, parameterized by how much of the encryption input is committed and by whether commitment is established through decryption (the D-notions) or through encryption agreement (the E-notions). We recall their definitions and the s-way generalization used throughout this paper, then recall the complementary *context-discoverability* notion of [MLGR23]. The two capture orthogonal facets of commitment: committing security is a collision requirement on the map from contexts to ciphertexts, while context discoverability is the corresponding preimage requirement.

What-is-committed functions. For $\ell \in \{1, 3, 4\}$, the *what-is-committed* function WiC_ℓ projects an encryption input $(K, N, A, M) \in \mathcal{K} \times \mathcal{N} \times \mathcal{A} \times \mathcal{M}$ to the portion it should commit:

$$\mathsf{WiC}_1(K, N, A, M) = K,$$
$$\mathsf{WiC}_3(K, N, A, M) = (K, N, A),$$
$$\mathsf{WiC}_4(K, N, A, M) = (K, N, A, M).$$

CMTDs-ℓ and CMTs-ℓ games. For $s \geq 2$, [BH22, Fig. 5] defines the s-way committing games on tuples $(K_i, N_i, A_i, M_i)_{i=1}^s$ with pairwise distinct WiC_ℓ-images, all entries in the scheme's syntactic domain and none equal to $\perp$. The s-way CMTDs-ℓ (decryption) game has the adversary output a ciphertext C alongside the tuples and returns 1 if $\mathsf{Dec}_{K_i}(N_i, A_i, C) = M_i$ for every i; the s-way CMTs-ℓ (encryption) game has the adversary output the tuples alone and returns 1 if all encryptions $\mathsf{Enc}_{K_i}(N_i, A_i, M_i)$ are equal. Advantages $\mathbf{Adv}^{\mathsf{CMTDs}\text{-}\ell}_{\mathsf{SE}}(\mathcal{A})$ and $\mathbf{Adv}^{\mathsf{CMTs}\text{-}\ell}_{\mathsf{SE}}(\mathcal{A})$ are the probabilities that the adversary wins the respective game; the parameter s is suppressed from the notation when clear from context. The two-tuple case $s = 2$ is the original CMTD-ℓ/CMT-ℓ pair of [BH22, Fig. 4]. These definitions are instantiated for TW128 in Section 4.1, with the public-permutation lift deferred to that section.

Tag-projected committing games. The [BH22] CMTD notions above treat the entire ciphertext as the committing object: an adversary wins only by exhibiting one common ciphertext that two distinct contexts both decrypt. Many uses of commitment store, transmit, or compare a short tag rather than the full ciphertext, and ask that the short string alone be binding—this is the compactly committing AEAD goal introduced by [GLR17] for message franking. To capture it we strengthen the D-notions through a public ciphertext projection. Let $\gamma : \mathcal{C} \to \mathcal{D}$ be such a projection. The s-way γ-CMTDs-ℓ game is the same as CMTDs-ℓ except that the adversary outputs ciphertexts $C_1, \ldots, C_s$ rather than one common ciphertext, and the challenger requires

$$\gamma(C_1) = \cdots = \gamma(C_s)$$

instead of requiring $C_1 = \cdots = C_s$. It then checks $\mathsf{Dec}_{K_i}(N_i, A_i, C_i) = M_i$ for every i, with the same syntax and pairwise-WiC_ℓ checks as above. Ordinary CMTDs-ℓ is the special case in which γ is the identity projection, so for any γ the projected game grants the adversary strictly more freedom—distinct openings need agree only on the projection—and an upper bound on the γ-CMTDs-ℓ advantage is therefore also an upper bound on the ordinary CMTDs-ℓ advantage, while additionally certifying $\gamma(C)$ as a binding commitment in its own right. For TW128, whose ciphertexts are written $C \parallel T$, the projection of interest keeps only the root tag,

$$\gamma_{\mathsf{tag}}(C \parallel T) = T,$$

and we call the resulting game tag-CMTDs-ℓ: the 256-bit tag T is the compact commitment string and the ciphertext bodies are opening data. Proving tag-CMTDs-4 (Theorem 3) establishes the binding, collision-resistant facet of this commitment for the full (K, N, A, M) encryption input, realizing the [GLR17] compactly committing goal natively rather than through an added commitment primitive; its preimage facet, which a cryptographic hash commitment equally requires, is supplied by the context discoverability of the next paragraph.

Relations. [BH22, Figs. 4 and 5] record the implications among the notions. For the D-notions we use:

$$\mathsf{CMTDs}\text{-}3 \;\Rightarrow\; \mathsf{CMTDs}\text{-}\ell \text{ for } \ell \in \{1, 4\}$$

(the $\ell = 1$ case is immediate since pairwise distinct keys imply pairwise distinct (K, N, A) triples; the $\ell = 4$ case uses deterministic decryption, as recalled in Section C). The E-notions are bounded directly in Section C by the same final-root candidate-collision argument, rather than through an encryption reduction.

Context discoverability. Where committing security forbids two contexts from colliding on one ciphertext, *context discoverability* [MLGR23] forbids an adversary from *finding* a decrypting context for a target ciphertext: it is to committing security what preimage resistance is to collision resistance. Following the query-independent context selector of [MLGR23], the discoverability game is given a target ciphertext fixed independently of the scheme's primitive and asks the adversary to output a decryption context under which that ciphertext decrypts without error. The notion is parameterized by which components of the context the selector fixes and which the adversary supplies; the unrestricted form, in which the adversary supplies the entire context, dominates the restricted variants of [MLGR23], in which the selector additionally pins two of the three context components and the adversary supplies only the remaining one. [MLGR23] show that schemes which delegate authenticity to a non-preimage-resistant MAC, including CCM, EAX, SIV, GCM, and OCB3, admit efficient discoverability attacks. The notion is instantiated for TW128 in Section 4.1; its preimage character is what makes TW128's root tag, a sponge image rather than an invertible MAC, resist these attacks. When the target ciphertext is itself projected through a public γ, the same experiment gives the projection analogue: the adversary is given a query-independent target value d^* and wins by finding a ciphertext C and context such that $\gamma(C) = d^*$ and C decrypts. The TW128 instantiation below is already the tag-projected case, fixing only the root tag T^* and leaving the ciphertext body free. It is the preimage facet of the same compact commitment whose collision facet is tag-CMTDs-4: together they make TW128's root tag preimage- as well as collision-resistant as a commitment to the encryption input, the two guarantees a cryptographic hash commitment provides.

3 The TW128 Construction

3.1 Design Overview

TW128 is a nonce-based authenticated-encryption scheme with associated data, built as a two-level tree of duplex objects over the Keccak-p[1600, 12] permutation. A single *root* duplex, initialized with the key K and nonce N, absorbs the associated data A when it is nonempty, emits keystream for the first plaintext chunk M_0 of at most β bytes, and absorbs the resulting ciphertext chunk C_0; when A is empty the associated-data phase is elided and the initialization call itself emits the initial M_0 keystream; the remaining plaintext is partitioned into *leaf* chunks $M_1, \ldots, M_n$, each handled by an independent leaf duplex initialized with (K, N, j). The leaf duplex emits the keystream for M_j, absorbs the resulting ciphertext chunk C_j, and produces a τ_{leaf}-bit leaf tag T_j. When $n \geq 1$ the leaf tags, together with the eight-byte little-endian leaf count $\langle n \rangle_8$, are absorbed into the root duplex, which emits a single τ_{root}-bit root tag T; when $n = 0$ the aggregation phase is elided and the closing message-phase call of the root chunk emits T directly, mirroring the way a leaf duplex emits its leaf tag. The construction is depicted in Figure 1.

Three design properties motivate the tree shape. First, the leaf duplexes operate on disjoint state and consume disjoint inputs once (K, N) is fixed, so leaves $1, \ldots, n$ can be evaluated in parallel across SIMD lanes; Section 9 quantifies the resulting throughput on AMD64 and ARM64 hardware. Second, every duplex operates on a bounded 1600-bit state regardless of message length, so the memory footprint is constant in $\|M\|$. Third, the explicit aggregation of leaf tags through a single deterministic root transcript gives the

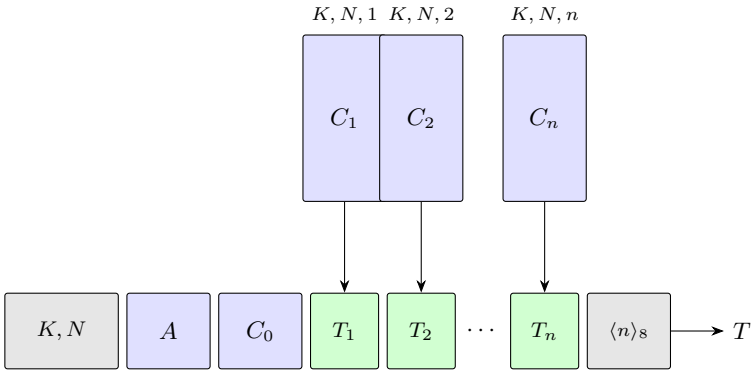

Figure 1: Schematic of TW128 for $n \geq 1$ leaves. The root duplex (bottom row) is initialized with (K, N) and absorbs in turn the associated data A, the root ciphertext chunk C_0, each leaf tag $T_1, \ldots, T_n$, and the eight-byte little-endian leaf count $\langle n \rangle_8$; it emits the root tag T. Each leaf duplex (top row) is initialized with (K, N, j), emits keystream for plaintext chunk M_j, absorbs the resulting ciphertext chunk C_j, and produces T_j. Leaves can be evaluated in parallel. The ciphertext chunks $C_0, C_1, \ldots, C_n$ are obtained by XORing the keystream emitted by the corresponding duplex with the plaintext chunks $M_0, M_1, \ldots, M_n$. When $n = 0$ the diagram reduces to the bottom row without any T_j boxes or aggregation block: the aggregation phase is elided and the closing message-phase call on C_0 emits the root tag T directly. When the associated data is empty the A block is absent: the associated-data phase is elided and the initialization call emits the initial M_0 keystream directly.

construction its commitment-security guarantees: distinct (K, N, A) triples force distinct final root transcripts before aggregation values matter (Corollary 2 and § 8), the structural fact behind the CMTDs-4 headline of Section 5.1. The tag-projected strengthening keeps only the emitted root tag as the compact commitment string and treats the ciphertext body as opening data; distinct bodies can merge only through the leaf tag collision term of Theorem 3. The same root tag, being a sponge image rather than an invertible MAC, is preimage-resistant and so yields the context-discoverability headline (Theorem 4).

3.2 Parameters and Domain

Table 2 lists the user-visible and derived scheme parameters. The implementation uses the same byte length for the root tag and each leaf tag, but the proofs of Sections 7 and 8 keep τ_{root} and τ_{leaf} as separate symbols.

The bound $M_{\max} = 8183 \cdot 2^{64}$ bytes is the largest plaintext whose leaf index fits in $\langle \cdot \rangle_8$: writing $\mathrm{leaves}(M) = \max(0, \lceil \|M\|/\beta \rceil - 1)$, every supported plaintext satisfies $0 \leq \mathrm{leaves}(M) \leq 2^{64} - 1$, so leaf indices lie in $\{1, \ldots, 2^{64} - 1\}$. The associated-data bound $A_{\max}$ is a fixed scheme parameter taken equal to $M_{\max}$ unless stated otherwise. Inputs exceeding these bounds are outside the scheme domain; every game in Section 4 rejects domain-violating queries.

The derived per-call absorption bound ρ follows the [BDPVA11] analysis: with $r = 1344$ and $\mathsf{pad10^*1}$ padding, $\rho_{\max}(\mathsf{pad10^*1}, r) = r - 2 = 1342$ bits. The byte-aligned data field shares this allotment with the four-bit phase suffix of Section 3.4, so ρ is the largest integer satisfying $8\rho + 4 \leq 1342$, giving $\rho = 167$ data bytes (and $8\rho + 4 = 1340 \leq 1342$).

Table 2: TW128 parameters.

Parameter	Value	Description
k	256	Key length (bits)
ν	256	Nonce length (bits)
τ_{root}	256	Root tag length (bits)
τ_{leaf}	256	Leaf tag length (bits)
β	8183	Plaintext chunk size (bytes)
$M_{\max}$	$8183 \cdot 2^{64}$ bytes	Maximum plaintext length
$A_{\max}$	$8183 \cdot 2^{64}$ bytes	Maximum associated-data length
b	1600	Permutation state width (bits)
n_r	12	Permutation rounds
r	1344	Sponge rate (bits)
c	256	Sponge capacity (bits)
ρ	167	Maximum byte-aligned absorption per duplex call (bytes)

3.3 Parameter Choice Rationale

The four free parameters c, k, $\tau_{\mathsf{root}} = \tau_{\mathsf{leaf}}$, and β (the rate $r = b - c$ is then determined by the capacity) are chosen to give 128-bit single-key security against every term of the concrete bounds derived in Section 5.4 while keeping per-byte cost low. Specifically:

- $c = 256$. The dominant single-key term is the [BDPVA08] flat-sponge loss

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}} + 1)/2^{c+1},$$

  which reaches the 2^{-128} threshold at $N_{\mathsf{tot}} \approx 2^{(c-127)/2} \approx 2^{64.5}$ Keccak-p[1600, 12] calls; the headline cap of $N_{\mathsf{tot}} \le 2^{63}$ used in Section 5.5 keeps this term at most $2^{-131} + 2^{-194}$, and hence below 2^{-130}. Setting $c = 256$ thereby delivers 128-bit single-key security on this term. Under the mu-ind bound of Section 5.1 the per-user security margin degrades by $\log D$ bits in D users; the concrete D-indexed envelope appears in Section 5.5.

- $r = 1344$. With state width $b = 1600$ fixed by the Keccak-p[1600, 12] choice, $r = b - c = 1344$ is the largest rate compatible with the chosen capacity. The 12-round Keccak-p[1600, 12] permutation matches KangarooTwelve's choice [BDP+16], with a substantial cryptanalytic safety margin relative to the 24-round version used in SHA-3 [Nat15].

- $\tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$. Matching the capacity ensures that the per-tag collision terms in the authenticity and committing bounds (Section 5.4) are dominated by the capacity birthday rather than the tag length.

- $k = 256$. Matching the capacity ensures that the per-user key guess

$$\mathsf{KeyGuess}_k(N_{\mathsf{prim}}) = N_{\mathsf{prim}}/2^k$$

  remains below the capacity birthday at the target D of Section 5.5.

- $\beta = 8183$. The chunk size is set to $49\rho = 49 \cdot 167$ bytes, the largest multiple of the per-call absorption bound ρ not exceeding 2^{13} bytes. Taking β to be an exact multiple of ρ makes every full leaf split into exactly 49 fully utilized message duplex calls with no short final block, so a full leaf costs one initialization duplex call plus 49 message duplex calls. This amortizes the fixed per-leaf initialization cost over a large chunk while keeping the leaf state working set ($8183 < 2^{13}$ bytes) small enough for L1 cache; Section 9.3 reports the resulting cycles-per-byte figures.

3.4 Domain Separation

Distinct duplex instances and distinct phases within a duplex are kept separate at the transcript level by two mechanisms. First, the initialization strings carry a 6-byte ASCII prefix and, for leaves, an eight-byte little-endian chunk index:

$$p_{\mathsf{root}} = \texttt{"TW128R"} \in \{0,1\}^{48},$$
$$p_{\mathsf{leaf}} = \texttt{"TW128L"} \in \{0,1\}^{48}.$$

The root duplex's first absorbed string is $p_{\mathsf{root}} \parallel K \parallel N$, and the j-th leaf duplex's first absorbed string is $p_{\mathsf{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$. Together with the fixed k-bit and ν-bit widths of K and N, these prefixes separate root from leaf transcripts and distinguish leaves with different chunk indices.

Second, every byte-aligned absorption carries a four-bit phase suffix. We identify each suffix with the four-bit string given by its Table 3 value written least-significant-bit first, so that a duplex call absorbing a byte-aligned block σ under a phase suffix σ_{ph} feeds the duplex interface of Section 2.2 the input $\sigma \parallel \sigma_{\mathsf{ph}}$ before $\mathsf{pad10^*1}$ padding. The seven phase suffixes are summarized in Table 3; they correspond to initialization, non-final and final associated-data blocks, non-final and final message blocks, and non-final and final aggregation blocks. The seven values are pairwise distinct, so any two transcripts that differ in the phase of any block also differ in the flattened sponge input.

Table 3: Phase suffixes (4 bits each, appended LSB-first).

Name	Hex	Use
σ_{init}	`0xC`	End of initialization (the sole init call)
σ^{-}_{ad}	`0x9`	Non-final associated-data block
σ^{+}_{ad}	`0xD`	Final associated-data block
$\sigma^{-}_{\mathsf{msg}}$	`0xA`	Non-final message block
$\sigma^{+}_{\mathsf{msg}}$	`0xE`	Final message block
$\sigma^{-}_{\mathsf{agg}}$	`0xB`	Non-final aggregation block
$\sigma^{+}_{\mathsf{agg}}$	`0xF`	Final aggregation block (emits root tag)

The combined effect of the initialization prefixes and the phase suffixes is that the flattened sponge input of any TW128 duplex call uniquely determines the duplex instance, the leaf index when applicable, the phase, and the duplex-call position within that phase. Definition 1 (Well-formed TW128 transcript prefixes), Lemma 5 (Transcript Injectivity), and its structural corollaries formalize this parsing fact for the proofs of Sections C and 6 to 8.

3.5 Encryption

Algorithm 2 specifies $\mathsf{Enc}_K(N, A, M)$ in terms of two helper procedures, Absorb and StreamEnc, defined in Algorithm 1. Absorb drives a duplex through a sequence of blocks of a byte string with non-final σ^- and final σ^+ suffix tags, returning the output of the final call; the empty-input case yields a single empty block carrying the σ^+ tag. The streaming helpers use the same empty-input convention, so a zero-length chunk still performs one final message-phase duplex call and returns a zero-length body. Algorithm 2 invokes Absorb for the associated-data phase only when $A \neq \varepsilon$; when $A = \varepsilon$ the phase is elided and the root σ_{init}call emits the initial M_0 keystream directly. StreamEnc encrypts a plaintext byte string block by block, XORing each plaintext block with the current keystream, absorbing the resulting ciphertext block back into the duplex with the appropriate phase tag, and using the duplex's output as the next keystream. This is the duplex-level presentation of the overwrite mechanism of [BDPVA11, §6.2 and Algorithm 5]:

Algorithm 1 TW128 helper procedures.

procedure ABSORB(D, s, σ^-, σ^+, ℓ)
    Partition s into byte blocks $s_1, \ldots, s_t$ of exactly ρ bytes each, except that the last block may be shorter, with $t \geq 1$ (the empty case yields the single empty block $s_1 = \varepsilon$).
    for $i \leftarrow 1$ **to** $t - 1$ **do**
        $D.\mathsf{duplexing}(s_i \,\|\, \sigma^-, 0)$
    end for
    return $D.\mathsf{duplexing}(s_t \,\|\, \sigma^+, \ell)$
end procedure

procedure STREAMENC(D, m, Z, σ^-, σ^+, ℓ)
    Partition m into byte blocks $m_1, \ldots, m_t$ of exactly ρ bytes each, except that the last block may be shorter, with $t \geq 1$ (if $m = \varepsilon$, set $t = 1$ and $m_1 = \varepsilon$).
    for $i \leftarrow 1$ **to** $t - 1$ **do**
        $c_i \leftarrow m_i \oplus Z$
        $Z \leftarrow D.\mathsf{duplexing}(c_i \,\|\, \sigma^-, 8\|m_{i+1}\|)$
    end for
    $c_t \leftarrow m_t \oplus Z$
    $Z' \leftarrow D.\mathsf{duplexing}(c_t \,\|\, \sigma^+, \ell)$
    return $(c_1 \,\|\, \cdots \,\|\, c_t, Z')$
end procedure

when the current rate prefix emitted as keystream is Z, XOR-absorbing $m \oplus Z$ has the same update on the message bytes as overwriting that prefix with m, with the phase tag still carried as part of the duplex input. The decryption helper STREAMDEC is identical to STREAMENC with the roles of the plaintext and ciphertext blocks exchanged: it takes ciphertext blocks c_i as input, recovers each plaintext block as $m_i = c_i \oplus Z$, absorbs the same c_i under the same phase suffix, and returns $(m_1 \,\|\, \cdots \,\|\, m_t, Z')$; we omit its pseudocode. In both helpers, each duplex call absorbs a data block followed by its phase suffix—σ^- for a non-final block and σ^+ for the final block—and requests the output bits the caller consumes next: ℓ (possibly zero) on the final call, and the next block's keystream length on non-final calls. Each call is the duplex interface of Section 2.2 applied to the suffixed input of Section 3.4.

The encryption procedure (Algorithm 2) instantiates the root duplex and obtains the initial keystream for the root chunk M_0: when $A \neq \varepsilon$ it absorbs the associated data A, and when $A = \varepsilon$ it elides the associated-data phase and takes the keystream from the σ_{init}call itself. It then encrypts M_0 while absorbing the resulting ciphertext chunk C_0. When $n = 0$ the closing σ^+_{msg}call of this root chunk requests τ_{root} output bits, which are the root tag T, and the aggregation phase is elided. When $n \geq 1$ it independently encrypts each leaf chunk M_j while absorbing C_j, producing the leaf tag T_j; the leaf tags together with $\langle n \rangle_8$ are then absorbed into the root duplex, and the final σ^+_{agg}call emits the root tag T. The ciphertext is the concatenation $C_0 \,\|\, C_1 \,\|\, \cdots \,\|\, C_n$. Thus this section writes C for the ciphertext body and $C \,\|\, T$ for the full AEAD ciphertext returned by $\mathsf{Enc}_K(N, A, M)$.

The keystream bit length requested for the first message block of M_0 is $8 \min(\|M_0\|, \rho)$, eight times the size of the next message block. It is emitted by the closing σ^+_{ad}call when $A \neq \varepsilon$, and by the root σ_{init}call itself when $A = \varepsilon$ and the associated-data phase is elided. Each leaf's σ_{init}call likewise emits $8 \min(\|M_j\|, \rho)$ keystream bits for M_j, so the empty-associated-data root σ_{init}call plays exactly the role a leaf σ_{init}call already plays. When the corresponding chunk is empty, the requested keystream length is zero and the next message-phase duplex call absorbs the empty ciphertext block carrying σ^+_{msg}. The final σ^+_{msg}call in the root chunk requests τ_{root} output bits when $n = 0$, namely the root tag

Algorithm 2 $\mathsf{Enc}_K(N, A, M)$.

Require: $K \in \{0,1\}^k$, $N \in \{0,1\}^\nu$, A with $\|A\| \le A_{\max}$, M with $\|M\| \le M_{\max}$.
Ensure: $C \,\|\, T$ with $\|C\| = \|M\|$ and $T \in \{0,1\}^{\tau_{\mathsf{root}}}$.

$M_0 \leftarrow$ first $\min(\|M\|, \beta)$ bytes of M (possibly ε).
Partition the remaining suffix into chunks $M_1, \ldots, M_n$ of exactly β bytes each, except that the last chunk may be shorter but nonempty, so $n = \mathrm{leaves}(M)$.
$D_{\mathrm{root}} \leftarrow$ new Duplex
if $A = \varepsilon$ **then**
    $Z \leftarrow D_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \,\|\, K \,\|\, N \,\|\, \sigma_{\mathsf{init}}, 8\min(\|M_0\|, \rho))$
else
    $D_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \,\|\, K \,\|\, N \,\|\, \sigma_{\mathsf{init}}, 0)$
    $Z \leftarrow \textsc{Absorb}(D_{\mathrm{root}}, A, \sigma^-_{\mathsf{ad}}, \sigma^+_{\mathsf{ad}}, 8\min(\|M_0\|, \rho))$
end if
if $n = 0$ **then**
    $(C_0,\, T) \leftarrow \textsc{StreamEnc}(D_{\mathrm{root}}, M_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{root}})$
    return $C_0 \,\|\, T$
end if
$(C_0,\, \_) \leftarrow \textsc{StreamEnc}(D_{\mathrm{root}}, M_0, Z, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, 0)$
for $j \leftarrow 1$ **to** n **in parallel do**
    $D_j \leftarrow$ new Duplex
    $Z_j \leftarrow D_j.\mathsf{duplexing}(p_{\mathsf{leaf}} \,\|\, K \,\|\, N \,\|\, \langle j \rangle_8 \,\|\, \sigma_{\mathsf{init}}, 8\min(\|M_j\|, \rho))$
    $(C_j, T_j) \leftarrow \textsc{StreamEnc}(D_j, M_j, Z_j, \sigma^-_{\mathsf{msg}}, \sigma^+_{\mathsf{msg}}, \tau_{\mathsf{leaf}})$
end for
$G \leftarrow T_1 \,\|\, \cdots \,\|\, T_n$.
$T \leftarrow \textsc{Absorb}(D_{\mathrm{root}},\, G \,\|\, \langle n \rangle_8,\, \sigma^-_{\mathsf{agg}},\, \sigma^+_{\mathsf{agg}},\, \tau_{\mathsf{root}})$
return $C_0 \,\|\, C_1 \,\|\, \cdots \,\|\, C_n \,\|\, T$

T emitted by the elided-aggregation path, and zero output bits when $n \ge 1$ because the next duplex operation on the root is then the first aggregation block; the corresponding call on a leaf requests τ_{leaf} bits, namely the leaf tag. When $n \ge 1$ the final σ^+_{agg}call requests τ_{root} bits, which is the root tag T.

The leaf loop is annotated **in parallel** to make explicit that the n leaf computations are mutually independent; Section 9.1 discusses the SIMD realizations.

3.6 Decryption

Algorithm 3 specifies $\mathsf{Dec}_K(N, A, C \,\|\, T)$. The procedure mirrors Enc block for block: each ciphertext block c_i is absorbed into the appropriate duplex (root or leaf) under the corresponding σ^-_{msg}or σ^+_{msg}suffix, and the plaintext block is recovered as $m_i = c_i \oplus Z^{(i)}$, writing $Z^{(i)}$ for the value the running keystream variable Z of StreamDec holds when block i is processed ($i < t$ inside the loop, $i = t$ after it), the same value the encryption procedure would have produced. The procedure absorbs ciphertext bytes before the final tag comparison, so the verification transcript is determined by (K, N, A, C) for every in-domain ciphertext, accepting or not. This property is used by the authenticity proof of Section 7 to argue that encryption-first and verification-first transcript classes are well defined regardless of the tag-comparison outcome.

The $n = 0$ and empty-body cases mirror their encryption counterparts (Section 3.5): for $n = 0$ the aggregation phase is elided and the closing σ^+_{msg}call of the root chunk emits the candidate tag T' directly, and an empty body still performs the single empty σ^+_{msg}call of Algorithm 1, now requesting τ_{root} output bits.

Algorithm 3 $\mathsf{Dec}_K(N, A, C \,\|\, T)$.

Require: K, N, A as in Algorithm 2, ciphertext C with $\|C\| \le M_{\max}$, candidate tag $T \in \{0,1\}^{\tau_{\mathsf{root}}}$.
Ensure: M with $\|M\| = \|C\|$, or $\perp$.

$C_0 \leftarrow$ first $\min(\|C\|, \beta)$ bytes of C (possibly ε).
Partition the remaining suffix into chunks $C_1, \ldots, C_n$ of exactly β bytes each, except that the last chunk may be shorter but nonempty, so $n = \mathrm{leaves}(C)$.
$D_{\mathrm{root}} \leftarrow$ new Duplex
if $A = \varepsilon$ **then**
    $Z \leftarrow D_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \,\|\, K \,\|\, N \,\|\, \sigma_{\mathsf{init}}, 8\min(\|C_0\|, \rho))$
else
    $D_{\mathrm{root}}.\mathsf{duplexing}(p_{\mathsf{root}} \,\|\, K \,\|\, N \,\|\, \sigma_{\mathsf{init}}, 0)$
    $Z \leftarrow \textsc{Absorb}(D_{\mathrm{root}}, A, \sigma^{-}_{\mathsf{ad}}, \sigma^{+}_{\mathsf{ad}}, 8\min(\|C_0\|, \rho))$
end if
if $n = 0$ **then**
    $(M_0,\, T') \leftarrow \textsc{StreamDec}(D_{\mathrm{root}}, C_0, Z, \sigma^{-}_{\mathsf{msg}}, \sigma^{+}_{\mathsf{msg}}, \tau_{\mathsf{root}})$
    return M_0 if $T' = T$ (compared in constant time), else $\perp$.
end if
$(M_0,\, \_) \leftarrow \textsc{StreamDec}(D_{\mathrm{root}}, C_0, Z, \sigma^{-}_{\mathsf{msg}}, \sigma^{+}_{\mathsf{msg}}, 0)$
for $j \leftarrow 1$ **to** n **in parallel do**
    $D_j \leftarrow$ new Duplex
    $Z_j \leftarrow D_j.\mathsf{duplexing}(p_{\mathsf{leaf}} \,\|\, K \,\|\, N \,\|\, \langle j \rangle_8 \,\|\, \sigma_{\mathsf{init}}, 8\min(\|C_j\|, \rho))$
    $(M_j, T_j) \leftarrow \textsc{StreamDec}(D_j, C_j, Z_j, \sigma^{-}_{\mathsf{msg}}, \sigma^{+}_{\mathsf{msg}}, \tau_{\mathsf{leaf}})$
end for
$G \leftarrow T_1 \,\|\, \cdots \,\|\, T_n$.
$T' \leftarrow \textsc{Absorb}(D_{\mathrm{root}},\, G \,\|\, \langle n \rangle_8,\, \sigma^{-}_{\mathsf{agg}},\, \sigma^{+}_{\mathsf{agg}},\, \tau_{\mathsf{root}})$
if $T' = T$ (compared in constant time) **then**
    return $M_0 \,\|\, M_1 \,\|\, \cdots \,\|\, M_n$
else
    return $\perp$
end if

Correctness. For every $K \in \{0,1\}^k$, $N \in \{0,1\}^\nu$, A with $\|A\| \le A_{\max}$, and M with $\|M\| \le M_{\max}$, $\mathsf{Dec}_K(N, A, \mathsf{Enc}_K(N, A, M)) = M$. This follows by inspection: every duplex call in Dec_K receives the same input as the corresponding call in Enc_K (each ciphertext block is absorbed under the same phase tag in both directions), so the duplex states and the recomputed root tag T' exactly match. The constant-time comparison $T' = T$ succeeds, and the recovered $m_i = c_i \oplus Z^{(i)} = (m_i \oplus Z^{(i)}) \oplus Z^{(i)} = m_i$ at every position.

4 Security Model

This section fixes the security model used throughout the paper. The scheme syntax and the authenticated-encryption and committing-security definitions of Section 2 are instantiated here for TW128 in the lifted public-permutation model (Section 4.1), and we define the intermediate random-oracle world through which every bound is first proved (Section 4.2). Section 4.3 fixes the resource accounting that all later bounds reference, and Section 4.4 instantiates the multi-user indistinguishability game for TW128.

4.1 Public-Permutation Games

The lifted AE-style public-permutation advantages below compare a real execution, with sampled $\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600})$ and direct primitive interface (π, π^{-1}), to a simulator-ideal execution. In the simulator-ideal execution, the construction interface is the ideal interface specified by the game and the public primitive interface is $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$. The simulator $\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}$ together with its inverse $\mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}}$ is the public-primitive simulator fixed in Section B; both directions share an internal lazy table for the random oracle $\mathsf{RO}_{\mathsf{flat}}$ of Section 4.2. INT-CTXT and the committing-security games remain one-world real public-permutation success probabilities; the simulator appears for them only inside the reductions of Section B.

In the real authenticated-encryption games (privacy, INT-CTXT, all-in-one AE), the challenger additionally samples a key $K \leftarrow\$ \{0,1\}^k$ and exposes the construction oracles $\mathsf{Enc}_K[\pi]$ (the TW128 encryption algorithm of Algorithm 2 with the sampled π) and $\mathsf{Ver}_K[\pi]$, defined by $\mathsf{Ver}_K[\pi](N, A, C, T) = 1$ if $\mathsf{Dec}_K[\pi](N, A, C \,\|\, T) \neq \perp$ and 0 otherwise. The committing-security games have no challenger-sampled key.

Validity convention. All construction-interface inputs are checked against TW128's public syntax and domain before any construction computation or construction-internal primitive lookup is performed. An encryption input is valid when $N \in \{0,1\}^\nu$, A is a byte string with $\|A\| \le A_{\max}$, and M is a byte string with $\|M\| \le M_{\max}$. A verification or decryption input is valid when $N \in \{0,1\}^\nu$, A is a byte string with $\|A\| \le A_{\max}$, and the full ciphertext parses as $C \,\|\, T$ with $\|C\| \le M_{\max}$ and $T \in \{0,1\}^{\tau_{\mathsf{root}}}$. In the committing games, each submitted tuple must be a valid encryption input including $K \in \{0,1\}^k$, and a submitted CMTDs ciphertext must be a valid verification/decryption ciphertext; in the tag-projected committing game below, each submitted body joined to the common tag must be a valid verification/decryption ciphertext. Invalid encryption queries are rejected and not recorded, invalid verification queries return 0, and invalid committing-game outputs make the game return 0. Such rejected inputs contribute no construction cost and create no construction transcripts. Throughout, an *accepted* encryption query is one that passes this validity check and any game-specific nonce or user check.

Privacy. The privacy game compares the real encryption interface with the fresh output oracle Rand that, on every accepted query (N, A, M), returns a uniformly random byte string of length $\|M\| + \tau_{\mathsf{root}}/8$. The adversary is *nonce-respecting*: no two encryption queries may share a nonce, and a query that repeats an earlier nonce is rejected and not recorded.

$$\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{TW128}}(\mathcal{A}) = \left|\Pr[\mathcal{A}^{\mathsf{Enc}_K[\pi],\,\pi,\,\pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Rand},\,\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}},\,\mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}}} \Rightarrow 1]\right|.$$

Ciphertext integrity (INT-CTXT). The INT-CTXT game exposes encryption, verification, and the public permutation interfaces π, π^{-1}. Each encryption query records the resulting tuple (N, A, C, T). The adversary wins if some verification query returns 1 on a tuple (N, A, C, T) that was not previously recorded by the encryption oracle.

$$\mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{TW128}}(\mathcal{A}) = \Pr[\mathcal{A}^{\mathsf{Enc}_K[\pi],\,\mathsf{Ver}_K[\pi],\,\pi,\,\pi^{-1}} \text{ wins}].$$

No nonce uniqueness is imposed.

All-in-one AE. The all-in-one authenticated-encryption game [RS06] replaces the construction interface by the ideal pair (Rand, Replay). On each accepted encryption query (N, A, M), Rand samples a uniformly random byte string of length $\|M\| + \tau_{\mathsf{root}}/8$, parses it as $C \,\|\, T$ with $T \in \{0,1\}^{\tau_{\mathsf{root}}}$, records (N, A, C, T), and returns $C \,\|\, T$. The oracle

Replay returns 1 exactly on recorded tuples (N, A, C, T) and 0 otherwise. The adversary is nonce-respecting on encryption queries.

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) = \big|\Pr[\mathcal{A}^{\mathsf{Enc}_K[\pi],\,\mathsf{Ver}_K[\pi],\,\pi,\,\pi^{-1}} \Rightarrow 1] - \Pr[\mathcal{A}^{\mathsf{Rand},\,\mathsf{Replay},\,\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}},\,\mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}}} \Rightarrow 1]\big|.$$

This lifted all-in-one AE advantage is the headline target of Section 5.1: a Bellare–Namprempre composition (Lemma 1) applied in the random-oracle model to derived privacy and INT-CTXT adversaries, combined with the flat-sponge lift, bounds it via the privacy and INT-CTXT envelopes of Section 5.3; the precise bound is Theorem 1.

Committing security. The s-way committing-security game follows [BH22, Figs. 4 and 5], extended to $s \geq 2$ tuples. There is no challenger-sampled key. After the primitive sampling, the adversary outputs a ciphertext $C \,\|\, T$ together with s tuples $(K_i, N_i, A_i, M_i)_{i=1}^{s}$. The challenger applies the following deterministic checks, with the eager length rejection justified by Algorithm 3: $\mathsf{Dec}_K(N, A, C \,\|\, T)$ returns either $\perp$ or a message of length $\|C\|$.

$\pi \leftarrow\!\$\ \mathsf{Perm}(\{0,1\}^{1600})$,
$\big(C \,\|\, T,\ (K_i, N_i, A_i, M_i)_{i=1}^{s}\big) \leftarrow\!\$\ \mathcal{A}^{\pi,\,\pi^{-1}}$,
return 0 if $C \,\|\, T$ is not a valid $\mathsf{TW128}$ ciphertext,
return 0 if any (K_i, N_i, A_i, M_i) is not in the $\mathsf{TW128}$ syntactic domain,
return 0 if any $\|M_i\| \neq \|C\|$,
return 0 if $(K_i, N_i, A_i)_{i=1}^{s}$ are not pairwise distinct,
return $\bigwedge_{i=1}^{s} \big[\mathsf{Dec}_{K_i}[\pi](N_i, A_i, C \,\|\, T) = M_i\big]$.

The advantage is

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathsf{TW128}}(\mathcal{A}) = \Pr[\text{the game returns 1}],$$

with the probability taken over the uniform choice of π and $\mathcal{A}$'s coins. No nonce uniqueness is imposed. For $\ell \in \{1, 4\}$, the CMTDs-ℓ game is identical except that the pairwise-distinctness check is applied to the WiC_ℓ-images—the keys K_i for $\ell = 1$ and the full tuples (K_i, N_i, A_i, M_i) for $\ell = 4$—with the same check order and the same eager length rejection; $\mathbf{Adv}^{\mathsf{CMTDs\text{-}}\ell}_{\mathsf{TW128}}(\mathcal{A})$ is defined analogously. The CMTs-ℓ source games are instantiated analogously from Section 2.5: after the primitive sampling, the adversary outputs s tuples, the challenger checks syntax and pairwise WiC_ℓ-distinctness, and then returns 1 iff the s deterministic encryptions $\mathsf{Enc}_{K_i}[\pi](N_i, A_i, M_i)$ are all equal.

The tag-projected full-input committing game instantiates Section 2.5 with $\gamma = \gamma_{\mathsf{tag}}$. After primitive sampling, the adversary outputs one candidate tag and one ciphertext body for each claimed opening:

$\pi \leftarrow\!\$\ \mathsf{Perm}(\{0,1\}^{1600})$,
$\big(T,\ (C_i, K_i, N_i, A_i, M_i)_{i=1}^{s}\big) \leftarrow\!\$\ \mathcal{A}^{\pi,\,\pi^{-1}}$,
return 0 if any $C_i \,\|\, T$ is not a valid $\mathsf{TW128}$ ciphertext,
return 0 if any (K_i, N_i, A_i, M_i) is not in the $\mathsf{TW128}$ syntactic domain,
return 0 if any $\|M_i\| \neq \|C_i\|$,
return 0 if $(K_i, N_i, A_i, M_i)_{i=1}^{s}$ are not pairwise distinct,
return $\bigwedge_{i=1}^{s} \big[\mathsf{Dec}_{K_i}[\pi](N_i, A_i, C_i \,\|\, T) = M_i\big]$.

The advantage is $\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A})$. The ordinary CMTDs-4 game is the special case in which all C_i are equal. Section C promotes the CMTDs-3 bound proved in Section 8 (Theorem 7) to the headline CMTDs-4 statement of Section 5.1 and proves matching direct bounds for the CMTs-ℓ source games; Section 8 proves the tag-projected CMTDs-4 bound directly.

Context discoverability. The context-discoverability game is the preimage analogue of the committing game, following the query-independent context selector of [MLGR23]. It is parameterized by a target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ fixed before the primitive sampling, hence independent of π and of the adversary's queries. There is no challenger-sampled key. After the primitive sampling, the adversary outputs a ciphertext body C together with a single decryption context (K, N, A), and the challenger runs the one deterministic decryption check on the assembled ciphertext $C \parallel T^*$.

$\pi \leftarrow\!\$\ \mathsf{Perm}(\{0,1\}^{1600})$,
$\big(C,\ (K,N,A)\big) \leftarrow\!\$\ \mathcal{A}^{\pi,\pi^{-1}}(T^*)$,
return 0 if $C \parallel T^*$ is not a valid TW128 ciphertext,
return 0 if (K,N,A) is not in the TW128 syntactic domain,
return $\big[\mathsf{Dec}_K[\pi](N,A,C \parallel T^*) \neq \perp\big]$.

The advantage is the worst case over query-independent targets,

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathsf{TW128}}(\mathcal{A}) = \max_{T^* \in \{0,1\}^{\tau_{\mathsf{root}}}} \Pr[\text{the game returns 1}],$$

with each probability taken over the uniform choice of π and $\mathcal{A}$'s coins. By Algorithm 3, the single check accepts exactly when the recomputed final root tag of (K, N, A, C) equals T^*, so the game asks the adversary to discover a context whose ciphertext body extends to the fixed tag T^*; the win condition makes no reference to a claimed message, so the committing game's eager length and pairwise-distinctness checks are absent. Fixing only the tag while leaving (K, N, A) and C free makes this the maximal-freedom discovery target: the restricted notions of [MLGR23], which additionally pin two of the three context components, are special cases, so the headline bound (Theorem 4) dominates each of them. Section B lifts the CDY bound proved in Section 8 (Theorem 9) to the public-permutation statement of Section 5.1.

4.2 The TW128 Random-Oracle Model

The proofs of Sections 6 to 8 and Section C, together with the headline reductions of Section 5.1, are carried out through an intermediate random-oracle model whose construction interfaces use the [BDPVA11] bridge of Section 2.2 to address a single random oracle. The flat-sponge lift of Section B then converts the resulting bounds to the public-permutation form.

The model has a single random oracle $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ over the unpadded H-input domain of the [BDPVA08] simple-padded sponge interface, with lazy sampling: each distinct input receives an independent infinite bit string, and repeated queries return the same string. A direct adversary query is finite-output: on input (Y, ℓ) with finite ℓ, the oracle returns the first ℓ bits of the lazy-sampled value for Y.

For a well-formed TW128 duplex transcript prefix X in the language of Definition 1, at one of the construction transcript lookup points formalized by Lemma 6 and Table 8, let $\mathsf{flat}(X) = I[1344, 1344](\mathsf{raw\_flat}(X))$ be the bridged image of the native flattened input under the [BDPVA11, Theorem 3] preprocessing map of Section 2.2, and define

$$\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X)).$$

The construction interfaces $\mathsf{Enc}^{\mathsf{RO}}_K$, $\mathsf{Dec}^{\mathsf{RO}}_K$, and $\mathsf{Ver}^{\mathsf{RO}}_K$ are the TW128 algorithms of Section 3 with every root or leaf duplex output replaced by the corresponding requested projection of $\mathsf{RF}(X)$; the projection length is the duplexing call's output parameter ℓ, which may be zero. The bridge and the typed-restriction convention are formalized in Lemma 6;

the direct-query key-hit predicate ignores the adversary's requested output length and is formalized as $\mathsf{KeyedTWOut}$ in Lemma 6.

The random-oracle-model games are the construction-interface variants of Sections 4.1 and 4.4: the primitive interface is replaced by direct adversary access to $\mathsf{RO}_{\mathsf{flat}}$, and each construction oracle uses the corresponding $^{\mathsf{RO}}$ algorithm, the ideal oracles $(\mathsf{Rand}, \mathsf{Replay})$ being unchanged. Under this substitution $\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{RO}}$, $\mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}$, and $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{RO}}$ are the Section 4.1 advantages with no simulator on the ideal side; the keyless $\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathsf{RO}}(\mathcal{B})$, $\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B})$, and, for $\ell \in \{1, 3, 4\}$, $\mathbf{Adv}^{\mathsf{CMTs}\text{-}\ell}_{\mathsf{RO}}(\mathcal{B})$ are the probabilities that the corresponding random-oracle committing games return 1, the CMTs challenger computing its deterministic encryption checks with $\mathsf{Enc}^{\mathsf{RO}}$. For mu-ind, $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B})$ denotes the Section 4.4 game with the same New interface and per-user nonce discipline—user d using $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ when $b = 1$ and $(\mathsf{Rand}, \mathsf{Replay})$ when $b = 0$—equivalently, in the two-game notation of Section 2.4,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) = \left|\Pr[\mathcal{B}^{\mathsf{Real}^{\mathsf{mu}}_{\mathsf{RO}}, \mathsf{RO}_{\mathsf{flat}}} \Rightarrow 1] - \Pr[\mathcal{B}^{\mathsf{Ideal}^{\mathsf{mu}}, \mathsf{RO}_{\mathsf{flat}}} \Rightarrow 1]\right|,$$

where the superscripts name the entire multi-user construction interface in the corresponding world. Adversary queries to $\mathsf{RO}_{\mathsf{flat}}$ are finite-output and counted in $q_{\mathsf{RO}}(\mathcal{B})$ of Section 4.3. Construction-internal $\mathsf{RF}(\cdot)$ lookups — those made by $\mathsf{Enc}^{\mathsf{RO}}_K$, $\mathsf{Dec}^{\mathsf{RO}}_K$, $\mathsf{Ver}^{\mathsf{RO}}_K$, or a committing challenger — are not.

4.3 Resources

Adversary resources are measured by fixed, execution-uniform caps on the number and size of oracle queries. An execution-uniform cap is one that holds on every execution path, including over adversary randomness, oracle randomness, and adaptive query choices. Proofs may also introduce local execution-uniform caps, such as q_e for the number of accepted encryption queries, solely to index a finite hybrid sequence. Such local caps are part of the adversary's fixed query bounds, but they are not listed as envelope parameters unless they affect the final bound.

N: the construction-interface cost, measured as the number of Keccak-p[1600, 12] calls that the adversary's construction-interface queries induce in the real $\mathsf{TW128}$ computation after the validity convention above has been applied. In ideal games, the same accounting applies to the corresponding real $\mathsf{TW128}$ computation on the valid query, even when an ideal oracle answers by table lookup or without running the construction. N counts at per-block granularity, summing each root initialization, associated-data block, root message block, leaf initialization, leaf message block, and aggregation block across the entire query sequence. Valid INT-CTXT verification computations are counted whether they accept or reject. In the committing and discoverability games there is no adversary construction oracle: for CMTDs-ℓ, N counts only the deterministic decryption checks actually performed by the challenger on the common ciphertext; for tag-CMTDs-4, N counts only the deterministic decryption checks actually performed by the challenger on the submitted bodies and common tag; for the context-discoverability game, N counts only the single deterministic decryption check the challenger performs on $C \parallel T^*$; while for the CMTs-ℓ source games of Section C, N counts only the deterministic encryption checks actually used by the challenger to compare the s ciphertexts.

N_{prim}: the number of direct adversary queries to the primitive interface (the pair (π, π^{-1}) in public-permutation games; the simulator's $\mathsf{RO}_{\mathsf{flat}}$ table is internal and is not exposed to the application adversary in those games).

$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$: the total query-sequence cost. The flat-sponge lift of Section B is applied at N_{tot} because the indifferentiability replacement sees both the construction-internal sponge calls counted by N and the adversary's direct primitive-interface queries counted by N_{prim} in one chronological public-primitive transcript.

$q_{\mathsf{v}}(\mathcal{A})$, Q_{v}: the number of verification queries with valid inputs, and a cap on it. Used in INT-CTXT and absorbed into the AE composition of Section 5.1. Every valid verification query contributes at least one verification/decryption construction call in the N accounting, so $q_{\mathsf{v}}(\mathcal{A}) \le N$.

$q_{\mathsf{RO}}(\mathcal{B})$, Q_{RO}: the number of direct finite-output queries to $\mathsf{RO}_{\mathsf{flat}}$ by a random-oracle-model adversary $\mathcal{B}$, and a cap on it. Repeated direct queries on the same Y are counted again. Construction-internal $\mathsf{RF}(\cdot)$ lookups are not counted. After the derived-adversary construction, q_{RO} counts only simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ calls from the adversary's direct primitive-interface queries, not the construction calls already counted in N for the flat-sponge lift.

$n_{\mathsf{leaf}}(\mathcal{B})$, N_{leaf}: the number of distinct construction-generated final leaf transcripts in the execution of $\mathcal{B}$, and a cap on it. Distinct final leaf transcripts are generated by valid encryption computations, by valid INT-CTXT verification computations (whether they accept or reject), and by committing, tag-projected committing, and discoverability challenger construction checks that pass the early validity checks.

For a byte string X,

$$\mathrm{leaves}(X) = \max\bigl(0,\ \lceil \|X\|/\beta \rceil - 1\bigr)$$

counts the leaf chunks induced by a body byte string of length $\|X\|$. Thus n_{leaf} is bounded by the sum of $\mathrm{leaves}(X)$ over the valid construction computations that create final leaf transcripts, where X is the plaintext body for encryption computations and the submitted ciphertext body for verification and committing or discoverability decryption checks. Each distinct final leaf transcript contains a final leaf duplex call counted in N, so $n_{\mathsf{leaf}}(\mathcal{B}) \le N$ unconditionally. The ratio n_{leaf}/N is largest when most processed bytes lie in leaf chunks, where each full leaf contributes $1 + \beta/\rho = 50$ duplex calls to N per final leaf transcript.

All theorems are stated for adversaries whose execution-uniform caps hold at the stated bounds.

4.4 Multi-User Setting

This subsection instantiates the mu-ind game of [BT16] (Section 2.4) for $\mathsf{TW128}$ in the lifted public-permutation model, using the same ideal-side simulator convention as the single-user AE-style games of Section 4.1. The challenger samples a challenge bit $b \leftarrow\$ \{0,1\}$. If $b = 1$, it samples a permutation $\pi \leftarrow\$ \mathsf{Perm}(\{0,1\}^{1600})$ and exposes (π, π^{-1}) as the primitive interface; if $b = 0$, it exposes $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ as the primitive interface. The adversary $\mathcal{A}$ creates users on demand via New, and the game maintains an active-user counter v. We write D for an execution-uniform cap on the number of New calls. Equivalently, the game may sample independent keys $K_1, \ldots, K_D$ at the start of the experiment and let the d-th New call activate K_d; inactive indices are never valid users.

The mu-ind construction oracles inherit the validity convention of Section 4.1. The encryption oracle on input (d, N, M, A) rejects, without recording, if (N, A, M) is outside the $\mathsf{TW128}$ encryption domain, if $d \notin \{1, \ldots, v\}$, or if $(d, N) \in V$; otherwise it sets $C = \mathsf{Enc}_{K_d}[\pi](N, A, M)$ when $b = 1$, and samples $C \leftarrow\$ \{0,1\}^{8\|M\|+\tau_{\mathsf{root}}}$ when $b = 0$. The game records (d, N) in V and (d, N, C, A) in W, and returns C. The verification oracle on input (d, N, C, A) rejects if $d \notin \{1, \ldots, v\}$ or if (N, A, C) is not a valid $\mathsf{TW128}$ verification/decryption input; if $(d, N, C, A) \in W$ it returns true; if $b = 0$ it returns false;

otherwise it returns $[\mathsf{Dec}_{K_d}[\pi](N, A, C) \neq \perp]$. Nonce uniqueness is enforced per user by the set V.

After querying these oracles, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. The mu-ind advantage of $\mathcal{A}$ against TW128 is

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) = |2\Pr[b' = b] - 1|.$$

Resource caps split per potential user $d \leq D$: $N^{(d)}$, $q_{\mathsf{v}}^{(d)}$, $n_{\mathsf{leaf}}^{(d)}$ are the per-user counts under Section 4.3, with inactive users contributing zero. The global totals $N = \sum_{d=1}^{D} N^{(d)}$, $q_{\mathsf{v}} = \sum_{d=1}^{D} q_{\mathsf{v}}^{(d)}$, $n_{\mathsf{leaf}} = \sum_{d=1}^{D} n_{\mathsf{leaf}}^{(d)}$ appear in the final bound: N through the lift term $\mathsf{Lift}_c(N_{\mathsf{tot}})$, and n_{leaf} and q_{v} through the leaf-collision and verification-tag terms. N_{prim} remains a single shared count over the primitive interface.

The mu-ind treatment applies only to indistinguishability. The committing-security games already give the adversary control of all s keys, so the s-way CMTDs-ℓ statements of Section 4.1 are themselves the multi-key committing statement and need no separate mu-cmt notion.

5 Main Results

5.1 Headline Theorems

TW128 delivers four standard headline guarantees in the lifted public-permutation model of Section 4.1: all-in-one AE security [RS06], its multi-user indistinguishability lift [BT16], and the two facets of commitment security—s-way CMTDs-4 committing security [BH22] and context discoverability [MLGR23]. Committing security is the collision facet and context discoverability the preimage facet; both follow from the same root tag and are stated co-equally. We also state tag-projected strengthenings showing that the root tag alone is the compact commitment target. For the AE-style notions the ideal side exposes the BDPV simulator; the committing and discoverability bounds remain real public-permutation success probabilities. Theorems 1 to 4 and Corollary 1 state these bounds; the loss terms Lift_c, $\mathsf{KeyGuess}_k$, $\mathsf{RootColl}_\tau$, $\mathsf{LeafColl}_\tau$, and $\mathsf{RootPre}_\tau$ are unpacked in Section 5.2, and the random-oracle envelopes $\varepsilon^{\mathsf{priv}}_{\mathsf{RO}}$, $\varepsilon^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}$ in Section 5.3.

Theorem 1 (All-in-One AE Security)**.** *For every nonce-respecting AE adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \varepsilon^{\mathsf{priv}}_{\mathsf{RO}}(N_{\mathsf{prim}}) + \varepsilon^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(N_{\mathsf{prim}}, N_{\mathsf{leaf}}, Q_{\mathsf{v}}).$$

Proof. By the Lift Application corollary (Corollary 4) for the all-in-one AE game, there is a random-oracle AE adversary $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ with $q_{\mathsf{RO}}(\mathcal{B}) \leq N_{\mathsf{prim}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \leq N_{\mathsf{leaf}}$, and $q_{\mathsf{v}}(\mathcal{B}) \leq Q_{\mathsf{v}}$ such that

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{ae}}_{\mathsf{RO}}(\mathcal{B}).$$

The all-in-one composition lemma (Lemma 1), applied in the random-oracle model, gives

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathbf{Adv}^{\mathsf{priv}}_{\mathsf{RO}}(\mathcal{B}_{\mathrm{priv}}) + \mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(\mathcal{B}_{\mathrm{int\text{-}ctxt}}),$$

for derived adversaries $\mathcal{B}_{\mathrm{priv}}$ (which runs $\mathcal{B}$, records the outputs of its encryption-oracle queries, and answers each verification query by replay-table lookup) and $\mathcal{B}_{\mathrm{int\text{-}ctxt}}$ (which runs $\mathcal{B}$, forwards encryption and verification queries to its own oracles, and halts on the first accepting non-replay verification response), both inheriting $\mathcal{B}$'s $(q_{\mathsf{RO}}, n_{\mathsf{leaf}}, q_{\mathsf{v}})$. Because $\mathcal{B}_{\mathrm{priv}}$ forwards $\mathcal{B}$'s encryption queries unchanged and $\mathcal{B}$ inherits the nonce-respecting discipline of $\mathcal{A}$, the adversary $\mathcal{B}_{\mathrm{priv}}$ is itself nonce-respecting, so the privacy envelope of Section 5.3 applies. Substituting the random-oracle envelopes of Section 5.3 gives the stated bound. □

Theorem 2 (Multi-User Indistinguishability)**.** *For every mu-ind adversary $\mathcal{A}$ against TW128 with execution-uniform user cap D and the global resources of Section 4.4,*

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{D(D-1)}{2^{k+1}} + 2D\cdot\mathsf{KeyGuess}_k(N_{\mathsf{prim}}) + \frac{N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Use the pre-sampled-key formulation of Section 4.4: sample keys $K_1,\dots,K_D$ at the start of the experiment, and let the d-th New call activate K_d, unused keys being ignored. By the Lift Application corollary (Corollary 4) for the mu-ind game, there is a random-oracle mu-ind adversary $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ with $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ and the per-user construction-side caps of Section 4.4 such that

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}).$$

Let coll denote the event that two of the pre-sampled keys $K_1,\dots,K_D$ collide; $\Pr[\mathsf{coll}] \le \binom{D}{2}/2^k = D(D-1)/2^{k+1}$ by union bound. In the random-oracle model, use a standard hybrid over the D users: H_d answers users $1,\dots,d$ with $(\mathsf{Rand},\mathsf{Replay})$ and users $d{+}1,\dots,D$ with the real construction interface, so H_0 is the real mu-ind game and H_D is the ideal one. Let $H^\star_d$ be H_d with a fixed output bit on coll. Since coll has the same probability in all hybrids,

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{RO}}(\mathcal{B}) \le \Pr[\mathsf{coll}] + \left|\Pr[\mathcal{B}^{H^\star_0}\Rightarrow 1] - \Pr[\mathcal{B}^{H^\star_D}\Rightarrow 1]\right|.$$

By the triangle inequality, it remains to bound each adjacent stopped hybrid. For each d, Lemma 16 gives the adjacent hybrid bound

$$\left|\Pr[\mathcal{B}^{H^\star_{d-1}}\Rightarrow 1] - \Pr[\mathcal{B}^{H^\star_d}\Rightarrow 1]\right| \le 2\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}(n^{(d)}_{\mathsf{leaf}}-1)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Using $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$, each hybrid step costs $2\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$ plus the per-user leaf-collision and verification-tag terms. Summing across $d = 1,\dots,D$ gives $2D\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$ for the key-guess contribution. The leaf-collision contribution satisfies

$$\sum_d \frac{n^{(d)}_{\mathsf{leaf}}(n^{(d)}_{\mathsf{leaf}}-1)}{2^{\tau_{\mathsf{leaf}}+1}} = \frac{1}{2^{\tau_{\mathsf{leaf}}}}\sum_d \binom{n^{(d)}_{\mathsf{leaf}}}{2} \le \frac{1}{2^{\tau_{\mathsf{leaf}}}}\binom{\sum_d n^{(d)}_{\mathsf{leaf}}}{2} \le \frac{N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)}{2^{\tau_{\mathsf{leaf}}+1}},$$

the first inequality because the unordered pairs drawn within the per-user blocks are a subset of all unordered pairs among $\sum_d n^{(d)}_{\mathsf{leaf}}$ items, and the second by $\sum_d n^{(d)}_{\mathsf{leaf}} = n_{\mathsf{leaf}} \le N_{\mathsf{leaf}}$ and monotonicity of $\binom{\cdot}{2}$. The verification-tag contribution satisfies

$$\sum_d q^{(d)}_{\mathsf{v}}/2^{\tau_{\mathsf{root}}} \le Q_{\mathsf{v}}/2^{\tau_{\mathsf{root}}}.$$

Combining with the coll penalty $D(D{-}1)/2^{k+1}$ and $\mathsf{Lift}_c(N_{\mathsf{tot}})$ yields the stated bound. □

Theorem 3 (Tag-Projected CMTDs-4 Committing Security)**.** *For every $s \ge 2$ and every s-way tag-CMTDs-4 adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}}+N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s).$$

Proof. Immediate from the public-permutation corollary Corollary 6, which lifts the random-oracle bound of Theorem 8. The theorem allows the s openings to use distinct ciphertext bodies sharing only the final root tag; the $\mathsf{LeafColl}$ term accounts for the only extra way distinct bodies can merge before the root tag is compared—a collision among the leaf tags absorbed by the root transcript. □

Corollary 1 (CMTDs-4 Committing Security)**.** *For every $s \geq 2$ and every s-way CMTDs-4 adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\mathbf{Adv}^{\text{CMTDs-4}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) \;=\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. The CMTDs-4 game is the special case of Theorem 3 in which all s openings share a single ciphertext body. With one common body no leaf tag collision can merge two openings, so the LeafColl term is absent and only the root-collision bound remains. Concretely, for deterministic Dec every CMTDs-4 winner is also a CMTDs-3 winner, yielding $\mathbf{Adv}^{\text{CMTDs-4}}_{\mathsf{TW128}}(\mathcal{A}) \leq \mathbf{Adv}^{\text{CMTDs-3}}_{\mathsf{TW128}}(\mathcal{A})$ by Section C, and the public-permutation CMTDs-3 bound Corollary 5 then gives the claim. □

Remark 1 (Looseness of the Root-Collision Cap). The N_{prim} in Corollary 1's root-collision term counts every primitive query, not only those whose induced simulator $\mathsf{RO}_{\mathsf{flat}}$ call lands at the root tag output point—$\mathcal{R}_{\mathsf{tag}}$, or the elided-aggregation $\mathcal{R}^{+}_{\mathsf{msg}}$ for single-chunk ciphertexts (Table 8). By Corollary 3, calls landing at any other output-point class cannot contribute to the candidate sequence, and hence cannot contribute to the candidate-collision event of Theorem 7's proof. A sharper candidate-sequence analysis could replace N_{prim} in the root-collision term by a class-filtered effective query count, no larger than N_{prim} and potentially smaller. The direct CMTs-ℓ bounds of Section C use the same accounting boundary: their challenger encryption checks are construction-internal work counted in N, so they do not enlarge the direct-query cap N_{prim}. The headline CMTDs-4 statement nevertheless keeps the N_{prim} form for simplicity and uniformity with Theorems 1 and 2; the birthday threshold at $\tau_{\mathsf{root}} = 256$ is unaffected.

Theorem 4 (Context Discoverability)**.** *For every context-discoverability adversary $\mathcal{A}$ against TW128 with the resources of Section 4.3,*

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathit{TW128}}(\mathcal{A}) \;\leq\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}) \;=\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{N_{\mathsf{prim}}+1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Immediate from the public-permutation corollary Corollary 7, which lifts the random-oracle bound of Theorem 9 through the flat-sponge transform. Where Corollary 1 bounds the probability that s distinct contexts collide on one root tag, this bounds the probability that a single context reaches a root tag fixed in advance; the candidate-collision argument of Theorem 7 is replaced by the candidate-preimage argument of Theorem 9, dropping the exponent from $\tau_{\mathsf{root}}(s-1)$ to τ_{root}. □

5.2 Loss Terms

Five closed-form expressions recur across the headline bounds of Section 5.1—Lift_c throughout, with $\mathsf{KeyGuess}_k$, $\mathsf{RootColl}_\tau$, $\mathsf{RootPre}_\tau$, and $\mathsf{LeafColl}_\tau$ in some. We collect them here for reference.

Flat-sponge loss. The flat-sponge differentiating loss is

$$\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}.$$

[BDPVA08, Lemmas 4 and 5] give the differentiating probabilities in exact product form: the random-transformation probability is $f_T(N_{\mathsf{tot}}) = 1 - \prod_{i=1}^{N_{\mathsf{tot}}}(1 - i/2^c)$, and the factor of the random-permutation probability $f_P(N_{\mathsf{tot}})$ corresponding to the f_T factor at index i is that factor divided by a quantity $1 - (i-1)/2^{r+c} \in (0,1]$, hence at least as large, so $f_P(N_{\mathsf{tot}}) \leq f_T(N_{\mathsf{tot}})$. For $N_{\mathsf{tot}} < 2^c$ every factor lies in $[0,1]$ and

$$f_P(N_{\mathsf{tot}}) \leq f_T(N_{\mathsf{tot}}) = 1 - \prod_{i=1}^{N_{\mathsf{tot}}}\Big(1 - \frac{i}{2^c}\Big) \leq \sum_{i=1}^{N_{\mathsf{tot}}} \frac{i}{2^c} = \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{c+1}} = \mathsf{Lift}_c(N_{\mathsf{tot}}),$$

the second inequality by the Weierstrass product inequality $\prod_i (1 - x_i) \geq 1 - \sum_i x_i$ for $x_i \in [0, 1]$. [BDPVA08, Theorem 2] carries the side condition $N_{\mathsf{tot}} < 2^c$; for $N_{\mathsf{tot}} \geq 2^c$, $\mathsf{Lift}_c(N_{\mathsf{tot}}) \geq 1$ and the advantage bound is trivial without any appeal to indifferentiability, and already at $N_{\mathsf{tot}} \geq 2^{c/2}$ the bound exceeds $1/2$ and provides no meaningful security.

Key-guessing term. The ideal-system key-guessing term of [BDPVA11] is

$$\mathsf{KeyGuess}_k(Q) = Q/2^k.$$

It accounts for direct $\mathsf{RO}_{\mathsf{flat}}$ queries whose inputs satisfy the decoder $\mathsf{KeyedTWOut}$ of Table 8. Each such query is an adaptive test of the decoded candidate key against the hidden key K, independent of the query's requested output length. Lemma 9 bounds the probability that any of Q direct queries tests the right key by $Q/2^k$.

Root-collision term. The s-way root tag multi-collision term is

$$\mathsf{RootColl}_\tau(Q, s) = \binom{Q+s}{s} / 2^{\tau(s-1)},$$

with the abbreviation $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(Q, s) = \mathsf{RootColl}_\tau(Q, s)$ at $\tau = \tau_{\mathsf{root}}$. The $+s$ is candidate-sequence slack for the s final root transcript inputs generated by the committing challenger after the adversary outputs its tuples; these lookups are construction-internal and are not added to $q_{\mathsf{RO}}(\mathcal{B})$. In the public-permutation CMTDs-3 and direct CMTs-ℓ bounds, the committing proof builds a chronological candidate sequence and bounds only the event that some s distinct candidates have equal root tag projections. The flat-sponge lift of Section B gives $Q = N_{\mathsf{prim}}$: only the adversary's N_{prim} direct primitive queries can induce simulator-side $\mathsf{RO}_{\mathsf{flat}}$ queries that contribute to that sequence.

Root-preimage term. The root tag preimage term is

$$\mathsf{RootPre}_\tau(Q) = (Q + 1)/2^\tau,$$

with the abbreviation $\mathsf{RootPre}_{\tau_{\mathsf{root}}}(Q) = \mathsf{RootPre}_\tau(Q)$ at $\tau = \tau_{\mathsf{root}}$. It is the preimage analogue of $\mathsf{RootColl}_\tau$: a single query-independent target tag replaces the s-way internal collision, the $+s$ candidate-sequence slack collapses to the $+1$ of the single discoverability challenger decryption check, and the exponent drops from $\tau(s-1)$ to τ. As in the root-collision term, the flat-sponge lift gives $Q = N_{\mathsf{prim}}$. The term bounds the query-independent target of context discoverability (Theorem 4).

Leaf-collision term. The known-key leaf tag collision term is

$$\mathsf{LeafColl}_\tau(Q) = \binom{Q}{2} / 2^\tau.$$

It is the pairwise birthday bound on τ-bit leaf tag projections used by the tag-projected CMTDs-4 theorem. Unlike the hidden-key leaf collision term in the INT-CTXT bound, the compact committing game has adversary-supplied keys, so direct primitive queries can pre-read candidate leaf tag points. The random-oracle proof therefore counts both direct leaf tag candidates and challenger-generated final leaf transcripts; after the flat-sponge lift this gives $Q = N_{\mathsf{prim}} + N_{\mathsf{leaf}}$.

5.3 Supporting Bounds

The headline theorems of Section 5.1 feed on five random-oracle-model bounds—the privacy and INT-CTXT envelopes, the CMTDs-3 bound, the tag-CMTDs-4 bound, and the CDY bound—whose proofs occupy Sections 6 to 8; the remaining committing-notion bounds live in Section C. Resource-indexed suprema range over flat-RO adversaries (Section 4.2) with the displayed resources, and the flat-sponge lift of Section B instantiates each envelope at $q_{\mathsf{RO}} = N_{\mathsf{prim}}$ when transferring to the public-permutation setting.

Privacy envelope. Define

$$\varepsilon^{\mathsf{priv}}_{\mathsf{RO}}(Q_{\mathsf{RO}}) = \sup\big\{\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{RO}}(\mathcal{B}) : q_{\mathsf{RO}}(\mathcal{B}) \le Q_{\mathsf{RO}}\big\},$$

the supremum taken over nonce-respecting privacy adversaries against $\mathsf{Enc}^{\mathsf{RO}}_K$. The flat-RO privacy theorem of Section 6 gives

$$\varepsilon^{\mathsf{priv}}_{\mathsf{RO}}(Q_{\mathsf{RO}}) \le \mathsf{KeyGuess}_k(Q_{\mathsf{RO}}).$$

INT-CTXT envelope. Define

$$\varepsilon^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(Q_{\mathsf{RO}}, N_{\mathsf{leaf}}, Q_{\mathsf{v}}) = \sup\big\{\mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(\mathcal{B}) : q_{\mathsf{RO}}(\mathcal{B}) \le Q_{\mathsf{RO}},\, n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}},\, q_{\mathsf{v}}(\mathcal{B}) \le Q_{\mathsf{v}}\big\}.$$

The flat-RO INT-CTXT theorem of Section 7 gives

$$\varepsilon^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(Q_{\mathsf{RO}}, N_{\mathsf{leaf}}, Q_{\mathsf{v}}) \le \mathsf{KeyGuess}_k(Q_{\mathsf{RO}}) + \frac{N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

CMTDs-3 bound. The flat-RO CMTDs-3 theorem of Section 8 is the direct root-collision bound

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s) = \binom{q_{\mathsf{RO}}(\mathcal{B}) + s}{s} \Big/ 2^{\tau_{\mathsf{root}}(s-1)},$$

with no key-guessing or leaf-collision term. The keys are adversary-supplied, so a direct query on an input accepted by the keyed transcript decoder $\mathsf{KeyedTWOut}$ is one of the adversary's visible $\mathsf{RO}_{\mathsf{flat}}$ queries and is counted in $q_{\mathsf{RO}}(\mathcal{B})$; leaf tag collisions cannot merge the final root transcripts whose tags are compared, since by Corollary 2 distinct (K, N, A) triples force distinct final root transcripts before aggregation values matter.

Tag-CMTDs-4 bound. The flat-RO tag-CMTDs-4 theorem of Section 8 is

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s).$$

The root-collision part is the same final-root candidate argument as CMTDs-3. The additional known-key leaf-collision term is needed because tag-projected openings may use different ciphertext bodies; absent such a leaf tag collision, pairwise distinct full inputs force pairwise distinct final root transcripts before the common tag is compared.

CDY bound. The flat-RO CDY theorem of Section 8 is the direct root-preimage bound

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})) = (q_{\mathsf{RO}}(\mathcal{B}) + 1)/2^{\tau_{\mathsf{root}}},$$

again with no key-guessing or leaf-collision term, by the same adversary-supplied-key accounting. A single context replaces the s tuples, so the candidate-collision argument

becomes a candidate-preimage argument against the query-independent target tag and no triple-separation is invoked.

Section B carries out the flat-sponge lift in two hops. Hop 1 applies the [BDPVA08] indifferentiability theorem through Lemma 18, replacing the real permutation by the corresponding flat-RO TW128 construction interface and the [BDPVA08] public-primitive simulator at cost at most $\mathsf{Lift}_c(N_{\mathsf{tot}})$, where $N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$ counts construction-internal sponge calls and direct primitive queries. Hop 2 (Lemma 19) defines a derived flat-RO adversary $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ that answers each primitive query by running the simulator against direct $\mathsf{RO}_{\mathsf{flat}}$ queries; this is a lossless equality of joint distributions, and the simulator-call filter of Lemma 17 makes each primitive query induce at most one $\mathsf{RO}_{\mathsf{flat}}$ call, so

$$q_{\mathsf{RO}}(\mathcal{B}_{\mathrm{derived}}) \le N_{\mathsf{prim}}.$$

The random-oracle envelopes of Section 5.3 are therefore instantiated at N_{prim}, not N_{tot}, in the headline theorems—as is the CMTDs-3 candidate-sequence length bound $m \le N_{\mathsf{prim}} + s$ of Section 8, the tag-CMTDs-4 leaf/root candidate bounds, and the direct CMTs-ℓ bounds of Section C, whose challenger encryption checks are construction-internal and counted in N, not in N_{prim}.

5.4 Concrete TW128 Bounds

The implemented parameters of Section 3.2 fix

$$k = c = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256.$$

Substituting into Theorems 1 to 4 and Corollary 1 and using the random-oracle envelopes of Section 5.3 gives the following closed-form bounds.

All-in-one AE.

$$\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{2N_{\mathsf{prim}} + Q_{\mathsf{v}}}{2^{256}} + \frac{N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)}{2^{257}}.$$

Multi-user indistinguishability.

$$\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{2DN_{\mathsf{prim}} + Q_{\mathsf{v}}}{2^{256}} + \frac{D(D-1) + N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)}{2^{257}}.$$

***s*-way tag-projected CMTDs-4 committing security.** For every $s \ge 2$,

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+N_{\mathsf{leaf}}}{2}}{2^{256}} + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{256(s-1)}}.$$

For $s = 2$, this is the compact two-opening bound in which the common commitment is only the tag:

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{(N_{\mathsf{prim}}+N_{\mathsf{leaf}})(N_{\mathsf{prim}}+N_{\mathsf{leaf}}-1)}{2^{257}} + \frac{(N_{\mathsf{prim}}+1)(N_{\mathsf{prim}}+2)}{2^{257}}.$$

***s*-way CMTDs-4 committing security.** For every $s \ge 2$, the all-bodies-equal special case drops the leaf tag term:

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \le \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{256(s-1)}}.$$

Table 4: Concrete TW128 security at the work cap $N_{\mathsf{tot}} \le 2^{63}$ (with $D \le 2^{63}$ for mu-ind), under $k = c = \tau_{\mathsf{root}} = \tau_{\mathsf{leaf}} = 256$ and the resource consequences $N_{\mathsf{prim}}, Q_{\mathsf{v}}, N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$ of Section 4.3. The committing rows hold for every $s \ge 2$; larger s only shrinks the root-collision term.

Notion	Dominant term	At the cap
All-in-one AE	$\mathsf{Lift}_c(N_{\mathsf{tot}})$	$\le 2^{-128}$
Multi-user ind.	$2D\,\mathsf{KeyGuess}_k(N_{\mathsf{prim}})$	$\le 2^{-128}$
Tag-CMTDs-4	$\mathsf{Lift}_c(N_{\mathsf{tot}})$	$\le 2^{-128}$
CMTDs-4	$\mathsf{Lift}_c(N_{\mathsf{tot}})$	$\le 2^{-128}$
Context discoverability	$\mathsf{Lift}_c(N_{\mathsf{tot}})$	$\le 2^{-128}$

Remark 2 (Two-Key CMTD-4 Specialization). Setting $s = 2$ recovers the ordinary two-key CMTD-4 statement of [BH22]:

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}4}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{\binom{N_{\mathsf{prim}}+2}{2}}{2^{256}} \;=\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1) + (N_{\mathsf{prim}}+1)(N_{\mathsf{prim}}+2)}{2^{257}}.$$

Context discoverability.

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathsf{TW128}}(\mathcal{A}) \;\le\; \frac{N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)}{2^{257}} + \frac{N_{\mathsf{prim}}+1}{2^{256}}.$$

5.5 Concrete Security Claim

TW128 attains a 128-bit security target across the notions of Section 5.4. This is an instantiation of the closed forms above under the resource consequences of Section 4.3: $N_{\mathsf{prim}} \le N_{\mathsf{tot}}$ by definition, and the pathwise bounds $q_{\mathsf{v}} \le N$ and $n_{\mathsf{leaf}} \le N$ let the envelope parameters Q_{v} and N_{leaf} be taken at most N_{tot}, so no separate top-level cap on Q_{v} or N_{leaf} is required; only the mu-ind user cap D remains an independent hypothesis, since created-but-idle users need not contribute to N. At the headline work cap $N_{\mathsf{tot}} \le 2^{63}$ (and $D \le 2^{63}$ for mu-ind) every bound is below 2^{-128}, with the flat-sponge loss $\mathsf{Lift}_c(N_{\mathsf{tot}})$ the dominant term in all but the multi-user case (Table 4).

Term by term at the cap, with $N_{\mathsf{prim}}, Q_{\mathsf{v}}, N_{\mathsf{leaf}} \le N_{\mathsf{tot}}$: the flat-sponge loss $\mathsf{Lift}_c(N_{\mathsf{tot}})$ and any leaf tag birthday term $N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)/2^{257}$ are each at most one 2^{-131}-sized term; the linear key-guess and verification contributions $(2N_{\mathsf{prim}} + Q_{\mathsf{v}})/2^{256}$ are at most $3 \cdot 2^{-193}$; the two-key root-collision term $\mathsf{RootColl}_{256}(N_{\mathsf{prim}}, 2)$ likewise matches Lift_c at $\le 2^{-131}$ and decays as $2^{-256(s-1)}$ for $s \ge 3$; the known-key $\mathsf{LeafColl}_{256}(N_{\mathsf{prim}} + N_{\mathsf{leaf}})$ is below 2^{-129}; and the root-preimage term $(N_{\mathsf{prim}}+1)/2^{256}$ is at most $2 \cdot 2^{-193}$. The single-user totals are therefore dominated by Lift_c and stay below 2^{-128}. The multi-user bound is the exception: its hybrid-induced term $2D \cdot \mathsf{KeyGuess}_k(N_{\mathsf{prim}}) \le 2DN_{\mathsf{tot}}/2^{256}$ reaches $\approx 4\mathsf{Lift}_c$ at the upper end and dominates—staying at most Lift_c once $D \le N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/(4N_{\mathsf{prim}})$, typical of any practical deployment—and the total there is at most $7 \cdot 2^{-131} + 3 \cdot 2^{-194} < 8 \cdot 2^{-131} = 2^{-128}$. Expressed in bytes, $N_{\mathsf{tot}} \le 2^{63}$ corresponds to at most $\rho \cdot 2^{63} = 167 \cdot 2^{63} < 2^{71}$ bytes of plaintext, associated data, ciphertext, and aggregation through the duplex (Section 3.2), well beyond any realistic deployment lifetime.

5.6 Discussion of the Bounds

The bounds of Section 5.4 aggregate the loss terms of Section 5.2, whose sizes at the work cap Table 4 records and whose dominance Section 5.5 discusses. We comment on their tightness.

Tightness.

- $\mathsf{KeyGuess}_k(Q) = Q/2^k$. Tight at constants: a direct random-guess attack on a k-bit key matches the per-query success probability $1/2^k$.
- $\mathsf{RootColl}_\tau(Q, s) = \binom{Q+s}{s}/2^{\tau(s-1)}$. Tight up to lower-order terms: it is the standard s-way multi-collision bound on a τ-bit projection of $\mathsf{RO}_{\mathsf{flat}}$, and a generic birthday-style adversary against the random oracle matches the leading-order term.
- *Leaf tag collision* $N_{\mathsf{leaf}}(N_{\mathsf{leaf}} - 1)/2^{\tau_{\mathsf{leaf}}+1}$. The pairwise union bound is tight up to constants: a generic birthday attack on the τ_{leaf}-bit leaf tag projection matches the leading $N_{\mathsf{leaf}}^2/2^{\tau_{\mathsf{leaf}}+1}$ rate.
- *Known-key leaf collision* $\mathsf{LeafColl}_\tau(Q)$. The same birthday attack applies when keys are adversary-supplied and direct queries can target leaf tag points; the compact CMTDs-4 proof uses $Q = N_{\mathsf{prim}} + N_{\mathsf{leaf}}$.
- *Flat-sponge lift.* The term $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}$ bounds the exact random-permutation differentiating probability of [BDPVA08, Lemma 5] via Section 5.2. The remaining slack is the unimproved tightness of the [BDPVA08] simulator itself, which is an open problem on the sponge side and not specific to TW128.

Cross-scheme bound-level comparisons against prior sponge AEADs and the committing-AEAD line of [BH22] are deferred to Sections 10.1 to 10.3.

6 Privacy

This section proves the random-oracle privacy bound feeding the all-in-one AE composition of Theorem 1. The public-permutation privacy statement follows by the flat-sponge lift of Section B.

Theorem 5 (Random-Oracle Privacy)**.** *For every nonce-respecting privacy adversary $\mathcal{B}$ in the TW128 random-oracle model (Section 4.2),*

$$\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) \;=\; q_{\mathsf{RO}}(\mathcal{B})/2^k.$$

Equivalently, the resource-indexed envelope satisfies $\varepsilon^{\mathsf{priv}}_{\mathsf{RO}}(Q_{\mathsf{RO}}) \le \mathsf{KeyGuess}_k(Q_{\mathsf{RO}})$.

Proof. Let P_0 be the real-side random-oracle privacy game: the challenger samples $K \leftarrow\!\$\ \{0,1\}^k$, $\mathcal{B}$ has direct access to $\mathsf{RO}_{\mathsf{flat}}$, and accepted construction-interface queries are answered by $\mathsf{Enc}^{\mathsf{RO}}_K$. Let P_1 be the corresponding ideal-side game, augmented with an independent auxiliary sample $K \leftarrow\!\$\ \{0,1\}^k$: construction-interface queries are answered by Rand, which returns a fresh uniformly random byte string of length $\|M\| + \tau_{\mathsf{root}}/8$, and direct queries are answered by the same lazy $\mathsf{RO}_{\mathsf{flat}}$ interface. The auxiliary key is used only for the abort predicate below and is ignored by Rand, so the adversary's marginal view is the ideal term in the definition of $\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{RO}}(\mathcal{B})$. Let q_e be an execution-uniform cap on the number of accepted encryption queries made by $\mathcal{B}$, as in Section 4.3; this cap indexes the finite replacement hybrid sequence and will not enter the final privacy bound.

Use the common abort event $\mathsf{bad}_{\mathsf{key}}$: a direct adversary query (Y, ℓ) to $\mathsf{RO}_{\mathsf{flat}}$ satisfies

$$\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$$

for this game's key K (the hidden key in P_0, the auxiliary key in P_1) and some counted output-point class $\mathcal{C}$ of Table 8; the requested length ℓ is ignored.

Let $\mathsf{P}_0^{\mathrm{stop}}$ and $\mathsf{P}_1^{\mathrm{stop}}$ be the immediate-abort variants of P_0 and P_1 on this event. By the Privacy Stopped-Game Advantage Bound (Lemma 13) applied to the event $\mathcal{B} \Rightarrow 1$,

$$\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{RO}}(\mathcal{B}) = \left|\Pr[\mathcal{B}^{\mathsf{P}_0} \Rightarrow 1] - \Pr[\mathcal{B}^{\mathsf{P}_1} \Rightarrow 1]\right| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

Taking the supremum over nonce-respecting privacy adversaries with $q_{\mathsf{RO}}(\mathcal{B}) \le Q_{\mathsf{RO}}$ gives the envelope statement $\varepsilon^{\mathsf{priv}}_{\mathsf{RO}}(Q_{\mathsf{RO}}) \le \mathsf{KeyGuess}_k(Q_{\mathsf{RO}})$. □

Lineage. The proof is the flat-random-oracle analog of the standard SpongeWrap privacy argument: stop on direct reads of hidden-key construction transcripts, replace each nonce-respecting encryption query by fresh random output under transcript freshness, and bound the stop event as an adaptive key test. The TW128-specific interfaces are Transcript Freshness (Lemma 11) and the privacy accounting lemmas (Lemmas 12 and 13).

7 Ciphertext Integrity

This section proves the random-oracle INT-CTXT bound feeding the all-in-one AE composition of Theorem 1. The public-permutation INT-CTXT statement follows by the flat-sponge lift of Section B.

Theorem 6 (Random-Oracle INT-CTXT)**.** *For every INT-CTXT adversary $\mathcal{B}$ in the TW128 random-oracle model (Section 4.2),*

$$\mathbf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}(\mathcal{B})\big(n_{\mathsf{leaf}}(\mathcal{B})-1\big)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{q_{\mathsf{v}}(\mathcal{B})}{2^{\tau_{\mathsf{root}}}}.$$

Equivalently, the resource-indexed envelope satisfies

$$\varepsilon^{\mathsf{int\text{-}ctxt}}_{\mathsf{RO}}(Q_{\mathsf{RO}}, N_{\mathsf{leaf}}, Q_{\mathsf{v}}) \;\le\; \mathsf{KeyGuess}_k(Q_{\mathsf{RO}}) + \frac{N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{Q_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Let G_0 be the real random-oracle INT-CTXT game: the challenger samples $K \leftarrow\!\$\ \{0,1\}^k$, $\mathcal{B}$ has direct access to $\mathsf{RO}_{\mathsf{flat}}$, and construction queries are answered by $\mathsf{Enc}^{\mathsf{RO}}_K$ and $\mathsf{Ver}^{\mathsf{RO}}_K$. Each encryption query (N, A, M) returns $C \,\|\, T = \mathsf{Enc}^{\mathsf{RO}}_K(N, A, M)$ and records the tuple (N, A, C, T); $\mathcal{B}$ wins if any verification query returns 1 on a tuple (N, A, C, T) that was not previously recorded by the encryption oracle.

Let $\mathsf{bad}_{\mathsf{key}}$ be the event that some direct adversary query (Y, ℓ) to $\mathsf{RO}_{\mathsf{flat}}$ satisfies $\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$ for a counted output-point class $\mathcal{C}$ of Table 8; the requested length ℓ is ignored. Let G_1 be G_0 with an immediate abort on $\mathsf{bad}_{\mathsf{key}}$. The games are identical until this event, so

$$\Pr[\mathsf{G}_0 \text{ wins}] \le \Pr[\mathsf{G}_1 \text{ wins}] + \Pr[\mathsf{bad}_{\mathsf{key}}].$$

Lemma 14 verifies the visible-prefix uniformity invariant required by the Adaptive Key-Test Bound (Lemma 9) in G_1, with the set of already tested candidate keys recovered by $\mathsf{KeyedTWOut}$. Therefore

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \;\le\; \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

Let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that two distinct construction-generated final leaf transcripts under K receive the same τ_{leaf}-bit RF projection. Let G_2 be G_1 with an additional immediate abort on $\mathsf{bad}_{\mathsf{leaf}}$. The games are identical until this event, so

$$\Pr[\mathsf{G}_1 \text{ wins}] \le \Pr[\mathsf{G}_2 \text{ wins}] + \Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{bad}_{\mathsf{key}}].$$

Before $\mathsf{bad}_{\mathsf{key}}$, no direct query has pre-read the bridged input of a hidden-key final leaf transcript before the construction first creates that transcript: such a query would be decoded by KeyedTWOut to the hidden key K and its $\mathcal{L}_{\mathsf{tag}}[j]$ class, independently of the requested output length, and would itself trigger $\mathsf{bad}_{\mathsf{key}}$. This supplies the event premise for the Leaf Tag Collision Bound (Lemma 10): thus $\{\mathsf{bad}_{\mathsf{leaf}}$ before $\mathsf{bad}_{\mathsf{key}}\}$ is contained in $\{\mathsf{bad}_{\mathsf{leaf}}$ before $\mathsf{hit}_{\mathsf{leaf}}\}$, where $\mathsf{hit}_{\mathsf{leaf}}$ is the event of Lemma 10 that a direct query hits the bridged input of a hidden-key final leaf transcript before the construction first creates it; hence

$$\Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{bad}_{\mathsf{key}}] \;\le\; \frac{n_{\mathsf{leaf}}(\mathcal{B})\big(n_{\mathsf{leaf}}(\mathcal{B})-1\big)}{2^{\tau_{\mathsf{leaf}}+1}}.$$

Bound on $\Pr[\mathsf{G}_2 \text{ wins}]$. In G_2, every accepting non-replay verification submission belongs to a final root transcript class. If the class is encryption-first—first generated by an encryption query returning (N_e, A_e, C_e, T_e)—then, since $\mathsf{bad}_{\mathsf{leaf}}$ has not occurred, the final-root determinacy part of Lemma 5 gives $(N, A, C) = (N_e, A_e, C_e)$ and the correct root tag is the value already returned as T_e, so the submission is a replay when $T = T_e$ and fails the tag comparison otherwise; it cannot be an accepting non-replay. The remaining accepting non-replay submissions are therefore verification-first. By Lemma 15, their total probability before $\mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}$ is bounded by the root tag guessing term:

$$\Pr[\mathsf{G}_2 \text{ wins}] \;\le\; q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Conclusion. Combining the three stopped-game bounds,

$$\mathbf{Adv}^{\text{int-ctxt}}_{\mathsf{RO}}(\mathcal{B}) = \Pr[\mathsf{G}_0 \text{ wins}] \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n_{\mathsf{leaf}}(\mathcal{B})\big(n_{\mathsf{leaf}}(\mathcal{B})-1\big)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{q_{\mathsf{v}}(\mathcal{B})}{2^{\tau_{\mathsf{root}}}}.$$

Taking the supremum over INT-CTXT adversaries with $q_{\mathsf{RO}}(\mathcal{B}) \le Q_{\mathsf{RO}}$, $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$, and $q_{\mathsf{v}}(\mathcal{B}) \le Q_{\mathsf{v}}$ gives the envelope statement. □

Lineage. The random-oracle decomposition of the bound into a key-guessing term and a tag-guessing term ($q\,2^{-k}$ and 2^{-t} in their notation) follows the authenticity bound of the ideal scheme $\mathcal{RO}_{\mathrm{wrap}}[\rho]$ in the SpongeWrap analysis [BDPVA11, Lemma 6], which carries no sponge-capacity term. We realize the key-guessing term as the $\mathsf{bad}_{\mathsf{key}}$ abort and the tag-guessing term as the residual root tag guessing bound on $\Pr[\mathsf{G}_2$ wins], with a TW128-specific leaf tag birthday abort $\mathsf{bad}_{\mathsf{leaf}}$ inserted between them. The sponge-capacity differentiating loss of [BDPVA11, §5, Theorem 1] does not appear at this random-oracle layer; it enters only through the flat-sponge lift of Section B, as the [BDPVA08] indifferentiability term written Lift_c. The TW128-specific steps come from Transcript Injectivity (Lemma 5), the Leaf Tag Collision Bound (Lemma 10), which bounds the $\mathsf{bad}_{\mathsf{leaf}}$ abort by a birthday on the τ_{leaf}-bit leaf tag space, and the INT-CTXT auxiliary lemmas (Lemmas 14 and 15), with the leaf-freshness and encryption-first-replay steps argued inline.

8 Committing Security and Context Discoverability

All three guarantees of this section rest on one construction, isolated below as the Root-Tag Candidate Sequence lemma (Lemma 2): a chronological candidate sequence of root tag inputs, whose collision or preimage probability the adaptive lemmas of Section A bound, controlling the chance that distinct decryption contexts land on the same τ_{root}-bit root tag projection. We instantiate it three ways in the random-oracle model; Section B transfers each instance to the public-permutation setting and Section 5.1 records the resulting concrete bounds.

First, the random-oracle CMTDs-3 bound (Theorem 7) is the pure s-way root tag collision case; the headline CMTDs-4 statement (Corollary 1) consumes it through the trivial promotion of Section C, and its public-permutation form is the flat-sponge lift of Corollary 5. Second, the random-oracle CDY bound (Theorem 9) runs the same argument with the s-way collision replaced by a preimage to a fixed target tag, underlying the headline CDY statement (Theorem 4) through Corollary 7. Third, the tag-projected CMTDs-4 bound (Theorem 8) keeps the root tag collision argument intact but, because compact openings may use different ciphertext bodies under one common tag, first rules out the only other way distinct bodies can reach a common root transcript—a collision among the absorbed leaf tags—through the known-key leaf tag collision lemma (Lemma 3); its public-permutation form is Corollary 6, feeding the headline Theorem 3.

Lemma 2 (Root-Tag Candidate Sequence)**.** *Consider a random-oracle game of Section 4.2 in which the adversary $\mathcal{B}$ first makes direct $\mathsf{RO}_{\mathsf{flat}}$ queries and then outputs values on which the challenger runs $m \geq 1$ deterministic decryption checks, the i-th reaching a final root transcript R_i whose root tag is read by a single $\mathsf{RO}_{\mathsf{flat}}$ lookup at $\mathsf{flat}(R_i)$. Every execution then admits a chronological sequence of root-tag inputs such that*

1. *each distinct direct query whose input Y decodes under $\mathsf{KeyedTWOut}$ to a root tag output-point class—$\mathcal{R}^{+}_{\mathsf{msg}}$ (the single-chunk root tag, aggregation elided) or $\mathcal{R}_{\mathsf{tag}}$ (the multi-chunk root tag)—is appended as a candidate before its $\mathsf{RO}_{\mathsf{flat}}$ lookup, and contributes at most one candidate;*
2. *if the challenger reaches its checks, each $\mathsf{flat}(R_i)$ is in the sequence, appended—if not already present—immediately before the i-th root tag answer is sampled;*
3. *no candidate's $\mathsf{RO}_{\mathsf{flat}}$ value is read before its append step, so the sequence satisfies the predictability and unrevealed-value premises of Lemmas 7 and 8; and*
4. *its length is at most $q_{\mathsf{RO}}(\mathcal{B}) + m$.*

Proof. The committing and discoverability games are keyless: all keys are adversary-supplied, so a direct query to a keyed transcript is one of $\mathcal{B}$'s visible $\mathsf{RO}_{\mathsf{flat}}$ queries counted in $q_{\mathsf{RO}}(\mathcal{B})$, and there is no $\mathsf{bad}_{\mathsf{key}}$ event. No construction oracle runs during the direct-query phase, so each direct-query candidate is first read no earlier than its append step. The append predicate depends on Y alone through the exact decoder of Lemma 6, which ignores the requested output length, and by Output-Point Separation (Corollary 3) each input decodes to at most one output-point class; hence each distinct direct query contributes at most one candidate, giving (1).

If the challenger reaches its m checks, append $\mathsf{flat}(R_i)$ immediately before the i-th root tag answer is sampled, unless it is already present. The intermediate $\mathsf{RO}_{\mathsf{flat}}$ lookups of check i lie at non-root-tag output points, so by Corollary 3 they do not read $\mathsf{flat}(R_i)$. If $\mathsf{flat}(R_i)$ was touched earlier—by a direct query or by an earlier check $j < i$—then Lemma 6 and Corollary 3 make that earlier access a final-root lookup at the same address, which was therefore already appended, so the input is in the sequence and is not appended again. Otherwise its value is still unread when appended. Either way no candidate's value is read before its append step, giving (2) and (3). At most $q_{\mathsf{RO}}(\mathcal{B})$ direct candidates and m challenger candidates are appended, giving (4). □

Theorem 7 (Random-Oracle CMTDs-3)**.** *For every $s \geq 2$ and every s-way CMTDs-3 adversary $\mathcal{B}$ in the TW128 random-oracle model (Section 4.2),*

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathsf{RO}}(\mathcal{B}) \;\leq\; \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s) = \frac{\binom{q_{\mathsf{RO}}(\mathcal{B})+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. Run the random-oracle CMTDs-3 game (Section 4.2). After rejecting on any failed syntax, message-length, or pairwise-distinct-triple check, its challenger performs $m = s$ deterministic decryption checks $\mathsf{Dec}^{\mathsf{RO}}_{K_i}(N_i, A_i, C \,\|\, T)$, reaching final root transcripts $R_1, \ldots, R_s$. The Root-Tag Candidate Sequence lemma (Lemma 2) yields a sequence of length at most $q_{\mathsf{RO}}(\mathcal{B}) + s$ meeting the premises of Lemma 7 and containing each $\mathsf{flat}(R_i)$.

On a winning execution the pairwise-distinct-triple check passed, so the (K_i, N_i, A_i) triples are pairwise distinct and, by Opening Triple Separation (Corollary 2), the bridged final-root inputs $\mathsf{flat}(R_1), \ldots, \mathsf{flat}(R_s)$ are pairwise distinct; leaf tag collisions cannot merge them. Since every decryption check accepts, the τ_{root}-bit projections at these s distinct candidates all equal the supplied tag T, so the win event is contained in the s-way candidate-collision event. Lemma 7 with $Q = q_{\mathsf{RO}}(\mathcal{B})$ and $\tau = \tau_{\mathsf{root}}$ gives

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathsf{RO}}(\mathcal{B}) \le \binom{q_{\mathsf{RO}}(\mathcal{B}) + s}{s} \cdot 2^{-\tau_{\mathsf{root}}(s-1)} = \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s).$$

□

Lemma 3 (Random-Oracle Known-Key Leaf Collision)**.** *For every tag-CMTDs-4 adversary $\mathcal{B}$ in the TW128 random-oracle model, let $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ be the event that two distinct final leaf transcripts generated by the tag-CMTDs-4 challenger decryption checks receive the same τ_{leaf}-bit RF projection. Then*

$$\Pr[\mathsf{bad}^{\star}_{\mathsf{leaf}}] \le \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) = \frac{\binom{q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})}{2}}{2^{\tau_{\mathsf{leaf}}}}.$$

Proof. Build a leaf tag candidate sequence for the tag-CMTDs-4 game. During $\mathcal{B}$'s direct oracle phase, append each distinct direct query input Y before answering it if $\mathsf{KeyedTWOut}(Y)$ decodes to a final leaf tag output-point class $\mathcal{L}_{\mathsf{tag}}[j]$. After $\mathcal{B}$ outputs its common tag and opening tuples, as the challenger performs its deterministic decryption checks, append the bridged input of each newly created distinct final leaf transcript immediately before the leaf tag oracle answer is sampled, unless that input is already in the sequence. If the input was touched earlier by a direct query, then Lemma 6 and Corollary 3 imply that the earlier query was already appended as a leaf tag candidate. If it was touched earlier by a construction computation, then Corollary 3, transcript injectivity (Lemma 5), and bridge injectivity imply that the same final leaf transcript had already been generated; repeated evaluations are not new collision opportunities. Thus every newly appended candidate satisfies the predictability and unrevealed-value premises of the adaptive candidate-collision argument, and the sequence length is at most $q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})$.

For any fixed unordered pair of candidate positions, expose the two positions in chronological order. When the later candidate is appended, its τ_{leaf}-bit projection is still unrevealed and uniform independently of the earlier one, so the pair collides with probability $2^{-\tau_{\mathsf{leaf}}}$. A union bound over at most $\binom{q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})}{2}$ pairs gives the claim. The event $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ is contained in this candidate collision event because every construction-generated final leaf transcript is either appended when first created or was already appended by a direct query to the same bridged input. □

Theorem 8 (Random-Oracle Tag-CMTDs-4)**.** *For every $s \ge 2$ and every s-way tag-CMTDs-4 adversary $\mathcal{B}$ in the TW128 random-oracle model,*

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s).$$

Proof. Run the random-oracle tag-CMTDs-4 game. By Lemma 3, the probability of $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ is at most the stated leaf-collision term, so it remains to bound the winning probability conditioned on $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$.

After rejecting on any failed syntax, message-length, or pairwise-full-input check, the challenger performs $m = s$ deterministic decryption checks $\mathsf{Dec}^{\mathsf{RO}}_{K_i}(N_i, A_i, C_i \,\|\, T)$, reaching final root transcripts $R_1, \ldots, R_s$. The Root-Tag Candidate Sequence lemma (Lemma 2) yields a sequence of length at most $q_{\mathsf{RO}}(\mathcal{B}) + s$ meeting the premises of Lemma 7 and containing each $\mathsf{flat}(R_i)$.

On a winning execution with $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$, the full tuples (K_i, N_i, A_i, M_i) are pairwise distinct and all decryption checks accept under the common tag T. We first show that the bridged final root inputs $\mathsf{flat}(R_1), \ldots, \mathsf{flat}(R_s)$ are pairwise distinct. Suppose instead that checks $i \neq j$ reach the same final root input. The bridge and transcript encodings are injective, so the root transcripts have the same key, nonce, associated data, root ciphertext chunk, root tag output point, and, when leaves are present, the same ordered leaf tag aggregation string. The trailing encoded leaf count gives the same number of leaves. For each leaf position, equality of the aggregated leaf tag values together with $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ implies that the two final leaf transcripts are the same; hence the corresponding ciphertext leaf chunks are the same. Thus the two decryption checks use the same $(K, N, A, C \,\|\, T)$. Determinism of Dec gives the same plaintext, so $(K_i, N_i, A_i, M_i) = (K_j, N_j, A_j, M_j)$, contradicting the tag-CMTDs-4 pairwise-full-input check. The final root inputs are therefore pairwise distinct.

Since every decryption check accepts, the τ_{root}-bit projections at these s distinct final-root candidates all equal the supplied tag T. The win event under $\neg\mathsf{bad}^{\star}_{\mathsf{leaf}}$ is contained in the s-way root candidate-collision event, which Lemma 7 bounds by $\mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$. Adding the $\mathsf{bad}^{\star}_{\mathsf{leaf}}$ probability gives the theorem. □

Theorem 9 (Random-Oracle CDY)**.** *For every query-independent target tag $T^* \in \{0,1\}^{\tau_{\mathsf{root}}}$ and every context-discoverability adversary $\mathcal{B}$ in the TW128 random-oracle model (Section 4.2),*

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathsf{RO}}(\mathcal{B}) \;\le\; \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})) = \frac{q_{\mathsf{RO}}(\mathcal{B}) + 1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. Run the random-oracle CDY game (Section 4.2) for the query-independent target T^*, which is independent of $\mathsf{RO}_{\mathsf{flat}}$. After rejecting on a failed syntax or ciphertext-validity precheck on $C \,\|\, T^*$, the challenger performs $m = 1$ deterministic decryption check $\mathsf{Dec}^{\mathsf{RO}}_{K}(N, A, C \,\|\, T^*)$, reaching a final root transcript R. The Root-Tag Candidate Sequence lemma (Lemma 2) yields a sequence of length at most $q_{\mathsf{RO}}(\mathcal{B}) + 1$ meeting the premises of Lemma 8 and containing $\mathsf{flat}(R)$; with a single context no pairwise distinctness is needed.

On a winning execution the precheck passes and the single decryption check accepts. By Algorithm 3, acceptance is the constant-time equality $T' = T^*$, where T' is the τ_{root}-bit projection at the final root transcript R; in the random-oracle model $T' = \left\lfloor \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(R)) \right\rfloor_{\tau_{\mathsf{root}}}$. Hence the win event is contained in the event that some candidate's τ_{root}-bit projection equals the fixed target T^*. Applying Lemma 8 with $Q = q_{\mathsf{RO}}(\mathcal{B})$, $\tau = \tau_{\mathsf{root}}$, and $t = T^*$ gives

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathsf{RO}}(\mathcal{B}) \le (q_{\mathsf{RO}}(\mathcal{B}) + 1) \cdot 2^{-\tau_{\mathsf{root}}} = \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B})).$$

□

9 Performance Evaluation

9.1 Optimized Implementations

We evaluate three implementations of TW128 sharing a common structure: a portable reference written in Go, an `amd64` implementation, and an `arm64` implementation. The two architecture-specific implementations replace only the KECCAK-p[1600, 12] permutation and the inner absorb/squeeze loops with hand-written assembly; the framing, domain

separation, padding, and tree bookkeeping remain in the shared Go code. The portable reference uses a scalar permutation and a scalar eight-instance loop, and serves both as a correctness oracle for the assembly cores and as the fallback on other architectures.

The tree structure of TW128 exposes two distinct permutation workloads, and both architectures provide a separate core for each. The root duplex—which absorbs the associated data, processes chunk 0, and absorbs the leaf tag aggregation transcript—is inherently sequential, since each of its calls consumes the previous call's output; it is driven by a *single-duplex* ($\times 1$) core that permutes one 1600-bit state at a time, except for its chunk-0 message phase, which is fused into the first batched leaf call as described below. The leaf duplexes, by contrast, operate on disjoint state and disjoint inputs (Section 3), so the implementation batches eight complete leaf chunks and runs them in lockstep through an *eight-way* ($\times 8$) core. Because every leaf chunk is exactly $\beta = 49\rho$ bytes, the eight instances step through an identical sequence of 49 full ρ-blocks (the first 48 closed with MSG_MORE, the last with MSG_LAST), so a single shared block position drives all eight. The eight-way state is held lane-major—25 lanes each holding the corresponding 64-bit word of all eight instances—so that the batched cores can address the eight instances of a lane with a single vector load.

Messages are decomposed into eight-chunk batches run on the $\times 8$ core, and a leftover run of two to seven complete chunks is drained by a narrower kernel: on `amd64` a register-resident kernel processes the remainder in a single call, reading the active chunks in place, and on `arm64` a 2-wide kernel consumes it two chunks at a time. These remainder kernels read each chunk as a sequential stream, so they prefetch as well on the cold tail of a long message as on a cache-resident short one; on the `amd64` host they outperform both repeated single-duplex calls and padding the remainder to a full eight-chunk batch, at every remainder width. Only a single leftover complete chunk, a partial trailing chunk, and the count-aggregation tail fall back to the single-duplex ($\times 1$) core.

Chunk 0 itself runs on the batched kernels rather than serially, a technique we refer to as *lane-0 fusion*. The root duplex's chunk-0 message phase has the same kernel-visible block schedule as a leaf chunk—the same 49 ρ-blocks with the same closing flags—and differs from a leaf only in its initial state, to which the kernels are indifferent; chunk 0 is also contiguous in the message with the first leaf chunks. The implementation therefore seeds lane 0 of the first batched call with the root state (after initialization and associated-data processing) and runs the chunk-0 phase alongside up to seven leaves: in the $\times 8$ core when the message has at least seven complete leaf chunks, and in the `amd64` remainder kernel or the `arm64` 2-wide kernel below that. Lane 0's output state is written back to the root duplex, which then absorbs the leaf tags produced by the same call; no serial pass over chunk 0 precedes the leaf work. On `amd64` the assembly kernels handle only the 48 full MSG_MORE blocks of each chunk; the final MSG_LAST block and the leaf tag extraction are performed in portable Go, shared with the reference path, which keeps the byte-granular tail logic out of assembly. The `arm64` kernels process the complete chunk, extracting the leaf tags and storing the permutation states back in assembly—the writeback that allows the root duplex to occupy lane 0.

Architecture-specific cores. On `amd64` the cores are built from AVX-512 (selected once at startup from the `AVX512VL` CPUID feature bit) and AVX2, holding the lane-major state in ZMM or YMM registers and permuting it with no cross-lane shuffles. The steady-state $\times 8$ kernel transposes each rate block into the lane-major layout with contiguous, prefetcher-friendly loads and stores, while the register-resident kernel that drains a two-to-seven-chunk remainder reads its active chunks in place by masked gather and scatter. On `arm64` the cores use NEON with the Armv8.2 SHA-3 extension (`FEAT_SHA3`, available on Apple Silicon and Neoverse V1/V2 and N2), one instance per 64-bit lane in the single-duplex core and two instances per 128-bit register in the $\times 8$ and 2-wide kernels. Both batched paths store

the permutation state back after the closing block—the writeback that lets the root duplex occupy lane 0. Section D gives the per-instruction detail, including the fused permutation instructions and the remainder-kernel strategies on each architecture.

Constant time. All three implementations are constant time with respect to the key, nonce, and plaintext. The permutation and the XOR-based absorb/squeeze contain no secret-dependent branches and no secret-dependent memory or vector indices. The only indexed table is the round-constant array, addressed by the public round number; the AVX-512 chunk transpose uses contiguous loads and stores, and the gather and scatter that remain—for the partial rate lane and the register-resident remainder kernel—are addressed by the fixed public chunk strides under a mask set by the public chunk count, and the AVX2 remainder kernel's redirection of inactive instances to its dummy block is likewise selected by the public chunk count. None of these depend on secret data.

9.2 Benchmark Methodology

We measure on two hosts, one per architecture. The `amd64` host is a Google Compute Engine `c4-standard-4` instance with an Intel Xeon Platinum 8581C (Emerald Rapids, 2.30 GHz base, AVX-512 including `AVX512VL`, `AVX512_VBMI2`, `GFNI`, and `VAES`), two physical cores with simultaneous multithreading enabled, 48 KiB L1d and 32 KiB L1i per core, 2 MiB L2 per core, and a 260 MiB shared L3, running Debian 12 (kernel 6.1) under KVM. The guest does not expose a `cpufreq` interface, so the frequency governor and turbo state are not under our control; we mitigate the resulting variance statistically (below). The `arm64` host is an Apple M4 Pro (10 performance cores and 4 efficiency cores; per performance core, 128 KiB L1i, 64 KiB L1d, and a 16 MiB shared L2) running macOS 26.5 (Darwin 25.5), with `FEAT_SHA3` present. macOS exposes no userspace frequency control.

Measurements are collected by a standalone harness (`cmd/cpb`) that locks its goroutine to an OS thread and, for each algorithm, operation, and message length, first calibrates an iteration count that fills at least 100 ms, then records 100 samples and reports the median cycles per byte and the median wall-clock throughput. Both metrics are read from the same timed loop, so the cycles-per-byte and throughput figures for a given measurement come from one interleaved run rather than two separately scheduled ones. Each sample reuses the same source and destination buffers, so the reported figures are warm-cache, steady-state rates. The single-threaded measurement leaves the SMT sibling of the `amd64` host idle. We report a sweep of seven message lengths from 1 B to 16 MiB.

The cycle counters differ by architecture, and this affects how the cycles-per-byte figures may be read. On `amd64` the harness reads `RDTSC`; on this part the time-stamp counter is invariant and ticks at the 2.30 GHz reference frequency, so the reported figures are *reference* cycles per byte and are unaffected by turbo (and convert directly to wall-clock time). On `arm64` the harness reads `CNTVCT_EL0` and scales it by the ratio of the peak core frequency to `CNTFRQ_EL0`; since Apple exposes no frequency API, the peak is auto-detected by timing a dependent integer-add chain and taking the top observed P-state, yielding *core* cycles per byte. Consequently the cycles-per-byte figures are comparable within an architecture and against the same-host AES-128-GCM baseline, but not directly across architectures; absolute throughput (Table 6) is the cross-platform metric.

The baseline is Go's AES-128-GCM implementation (`crypto/aes` with `crypto/cipher`). It uses AES-NI and `PCLMULQDQ` on `amd64`, and the AES and `PMULL` instructions on `arm64`. Each timed AES-128-GCM call constructs its cipher, so its measurement includes the key schedule and the GHASH key-power precomputation; this mirrors the TW128 measurement, whose one-shot seal and open perform the root duplex initialization (one permutation) inside the timed call. Both directions are measured for each scheme; the two directions track each other to within a few percent across the sweep, with a worst-case divergence of 10% (TW128 on `arm64` at 64 B).

Table 5: Median cycles per byte (100 samples). The `amd64` figures are `RDTSC` reference cycles at the 2.30 GHz invariant time-stamp counter; the `arm64` figures are core cycles at the auto-detected peak performance-core frequency. Cycle counts are not directly comparable across architectures (Section 9.2).

	Message length						
	1 B	64 B	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
Intel Xeon 8581C (Emerald Rapids), AVX-512							
TW128 encrypt	730	11.23	2.31	0.91	0.48	0.47	0.47
TW128 decrypt	733	11.53	2.29	0.91	0.48	0.48	0.47
AES-128-GCM seal	897	15.60	0.46	0.38	0.37	0.36	0.36
AES-128-GCM open	923	15.38	0.45	0.37	0.36	0.35	0.35
Intel Xeon 8581C (Emerald Rapids), AVX2							
TW128 encrypt	835	13.05	2.81	1.35	1.35	1.27	1.24
TW128 decrypt	864	13.33	2.82	1.42	1.35	1.33	1.35
Apple M4 Pro, NEON + `FEAT_SHA3`							
TW128 encrypt	620	9.63	1.93	0.91	0.88	0.87	0.89
TW128 decrypt	658	10.61	1.99	0.90	0.86	0.85	0.86
AES-128-GCM seal	1083	17.32	0.55	0.46	0.44	0.44	0.44
AES-128-GCM open	1081	17.38	0.53	0.44	0.42	0.42	0.42

9.3 Throughput

Table 5 reports cycles per byte across the message-length sweep and Table 6 the corresponding wall-clock throughput in the bulk regime. The dominant feature of the TW128 rows is the descent produced by the tree-parallel leaf core: the per-byte cost falls as a message lengthens and its chunks occupy more SIMD lanes. A message of at most one chunk has no complete leaf chunk for chunk 0 to share a kernel with, so it runs entirely on the single-duplex core, and the 8 KiB column sits at that core's rate. Past one chunk, lane-0 fusion (Section 9.1) keeps every complete chunk on the batched kernels, and the descent tracks lane occupancy: at 32 KiB, chunk 0 and three complete leaves run four-wide in the `amd64` remainder kernels and as two full pairs on `arm64`, and 64 KiB is the first length whose eight complete chunks ($8\beta \approx 64$ KiB) fill the eight-way ($\times 8$) core. On the `amd64` AVX-512 path TW128 encryption falls $2.31 \to 0.91 \to 0.48$ across the 8, 32, and 64 KiB columns and is already at its bulk rate at 64 KiB, holding at 0.47–0.48 through 16 MiB. The AVX2 path follows the same shape at lower throughput ($2.81 \to 1.35$ over the same span, thereafter between 1.24 and 1.35); it runs the eight instances as two groups of four rather than in a single 8-wide register, leaving the AVX-512 core about $2.6\times$ faster in bulk. The `arm64` curve flattens earliest ($1.93 \to 0.91 \to 0.88 \to 0.87 \to 0.89$ cycles per byte from 8 KiB to 16 MiB): the eight-way core itself processes its batch as four two-instance pairs, so the fused 2-wide kernel runs at essentially the same per-chunk rate, and a 32 KiB message—chunk 0 and three leaves as two full pairs—is already within 5% of the 16 MiB floor. In wall-clock terms TW128 reaches 4.92 GB/s on the `amd64` AVX-512 host and 5.03 GB/s on the M4 Pro for 16 MiB messages.

9.4 Latency

For short messages the cost is set by the serial root duplex and is independent of the parallel leaf machinery. A message of at most $\rho = 167$ bytes occupies a single rate block and never enters leaf mode, so it costs exactly two permutations—one to initialize the root duplex and one to close the message block and extract the tag. The 1 B and 64 B columns

Table 6: Measured wall-clock throughput (GB/s, GB = 10^9 bytes) from the same `cmd/cpb` run as Table 5, encryption direction; the decryption direction tracks it. The `amd64` AES-128-GCM row is the AVX-512 host and is unaffected by the TW128 core selection.

	8 KiB	32 KiB	64 KiB	1 MiB	16 MiB
Intel Xeon 8581C (Emerald Rapids)					
TW128 (AVX-512)	1.00	2.53	4.77	4.85	4.92
TW128 (AVX2)	0.82	1.70	1.71	1.81	1.85
AES-128-GCM	5.04	6.04	6.24	6.47	6.46
Apple M4 Pro					
TW128 (NEON+SHA3)	2.31	4.90	5.06	5.12	5.03
AES-128-GCM	8.17	9.68	10.18	10.05	10.20

of Table 5 bear this out: per-message latency is nearly flat across them at roughly 720 reference cycles on `amd64` (730 at 1 B versus $11.23 \times 64 \approx 719$ at 64 B) and 620 core cycles on `arm64` (620 versus $9.63 \times 64 \approx 616$), i.e. about 360 and 310 cycles per KECCAK-p[1600, 12] respectively. Because such messages bypass the leaf core entirely, leaf-level parallelism imposes no latency penalty on them.

The latency/throughput trade-off appears instead in the intermediate regime, which lane-0 fusion has narrowed to roughly the one-to-eight-chunk band. The worst per-byte cost in the sweep is the single-chunk regime at 8 KiB (1.9–2.8 cycles per byte across the three paths), where the message is the chunk-0 phase alone on the single-duplex core. Once the message has complete leaf chunks for chunk 0 to share a kernel with, the cost falls quickly: by 32 KiB TW128 is within a factor of two of its AVX-512 floor and within 5% of its `arm64` floor, and the floor itself is reached once the eight-way core fills at 64 KiB. The root duplex also bounds the tail of a long message: the final tag cannot be produced until chunk 0 has been processed and every leaf tag has been absorbed into the aggregation transcript. That absorption is cheap—each rate block of the root absorbs five 32-byte leaf tags, so a 16 MiB message ($\approx$ 2000 leaves) adds on the order of 400 root permutations against roughly 10^5 permutations of leaf work, under 0.5% overhead—so the serial root does not materially cap bulk throughput.

9.5 Comparison with Other AEADs

We compare against the one baseline measured under the same harness on the same hosts, the Go standard library's hardware-accelerated AES-128-GCM (Tables 5 and 6). The harness times TW128 and AES-128-GCM back-to-back at each length and reads both metrics from the same loop, so the cycles-per-byte and throughput comparisons hold under identical conditions and agree. In the bulk regime AES-128-GCM is faster on both architectures, as expected for a primitive with dedicated AES and carryless-multiply instructions: at 16 MiB it runs at 0.36 cycles per byte on `amd64` and 0.44 on `arm64`, against 0.47 and 0.89 for TW128—about 1.3× and 2× faster. The wall-clock throughputs show the same margins: TW128 reaches 4.92 and 5.03 GB/s against AES-128-GCM's 6.46 and 10.20 GB/s. The wider `arm64` gap reflects the strength of Apple's AES and `PMULL` units. TW128 attains these rates with a permutation that has no dedicated hardware support, relying only on the SHA-3 extension on `arm64` and on general SIMD on `amd64`.

The ordering reverses for very short messages, where AES-128-GCM's key schedule and GHASH key-power precomputation dominate: at 1 B AES-128-GCM costs 897 and 1083 cycles on the two hosts against TW128's 730 and 620. This comparison is setup-bound rather than representative of steady state, since each timed call rebuilds its cipher; it nonetheless reflects the per-message cost an application pays when it does not amortize

Table 7: Structural comparison of TW128 with representative AEADs. Sizes are in bytes; AES-GCM-SIV also admits a 32-byte key. "Misuse-res." is nonce-misuse resistance in the sense of [RS06]; the CMT-4 column records full-input commitment in the sense of [BH22], abstracting over its decryption (CMTD) and encryption (CMT) forms, of which TW128's is the decryption form CMTDs-4; CDY is context discoverability in the sense of [MLGR23]. A dash means the entry is inherited from the base scheme, and "?" that no result is established in the literature. TW128's concrete bounds appear in Section 5.6.

Scheme	Primitive	Par.	K/N/T	AD	Misuse	CMT-4	CDY
TW128	Keccak-p[1600, 12]	yes	32/32/32	yes	no	yes	yes
Ascon-128a [DEMS21]	Ascon-p (320 b)	no	16/16/16	yes	no	yes	?
Xoodyak [DHP+20]	Xoodoo (384 b)	no	16/16/16	yes	no	no	?
AES-GCM-SIV [GLL17]	AES	yes	16/12/16	yes	yes	no	?
nAE + CTX [CR22]	any nAE	—	—	yes	—	yes	?

key setup across messages. In the sub-bulk regime AES-128-GCM is several times faster at one chunk (0.46 versus 2.31 cycles per byte at 8 KiB on `amd64`), but the gap narrows quickly once the fused kernels engage: about 2.4× at 32 KiB, and the bulk ratio from 64 KiB on. TW128's intended value relative to AES-128-GCM lies in the tree structure and committing-security properties analyzed in Sections 6 to 8 rather than in raw speed.

10 Discussion

10.1 Comparison Table

Table 7 situates TW128 against representative authenticated encryption schemes along the axes that distinguish it: the underlying primitive, whether the mode is parallelizable, the key, nonce, and tag sizes, associated-data support, nonce-misuse resistance, full-input key commitment (CMT-4), and context discoverability (CDY). The comparison is structural; the concrete forms of TW128's own privacy, AE, and committing bounds, and the regimes in which each term dominates, are treated in Section 5.6. The CMT-4 column records full-input commitment: TW128 provides it within the mode, as does Ascon [KSW23], a committing transform supplies it for an arbitrary base scheme, and the remaining listed schemes do not provide it. The CDY column records context-discovery resistance: TW128 provides it within the mode (Theorem 4), and the remaining entries are marked "?" because, to our knowledge, no discoverability result—attack or proof—has been published for them. Context discoverability is a recent notion [MLGR23], so its absence of entries reflects the state of the literature rather than a negative result; the schemes [MLGR23] do break on discoverability (GCM, CCM, EAX, SIV, OCB3) are not among the rows above. The table does not separately track tag-projected compactness, since that projection strengthening has not been systematically analyzed for the comparison schemes.

10.2 Comparison with Prior Sponge AEADs

TW128 belongs to the family of single-permutation duplex AEADs initiated by [BDPVA11]. The dedicated lightweight members of that family, Ascon [DEMS21] and Xoodyak [DHP+20], target 128-bit security on small permutations (320 and 384 bits) with a strictly serial duplex: each call depends on the previous one, so there is no mode-level parallelism to exploit. TW128 instead pays for a larger permutation, Keccak-p[1600, 12], and a 256-bit capacity—the same round count as KangarooTwelve [BDP+16]—in exchange for a tree that processes leaf chunks in parallel and for the headroom that yields 128-bit security with a comfortable margin. The cost of this choice is visible in Section 9: on

short, single-leaf inputs TW128 is slower per byte than a compact serial duplex would be, and its advantage appears only once a message spans enough leaves to fill the vector units.

Keyak [BDP+15] is the closest prior duplex AEAD in spirit, since its Motorist mode is already parallelizable. As discussed in Section 1.2, the difference is that Keyak fixes its degree of parallelism as a mode parameter that the encryptor and decryptor must share, whereas in TW128 the number of leaves evaluated at once is an implementation choice invisible to the ciphertext. Strobe [Ham17] is a serial sponge framework aimed at whole protocols rather than at a standalone AEAD, and like Ascon and Xoodyak it does not parallelize. Committing security is not unique to TW128 in this family: a dedicated analysis of the sponge-based NIST lightweight finalists proves Ascon committing while breaking others [KSW23], and some constructions, such as Strobe, have not been examined in that setting at all. What distinguishes TW128 is that it targets the strongest s-way full-input notion (CMT-4) as a proven, designed property of the mode, together with its preimage counterpart, context discoverability, established without leaving the single-permutation duplex setting; the discoverability guarantee in particular is not known to hold for the other members of the family.

10.3 Comparison with Committing AEAD Constructions

The dominant route to committing AEAD is a generic transform over a non-committing base scheme. [BH22] give the UtC, RtC, and HtE transforms, which add key- or context-commitment to an arbitrary AEAD, and [CR22] give CTX, which makes any nonce-based AE scheme commit to its full context at the cost of a single hash over a short string and with no ciphertext expansion. These transforms are inexpensive and broadly applicable; CTX in particular is nearly free. TW128 differs not by undercutting their cost but by integration: it is committing as the mode itself, with the same permutation and the same tag that already provide confidentiality and integrity, so no separate commitment primitive, key-derivation step, or auxiliary hash is invoked, and the strongest s-way full-input notion follows from a single root tag collision bound (Section 8). More strongly, Theorem 3 shows that the root tag alone is a compact commitment string even when the ciphertext bodies used as openings differ, up to the additional leaf tag birthday term. The same tag, being preimage-resistant, additionally yields context discoverability against that fixed tag, which the context-committing transforms are not known to provide. This places TW128 alongside the dedicated committing primitives—the compactly committing AEAD of [GLR17] and the encryptment of [DGRW18]—which likewise build commitment in rather than bolt it on, but which were designed for message franking rather than as general nonce-based AEADs. The contrast with transformed conventional schemes is sharpened by the attacks of Section 1.2: base schemes such as GCM and OCB are not committing on their own [CR22], and a transform is the only way to make them so.

Closest to TW128 here is the recent work of [DHM+24], which builds nonce-based AE directly on the SHA-3 functions SHAKE and TurboSHAKE—the latter using the same round-reduced Keccak-p permutation as TW128—and likewise achieves CMT-4 natively, from the collision resistance of the underlying function rather than from a transform. The two designs make different structural choices: [DHM+24] targets session-based communication, chaining a stream of messages through intermediate tags and generalizing the keyed duplex into a *duplex cipher*, whereas TW128 is a single two-level tree whose independent leaves expose run-time-selectable parallelism at constant memory (Sections 1.2 and 9.1). Because a session chain processes each message serially, TW128's principal advantage is throughput on large messages (Section 9): its parallel leaves fill vector units a serial duplex leaves idle, at the cost of the session support [DHM+24] provides and TW128 does not. Both demonstrate that committing AE needs no add-on transform once it is built on a well-scrutinized Keccak-p permutation.

10.4 Limitations and Open Problems

Nonce misuse. TW128's privacy analysis requires a nonce-respecting adversary, and the mode is not misuse-resistant: a repeated nonce reuses leaf keystream and breaks confidentiality, as for any online duplex stream cipher. Schemes such as AES-GCM-SIV [GLL17], in the deterministic/misuse-resistant lineage of [RS06], tolerate nonce repetition at the price of a second pass. A misuse-resistant variant of TW128 is left open.

Release of unverified plaintext. The analysis assumes that plaintext is released only after the tag verifies. TW128 is online and single-pass, so an implementation that streams decrypted leaf output before checking the root tag operates in the release-of-unverified-plaintext setting of [ABL+14], which our bounds do not cover; plaintext-awareness and INT-RUP security for TW128 are unstudied.

Bound tightness. Two terms are knowingly conservative. The flat-sponge differentiating loss $\mathsf{Lift}_c(N_{\mathsf{tot}}) = N_{\mathsf{tot}}(N_{\mathsf{tot}}+1)/2^{c+1}$ bounds the indifferentiability loss of [BDPVA08], used here as a generic lift from the random-oracle model. A direct analysis of the keyed duplex is tighter: the modern keyed-duplex bounds synthesized by [Men23], building on the full-state keyed-duplex line, bound a keyed duplex against an ideal permutation without passing through indifferentiability, and recasting TW128's leaf and root duplexes in that framework is a promising route to a sharper capacity term. This route is not turnkey, however: the keyed duplex bundles squeezing and absorption into a single call, so its authenticated-encryption mode MonkeySpongeWrap has the decryptor *overwrite* the rate with each ciphertext block rather than feed back the ciphertext it has just produced. On the final, padded block this overwrite no longer matches the encryptor's XOR-absorbed padded plaintext outside the message bits, so the decryptor cannot reconstruct the final block of duplex output and recomputes the tag from a divergent state [Men23, Alg. 8]. TW128 sidesteps this by absorbing the ciphertext in both directions (Section 3), so recasting it on the keyed duplex would first require repairing the mode. We retain the indifferentiability lift for its modularity, as the resulting Lift_c already meets the 128-bit target (Section 5.6). The leaf tag collision term $N_{\mathsf{leaf}}(N_{\mathsf{leaf}}-1)/2^{\tau_{\mathsf{leaf}}+1}$ is a pairwise union bound whose necessity for the AE guarantee, as opposed to an artifact of the accounting, is not settled.

Beyond-birthday and leakage. All bounds are birthday-type on the 256-bit capacity, delivering the 128-bit target but no more; a beyond-birthday analysis is open. The model is also single-stage and leakage-free: side-channel resistance is addressed only at the implementation level, through the constant-time properties of Section 9.1, and is not part of the provable-security claims.

11 Conclusion

TW128 is a two-level tree of keyed duplexes over a single KECCAK-p[1600, 12] permutation that combines mode-level parallelism, constant memory, and commitment security in both its collision and preimage facets. Its concrete public-permutation bounds—for all-in-one AE, multi-user indistinguishability, s-way full-input committing security (CMTDs-4), the tag-projected strengthening in which the root tag alone is the compact commitment, and tag-projected context discoverability—meet a 128-bit target at the implemented parameters, and vectorized `amd64` and `arm64` implementations turn the tree's leaf parallelism into multi-gigabyte-per-second throughput within a small factor of hardware-accelerated AES-128-GCM.

Several questions remain open (Section 10.4): a nonce-misuse-resistant variant, security under the release of unverified plaintext, and sharper bounds—whether by recasting the

leaf and root duplexes on the modern keyed-duplex analysis or through a beyond-birthday argument. More broadly, TW128 adds to the growing evidence that a single, well-scrutinized KECCAK-p permutation suffices to build authenticated encryption that is at once fast, parallel, and committing.

Acknowledgments

A hearty thank-you to my wife, who remains reasonably mystified at my choice of hobbies.

References

[ABL+14] Elena Andreeva, Andrey Bogdanov, Atul Luykx, Bart Mennink, Nicky Mouha, and Kan Yasuda. How to securely release unverified plaintext in authenticated encryption. In *Advances in Cryptology – ASIACRYPT 2014*, volume 8873 of *Lecture Notes in Computer Science*, pages 105–125. Springer, 2014.

[ADG+22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. How to abuse and fix authenticated encryption without key commitment. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3291–3308. USENIX Association, 2022.

[BDP+15] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. CAESAR submission: Keyak v2. CAESAR Competition, Round 3 Candidate, 2015.

[BDP+16] Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, Ronny Van Keer, and Benoît Viguier. KangarooTwelve: Fast hashing based on Keccak-p. Cryptology ePrint Archive, Paper 2016/770, 2016.

[BDPVA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.

[BDPVA11] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplexing the sponge: Single-pass authenticated encryption and other applications. In *Selected Areas in Cryptography – SAC 2011*, volume 7118 of *Lecture Notes in Computer Science*, pages 320–337. Springer, 2011.

[BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, volume 13276 of *Lecture Notes in Computer Science*, pages 845–875. Springer, 2022.

[BN00] Mihir Bellare and Chanathip Namprempre. Authenticated encryption: Relations among notions and analysis of the generic composition paradigm. In *Advances in Cryptology – ASIACRYPT 2000*, volume 1976 of *Lecture Notes in Computer Science*, pages 531–545. Springer, 2000.

[BT16] Mihir Bellare and Björn Tackmann. The multi-user security of authenticated encryption: AES-GCM in TLS 1.3. In *Advances in Cryptology – CRYPTO 2016*, volume 9814 of *Lecture Notes in Computer Science*, pages 247–276. Springer, 2016.

[CR22] John Chan and Phillip Rogaway. On committing authenticated encryption. In *Computer Security – ESORICS 2022*, Lecture Notes in Computer Science. Springer, 2022.

[DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schläffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.

[DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryptment. In *Advances in Cryptology – CRYPTO 2018*, volume 10991 of *Lecture Notes in Computer Science*, pages 155–186. Springer, 2018.

[DHM+24] Joan Daemen, Seth Hoffert, Silvia Mella, Gilles Van Assche, and Ronny Van Keer. Shaking up authenticated encryption. Cryptology ePrint Archive, Paper 2024/1618, 2024.

[DHP+20] Joan Daemen, Seth Hoffert, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer. Xoodyak, a lightweight cryptographic scheme. *IACR Transactions on Symmetric Cryptology*, 2020(S1):60–87, 2020.

[GLL17] Shay Gueron, Adam Langley, and Yehuda Lindell. AES-GCM-SIV: Specification and analysis. Cryptology ePrint Archive, Paper 2017/168, 2017.

[GLR17] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. Message franking via committing authenticated encryption. In *Advances in Cryptology – CRYPTO 2017*, volume 10403 of *Lecture Notes in Computer Science*, pages 66–97. Springer, 2017.

[Ham17] Mike Hamburg. The STROBE protocol framework. Cryptology ePrint Archive, Paper 2017/003, 2017.

[IR23] Takanori Isobe and Mostafizar Rahman. Key committing security analysis of AEGIS. Cryptology ePrint Archive, Paper 2023/1495, 2023.

[KSW23] Juliane Krämer, Patrick Struck, and Maximiliane Weishäupl. Committing AE from sponges: Security analysis of the NIST LWC finalists. Cryptology ePrint Archive, Paper 2023/1525, 2023.

[LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 195–212. USENIX Association, 2021.

[Men23] Bart Mennink. Understanding the duplex and its security. *IACR Transactions on Symmetric Cryptology*, 2023(2):1–46, 2023.

[MLGR23] Sanketh Menda, Julia Len, Paul Grubbs, and Thomas Ristenpart. Context discovery and commitment attacks: How to break CCM, EAX, SIV, and more. In *Advances in Cryptology – EUROCRYPT 2023*, volume 14007 of *Lecture Notes in Computer Science*, pages 379–407. Springer, 2023.

[Nat15] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Federal Information Processing Standards Publication FIPS PUB 202, U.S. Department of Commerce, August 2015.

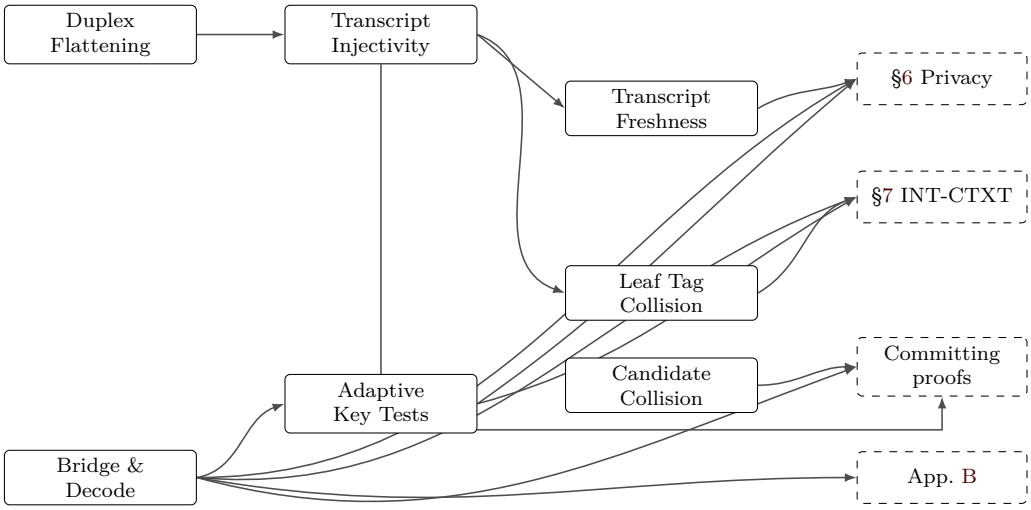

Figure 2: Dependency graph of the supporting lemmas in this appendix and their consumers. Solid boxes are lemmas; dashed boxes are the proof sections that consume them. The edge from Lemma 5 into the committing proofs of Sections C and 8 is mediated by Corollary 2 for final-root distinctness and Corollary 3 for candidate-sequence filtering; the edge from Lemma 7 records the collision accounting used by the CMTDs-3 and CMTs proofs. The edge from Lemma 5 into Lemma 11 uses Corollary 3; the verification-first root tag guessing accounting reached through the same separation is carried out in Section A.10 (Lemma 15) and consumed by Section 7. The Lemma 9 edge records the shared $\mathsf{bad}_{\mathsf{key}}$ accounting used by privacy and INT-CTXT.

[RS06] Phillip Rogaway and Thomas Shrimpton. A provable-security treatment of the key-wrap problem. In *Advances in Cryptology – EUROCRYPT 2006*, volume 4004 of *Lecture Notes in Computer Science*, pages 373–390. Springer, 2006.

[WP14] Hongjun Wu and Bart Preneel. AEGIS: A fast authenticated encryption algorithm. In *Selected Areas in Cryptography – SAC 2013*, volume 8282 of *Lecture Notes in Computer Science*, pages 185–201. Springer, 2014.

A Supporting Lemmas

The proof core separates structural decoding from probabilistic accounting. The well-formed transcript language of Definition 1 fixes which flattened queries are valid TW128 construction prefixes. Lemmas 4 to 6 then show how such a query is flattened, parsed as a typed transcript, assigned an output point, and decoded to a candidate key, when any. Lemma 9 turns those decoded keys into an adaptive key-test bound. The remaining proof interfaces expose the events and stopped-game equalities their consumers must control: privacy freshness and hybrid replacement, INT-CTXT hidden-key exposure and root tag guessing, leaf tag collisions, and the committing candidate-sequence collision event. Thus the privacy, INT-CTXT, and committing proofs only have to derive the relevant local event premise before applying the corresponding accounting argument.

A.1 Duplex Flattening

Lemma 4 (Duplex Flattening)**.** *Every TW128 root or leaf duplex output is equal to a [BDPVA11] sponge output on the corresponding flattened duplex-call transcript, and the ciphertext-absorbing message phase admits a flattened duplex transcript that is well defined for both encryption and decryption.*

Proof. Let D be a duplex object using permutation π, rate r, and padding $\mathsf{pad10^*1}$. For a sequence of duplex calls $Z_i = \mathsf{D.duplexing}(\sigma_i, \ell_i)$ with each σ_i short enough to fit in one duplex block after padding, [BDPVA11, Lemma 3] gives

$$Z_i = \mathsf{Sponge}[\pi, \mathsf{pad10^*1}, r]\big(\sigma_0 \parallel \mathsf{pad}_0 \parallel \sigma_1 \parallel \mathsf{pad}_1 \parallel \cdots \parallel \sigma_i,\ \ell_i\big),$$

where $\mathsf{pad}_j = \mathsf{pad10^*1}[r](|\sigma_j|)$, and Lemma 4 says that, for fixed $\mathsf{pad10^*1}$ and r, the map from the duplex-call sequence to the flattened sponge input is injective.

In TW128, each duplex call absorbs a byte-aligned block σ followed by a 4-bit phase suffix $\sigma_{\mathsf{ph}} \in \{\sigma_{\mathsf{init}}, \sigma^-_{\mathsf{ad}}, \ldots, \sigma^+_{\mathsf{agg}}\}$ (Section 3.4), so the duplex-call input is $\sigma \parallel \sigma_{\mathsf{ph}}$ followed by $\mathsf{pad10^*1}$. With $r = 1344$ and $\rho_{\max}(\mathsf{pad10^*1}, r) = r - 2 = 1342$ bits, the maximum byte-aligned input is $\rho = 167$ bytes (Section 3.2), giving a worst-case duplex-call input of $8{\cdot}167{+}4 = 1340 \le 1342$ bits. The two initialization calls are shorter still: $|p_{\mathsf{root}} \| K \| N| {+}4 = 8 \cdot (6 + 32 + 32) + 4 = 564$ bits and $|p_{\mathsf{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8| + 4 = 8 \cdot (6 + 32 + 32 + 8) + 4 = 628$ bits. [BDPVA11, Lemmas 3 and 4] therefore apply independently to every TW128 duplex object.

The root duplex is initialized by absorbing $p_{\mathsf{root}} \parallel K \parallel N$ under σ_{init}(Algorithm 2), then absorbs the associated-data blocks under $\sigma^-_{\mathsf{ad}}/\ \sigma^+_{\mathsf{ad}}$, the root ciphertext blocks in the message phase under $\sigma^-_{\mathsf{msg}}/\ \sigma^+_{\mathsf{msg}}$, and the leaf tag aggregation blocks under $\sigma^-_{\mathsf{agg}}/\ \sigma^+_{\mathsf{agg}}$. By the duplex-to-sponge equality, every root duplex output is the sponge function evaluated on the root transcript prefix accumulated up to the relevant output point. The same holds for each leaf duplex, initialized by $p_{\mathsf{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$ and absorbing only its own ciphertext blocks in the message phase. In decryption and verification, Algorithm 3 absorbs the submitted ciphertext blocks before the root tag comparison, so the same flattened message-phase transcript is defined whether the comparison accepts or rejects. □

A.2 Well-formed Transcript Prefixes

For a duplex-call prefix $((\sigma_0, \sigma_{\mathsf{ph},0}), \ldots, (\sigma_i, \sigma_{\mathsf{ph},i}))$, write

$$\mathsf{Flat}\big((\sigma_0, \sigma_{\mathsf{ph},0}), \ldots, (\sigma_i, \sigma_{\mathsf{ph},i})\big) = \Big\|_{j=0}^{i-1} \big(\sigma_j \parallel \sigma_{\mathsf{ph},j} \parallel \mathsf{pad}_j\big) \parallel \sigma_i \parallel \sigma_{\mathsf{ph},i}.$$

The current call's padding pad_i is not part of Flat; it is applied internally by the sponge function. Equivalently, after the i-th call the duplex state is the sponge state after absorbing $\mathsf{Flat}(\cdot) \parallel \mathsf{pad}_i$.

Definition 1 (Well-Formed TW128 Transcript Prefixes)**.** A typed TW128 transcript prefix is *well formed* if it is a prefix, ending at a construction transcript lookup point, of one of the following two call languages.

- A root transcript starts with $(p_{\mathsf{root}} \parallel K \parallel N, \sigma_{\mathsf{init}})$, with fixed-width fields $K \in \{0,1\}^k$ and $N \in \{0,1\}^\nu$. It then absorbs the associated-data byte string (only when the associated data is nonempty; the phase is otherwise elided, Section 3.5), the root ciphertext byte string, and the aggregation byte string (only when $n \ge 1$; when $n = 0$ the aggregation phase is elided, Section 3.5, and the root tag is emitted by the final σ^+_{msg} message call), in that order. Each present phase byte string is partitioned

by the block rule of Algorithm 1: the empty string gives one empty final block, and a nonempty string gives zero or more ρ-byte non-final blocks followed by one final block. The phases use respectively $\sigma^-_{\mathsf{ad}}/\sigma^+_{\mathsf{ad}}$, $\sigma^-_{\mathsf{msg}}/\sigma^+_{\mathsf{msg}}$, and $\sigma^-_{\mathsf{agg}}/\sigma^+_{\mathsf{agg}}$.

- A leaf-j transcript starts with $(p_{\mathsf{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8, \sigma_{\mathsf{init}})$, where $1 \le j < 2^{64}$ and the fields have fixed width. It then absorbs the leaf ciphertext byte string under $\sigma^-_{\mathsf{msg}}/\sigma^+_{\mathsf{msg}}$, using the same block rule.

The message byte string in this language is always the ciphertext byte string absorbed by the construction: encryption obtains it by XORing the plaintext with the current keystream, while decryption and verification use the submitted ciphertext. Thus a verification transcript is well formed for every in-domain ciphertext before the tag comparison is made; acceptance is not part of the transcript language.

The root aggregation byte string has the form $T_1 \parallel \cdots \parallel T_n \parallel \langle n \rangle_8$, where each T_j has fixed byte length $\tau_{\mathsf{leaf}}/8$. The trailing fixed-width field therefore gives a unique parse of the leaf tag sequence. The output-point class of a well-formed counted prefix is the class in Table 8 determined by its duplex instance and final-call suffix.

Name the tag-emitting transcript families:

$$\mathcal{R}_{\mathsf{tag}} = \text{root transcript prefix whose sponge output gives the final root tag,}$$
$$\mathcal{L}_{\mathsf{tag}}[j] = \text{leaf-}j\text{ transcript prefix whose sponge output gives the leaf tag.}$$

The final root tag is emitted by the σ^+_{agg} aggregation call (the $\mathcal{R}_{\mathsf{tag}}$ family) when $n \ge 1$, and by the closing σ^+_{msg} message call (an $\mathcal{R}^+_{\mathsf{msg}}$ prefix, in the notation of Corollary 3) when $n = 0$ and the aggregation phase is elided. Since $n = \mathrm{leaves}(C)$ is fixed by $\|C\|$, every computation on a given ciphertext emits its tag at the one of these two output points selected by $\|C\|$. Intermediate construction transcript lookup points (init, non-final AD, non-final message, and non-final aggregation) are also typed prefixes covered by Lemma 6; the exact keyed direct-query decoder names them in Table 8.

A.3 Transcript Injectivity

Lemma 5 (Transcript Injectivity)**.** *Well-formed TW128 flattened duplex-call transcript prefixes are injectively encoded and separated by duplex instance (root vs leaf), leaf index j, phase suffix, and duplex-call order. For final-root prefixes, equality of flattened inputs recovers the same root initialization, AD blocks, root ciphertext blocks, aggregation input, and leaf tag sequence. When the compared final-root prefixes are construction-generated, if no two distinct construction-generated final leaf transcripts produce the same leaf tag, equality also recovers the same leaf transcript sequence and therefore the same full absorbed ciphertext.*

Proof. By Lemma 4, every well-formed output point can be written as a [BDPVA11] flattened sponge input for the corresponding duplex-call prefix, namely Flat as defined above. The deterministic current-call pad10*1 is applied internally by the sponge function and is not part of the input to which [BDPVA11, Lemma 4] is applied. That lemma recovers the duplex-call sequence from the flattened sponge input; in TW128, each call input is $\sigma \parallel \sigma_{\mathsf{ph}}$ with byte-aligned σ and fixed 4-bit suffix σ_{ph}, so recovering the call input recovers the pair $(\sigma, \sigma_{\mathsf{ph}})$.

Definition 1 fixes the root and leaf initialization fields, the admissible phase order, and the final-call suffixes. The distinct prefixes p_{root} and p_{leaf} separate root from leaf transcripts; the fixed-width leaf field $\langle j \rangle_8$ separates leaves; and the seven suffix values of Table 3 separate phases and non-final/final calls. Thus equality of flattened inputs forces equality of the recovered instance type, key, nonce, leaf index when present, phase suffixes, and duplex-call order.

Phase-recovery sub-claim. For each present phase $P \in \{\mathsf{ad}, \mathsf{msg}, \mathsf{agg}\}$ (the ad phase is present exactly when the associated data is nonempty; see Section 3.5), TW128 absorbs the phase's input byte string s_P by partitioning into ρ-byte blocks (with the empty case yielding a single empty block, per Section 3.5) and feeding each block to the duplex with suffix σ_P^- (non-final) or σ_P^+ (final). Lemma 4 and [BDPVA11, Lemma 4] recover the corresponding $(\sigma, \sigma_{\mathsf{ph}})$ sequence from the flattened input. Because the partition map is deterministic and uniquely invertible on its image (the empty case yielding the single empty block carrying σ_P^+), concatenating the recovered σ blocks reconstructs s_P exactly. The map from s_P to the phase's $(\sigma, \sigma_{\mathsf{ph}})$ sequence is therefore injective.

Applied phase by phase to a well-formed final-root transcript:

- AD. The associated-data phase contributes a σ_{ad}^+-tagged block exactly when the associated data is nonempty; an empty associated data elides the phase and contributes no block. When the phase is present the sub-claim recovers the AD byte string. Thus distinct associated-data strings $A_i \neq A_j$ are separated either by the presence versus absence of the AD phase (when exactly one is empty) or, when both are nonempty, by their distinct AD σ-block sequences.

- MSG. Distinct absorbed root or leaf ciphertext byte strings produce distinct MSG σ-block sequences; the message language of Definition 1 is ciphertext-absorbing for encryption and decryption alike.

- AGG. The aggregation phase is present exactly when $n \geq 1$ (Section 3.5). When it is present, the sub-claim applied to the aggregation byte string $T_1 \,\|\, \cdots \,\|\, T_n \,\|\, \langle n \rangle_8$ recovers that byte string. The final eight bytes give $\langle n \rangle_8$, and each leaf tag has fixed byte length $\tau_{\mathsf{leaf}}/8$, so the byte string has a unique parse as a leaf tag sequence. Two root aggregation encodings are therefore equal exactly when their leaf tag sequences are equal. Under the structural condition that distinct construction-generated final leaf transcripts receive distinct leaf tag values, equality of leaf tag sequences implies equality of the corresponding final leaf transcripts. When $n = 0$ the aggregation phase is absent and the final-root transcript ends at the recovered σ_{msg}^+ root message call; the MSG bullet recovers the root ciphertext byte string, and the absence of any $\sigma_{\mathsf{agg}}^-/\sigma_{\mathsf{agg}}^+$ call distinguishes such a prefix from every $n \geq 1$ final-root prefix.

Therefore the recovered final-root transcript determines the root initialization, AD blocks, root ciphertext blocks, and, when $n \geq 1$, the aggregation byte string and leaf tag sequence. For construction-generated final-root prefixes with $n \geq 1$, under the stated leaf tag structural condition, the leaf tag sequence determines the corresponding final leaf transcripts as well, so the same full absorbed ciphertext is recovered; for $n = 0$ the root ciphertext blocks already constitute the full absorbed ciphertext. □

Bridge injectivity on transcript prefixes. For TW128, the [BDPVA11, Theorem 3] bridge has $r = r_{\max}$ and maps a native flattened input U to $U \,\|\, 1 \,\|\, 0^q$, where $q = (-|U| - 2) \bmod r$. Any string in this image has a unique inverse: q is the number of trailing zero bits, the preceding bit is the inserted 1, and deleting this suffix recovers U. Thus equality of bridged images $\mathsf{flat}(X) = \mathsf{flat}(X')$ for well-formed typed TW128 prefixes implies equality of their native flattened inputs, after which Lemma 5 gives the same recovery consequences as above.

Corollary 2 (Opening Triple Separation)**.** *If two TW128 root computations have distinct (K, N, A) triples, then their bridged final root transcript inputs under $\mathsf{flat}(\cdot)$ are distinct, regardless of whether each computation is an encryption or a decryption check on a common ciphertext.*

Proof. Suppose for contradiction that $\mathsf{flat}(R_i) = \mathsf{flat}(R_j)$. By the bridge-injectivity observation above, the native flattened inputs of the two final-root prefixes are equal. Then Lemma 5 recovers the full duplex-call sequence and reads off the pair $(\sigma, \sigma_{\mathsf{ph}})$ for each call. The seven 4-bit phase suffixes are pairwise distinct, so the σ_{init}, $\sigma^{\pm}_{\mathsf{ad}}$, $\sigma^{\pm}_{\mathsf{msg}}$, and (when present) $\sigma^{\pm}_{\mathsf{agg}}$ calls in the recovered sequence are unambiguously identified. The aggregation phase is present exactly when $n \geq 1$, and $n = \mathrm{leaves}(C)$ is determined by the common ciphertext, so the two recovered sequences either both carry an aggregation phase or both omit it; either way the recovery of (K, N, A) below uses only the $\sigma_{\mathsf{init}}$and AD calls, which precede any message or aggregation block.

Equality of the recovered sequences forces equality of the σ_{init}call, whose σ is $p_{\mathsf{root}} \| K \| N$ with fixed-width K and N, hence $K_i = K_j$ and $N_i = N_j$. It also forces equality of the recovered AD subsequences. By the AD bullet of Lemma 5 the associated-data phase contributes a σ^{+}_{ad}-tagged block exactly when the associated data is nonempty, so the recovered sequences agree on the presence of the AD phase. If exactly one of A_i, A_j were empty the recovered sequences would differ in the presence of a σ^{+}_{ad} call, contradicting their equality; hence either both are empty, giving $A_i = A_j = \varepsilon$, or both are nonempty, in which case the phase-recovery sub-claim of Lemma 5 inverts the ρ-byte-block partition map on the AD byte string and equal AD σ-block sequences imply $A_i = A_j$. In all cases $(K_i, N_i, A_i) = (K_j, N_j, A_j)$, contradicting the hypothesis of distinct triples. □

Corollary 3 (Output-Point Separation)**.** *Every well-formed TW128 typed transcript prefix at a counted construction transcript lookup point belongs to exactly one of the following output-point classes, identified by the recovered duplex instance and the* 4*-bit suffix of its final call:*

- $\mathcal{R}_{\mathsf{init}}$*(root duplex, suffix* σ_{init}*),*
- $\mathcal{L}_{\mathsf{init}}(j)$ *(leaf-*j *duplex, suffix* σ_{init}*),*
- $\mathcal{R}^{-}_{\mathsf{ad}}$ */* $\mathcal{R}^{+}_{\mathsf{ad}}$ *(root duplex, suffix* σ^{-}_{ad} *or* σ^{+}_{ad}*),*
- $\mathcal{R}^{-}_{\mathsf{msg}}$ */* $\mathcal{R}^{+}_{\mathsf{msg}}$ *(root duplex, suffix* $\sigma^{-}_{\mathsf{msg}}$ *or* $\sigma^{+}_{\mathsf{msg}}$*),*
- $\mathcal{L}^{-}_{\mathsf{msg}}(j)$ *(leaf-*j *duplex, suffix* $\sigma^{-}_{\mathsf{msg}}$*),*
- $\mathcal{L}_{\mathsf{tag}}[j]$ *(leaf-*j *duplex, suffix* $\sigma^{+}_{\mathsf{msg}}$*),*
- $\mathcal{R}_{\mathsf{agg}}$*(suffix* $\sigma^{-}_{\mathsf{agg}}$*),*
- $\mathcal{R}_{\mathsf{tag}}$ *(root duplex, suffix* $\sigma^{+}_{\mathsf{agg}}$*).*

For any two such prefixes whose bridged flattened inputs are equal, the recovered duplex-call sequences are equal, and therefore:

(i) the prefixes have the same (K, N) *pair, recovered from the first call's* σ*;*

(ii) the prefixes have the same instance type (root or leaf), separated by p_{root} *vs* p_{leaf} *in the first call's* σ*;*

(iii) within leaf instances, the prefixes have the same chunk index j*, separated by the fixed-width* $\langle j \rangle_8$ *field;*

(iv) the prefixes have the same output-point class, since the seven phase suffixes of the final call are pairwise distinct;

(v) within a single output-point class on the same duplex instance, the prefixes have the same duplex-call sequence, since two distinct prefixes in the same class differ in the contents of some recovered call.

Equivalently, any two distinct counted transcript prefixes are separated by the first of (i)–(v) that applies.

Proof. Items (i)–(iv) restate the recovery established by Definition 1 and Lemma 5: bridge injectivity first recovers equality of native flattened inputs from equality of bridged inputs, and then Lemma 5 recovers the duplex-call sequence. The first call's σ has the form $p_{\mathsf{root}} \,\|\, K \,\|\, N$ for root instances and $p_{\mathsf{leaf}} \,\|\, K \,\|\, N \,\|\, \langle j \rangle_8$ for leaf instances, with fixed-width K, N, and $\langle j \rangle_8$; the seven phase suffixes are pairwise distinct. Item (v) is the contrapositive: two distinct well-formed prefixes in the same class on the same duplex instance differ in some recovered call's σ or in the sequence length, and therefore in their flattened inputs. □

A.4 Bridge and Typed Decoding

Lemma 6 (Bridge and Typed Decoding)**.** *The random-oracle games used in Sections C and 6 to 8 are ordinary random-oracle games on a single random oracle* $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ *over the unpadded [BDPVA08] H-input domain (Section 4.2), with TW128 root and leaf duplex outputs answered by the typed restriction*

$$\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X)), \qquad \mathsf{flat}(X) = I[r, r_{\max}]\big(\mathsf{raw\_flat}(X)\big),$$

for $r = r_{\max} = 1344$*. Here* $\mathsf{raw\_flat}(X)$ *is the native flattened input* $\mathsf{Flat}(\cdot)$ *of Section A.2 applied to the duplex-call sequence of* X*. The bridge* $I[r, r_{\max}]$ *is the [BDPVA11] Theorem 3 preprocessing map, and the induced map* $X \mapsto \mathsf{flat}(X)$ *is injective on well-formed typed TW128 duplex transcript prefixes. The exact decoder* KeyedTWOut *defined below maps any direct-query input to either* $\perp$ *or a unique candidate key and counted output-point class.*

Proof. The bridge $I[r, r_{\max}]$ is described by the proof of [BDPVA11, Theorem 3] in three steps: (1) pad the input with $\mathsf{pad10^*1}[r]$; (2) for each r-bit block, append $r_{\max} - r$ zero bits; (3) strip the trailing $\mathsf{pad10^*}$ from the result. The $\mathsf{pad10^*1}$ padding in step (1) appends a 1, then $q = (-|\mathsf{raw\_flat}(X)| - 2) \bmod r$ zero bits, then a closing 1; the $\mathsf{pad10^*}$-unpadding in step (3) strips the $r_{\max} - r$ trailing zero bits introduced by step (2) together with the closing 1 of step (1). For TW128 with $r = r_{\max}$, step (2) appends an empty string and step (3) strips zero trailing zeros and the closing 1, so the net effect of steps (1) and (3) is to append $1 \,\|\, 0^q$. The bridge-injectivity paragraph above gives the direct inverse on this image: count the trailing zeros, delete them and the preceding inserted 1, and recover the native flattened input.

Injectivity of flat on well-formed typed TW128 prefixes then follows from Lemma 5: distinct well-formed typed prefixes have distinct $\mathsf{raw\_flat}(\cdot)$ images, and the bridge is injective on that image. Direct adversary queries on inputs outside the bridged TW128 well-formed transcript language still count in $q_{\mathsf{RO}}(\mathcal{B})$ (Section 4.3) but cannot trigger $\mathsf{bad}_{\mathsf{key}}$.

Counted transcript lookup points are the well-formed prefixes of Definition 1 whose classes are listed in Table 8: root or leaf initialization, root AD blocks, root and leaf message blocks, root aggregation blocks, and the final root tag point. Corollary 3 separates these output points, so an input accepted by KeyedTWOut fixes at most one candidate key and one output-point class, and inputs outside this well-formed transcript language decode to none. □

Exact keyed decoder.

For a direct query to $\mathsf{RO}_{\mathsf{flat}}$ with input Y and finite length ℓ, define $\mathsf{KeyedTWOut}(Y)$ as a partial map from $\{0,1\}^*$ to $\{0,1\}^k \times \mathcal{C}_{\mathsf{out}}$. The map ignores ℓ. It returns $(K, \mathcal{C})$ iff there exists a typed TW128 duplex-call prefix X such that $Y = \mathsf{flat}(X)$, the raw flattened prefix of X parses as a well-formed TW128 transcript prefix in the class $\mathcal{C}$ listed

in Table 8, and the first call of X contains the candidate key K in the fixed-width root field $p_{\mathsf{root}} \,\|\, K \,\|\, N$ or leaf field $p_{\mathsf{leaf}} \,\|\, K \,\|\, N \,\|\, \langle j \rangle_8$. If no such prefix exists, $\mathsf{KeyedTWOut}(Y) = \perp$. In particular, inputs with the right final suffix but a malformed phase order, noncanonical block partition, malformed aggregation string, or out-of-range leaf index are outside the well-formed language and decode to $\perp$.

Table 8: Counted keyed transcript classes for KeyedTWOut.

Class $\mathcal{C}$	Instance	Final call / role
$\mathcal{R}_{\mathsf{init}}$	root	σ_{init}, root initialization
$\mathcal{R}^{-}_{\mathsf{ad}}$	root	σ^{-}_{ad}, non-final AD block
$\mathcal{R}^{+}_{\mathsf{ad}}$	root	σ^{+}_{ad}, final AD block
$\mathcal{R}^{-}_{\mathsf{msg}}$	root	$\sigma^{-}_{\mathsf{msg}}$, non-final root message block
$\mathcal{R}^{+}_{\mathsf{msg}}$	root	$\sigma^{+}_{\mathsf{msg}}$, final root message block
$\mathcal{R}_{\mathsf{agg}}$	root	$\sigma^{-}_{\mathsf{agg}}$, non-final aggregation block
$\mathcal{R}_{\mathsf{tag}}$	root	$\sigma^{+}_{\mathsf{agg}}$, final aggregation / root tag
$\mathcal{L}_{\mathsf{init}}(j)$	leaf j	σ_{init}, leaf initialization
$\mathcal{L}^{-}_{\mathsf{msg}}(j)$	leaf j	$\sigma^{-}_{\mathsf{msg}}$, non-final leaf message block
$\mathcal{L}_{\mathsf{tag}}[j]$	leaf j	$\sigma^{+}_{\mathsf{msg}}$, final leaf message block / leaf tag

The table is a transcript-lookup table, not an observation-length table. A class is counted even when the construction's requested output length at that call is zero, including root initialization, non-final AD and aggregation calls, a final AD call whose following root chunk requests no keystream bytes, and the final root message call. A direct query triggers the same decoder regardless of ℓ: short finite-output queries and zero-length queries $(Y, 0)$ still count as direct queries to the address Y. Leaves have no AD or aggregation classes; a leaf transcript enters the table only through leaf initialization, non-final leaf message blocks, or the final leaf tag point. When the associated data is empty the root transcript likewise has no $\mathcal{R}^{-}_{\mathsf{ad}}$ or $\mathcal{R}^{+}_{\mathsf{ad}}$ class (Section 3.5); the $\mathcal{R}_{\mathsf{init}}$class then supplies the first root keystream block, exactly as $\mathcal{L}_{\mathsf{init}}(j)$ supplies the first leaf keystream block, and the decoder still ignores ℓ. Symmetrically, when $n = 0$ the root transcript has no $\mathcal{R}_{\mathsf{agg}}$or $\mathcal{R}_{\mathsf{tag}}$ class (Section 3.5); the final root message call $\mathcal{R}^{+}_{\mathsf{msg}}$ then supplies the root tag, exactly as $\mathcal{L}_{\mathsf{tag}}[j]$ supplies a leaf tag, and the decoder again ignores ℓ. An $n \geq 1$ computation also reaches an $\mathcal{R}^{+}_{\mathsf{msg}}$ point when closing chunk 0, but there the construction requests zero output and continues into the aggregation phase, so the $\mathcal{R}^{+}_{\mathsf{msg}}$ value is exposed as a tag only on the $n = 0$ path; since $n = \mathrm{leaves}(C)$ is fixed by $\|C\|$, the two uses never coincide on a single ciphertext.

A.5 Adaptive Candidate Collisions

Lemma 7 (Adaptive Candidate Collision)**.** *Fix $s \geq 2$, a projection length τ, and a cap Q. Consider any interaction with a lazy random oracle $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ that builds a sequence of distinct* candidate *inputs $Y_1, \ldots, Y_m$ with $m \leq Q + s$. The process may make arbitrary non-candidate oracle queries and may choose each candidate adaptively from the full prior transcript. Whenever a new candidate Y_j is appended, two premises hold:* predictability*—both the decision to append and the string Y_j are determined by the transcript preceding the append step—and* unrevealed value*—no oracle lookup before the append step has requested a positive number of bits of $\mathsf{RO}_{\mathsf{flat}}(Y_j)$; zero-length lookups at Y_j are permitted. Let $\mathsf{coll}_{s,\tau}$ be the event that some s distinct candidates have equal τ-bit projections:*

$$\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{i_1}) \right\rfloor_\tau = \cdots = \left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_{i_s}) \right\rfloor_\tau \quad \text{for some } 1 \leq i_1 < \cdots < i_s \leq m.$$

Then

$$\Pr[\mathsf{coll}_{s,\tau}] \le \binom{Q+s}{s} \cdot 2^{-\tau(s-1)}.$$

Proof. For a fixed position set $I = \{i_1 < \cdots < i_s\} \subseteq \{1, \ldots, Q+s\}$, let E_I be the event that these candidate positions exist and their τ-bit projections are all equal. Expose the interaction in chronological order. Condition on any positive-probability prefix immediately before candidate position i_r is appended, for $r \ge 2$, and on the values sampled at the earlier candidate positions in I. By the predictability premise, that prefix determines Y_{i_r}; by the unrevealed-value premise, no bit of $\mathsf{RO}_{\mathsf{flat}}(Y_{i_r})$ has been revealed in it, so under lazy sampling the value of $\mathsf{RO}_{\mathsf{flat}}(Y_{i_r})$ may be sampled after the prefix is fixed and is uniform and independent of the conditioning. Its τ-bit projection therefore equals the already fixed projection of Y_{i_1} with probability $2^{-\tau}$. Multiplying over $r = 2, \ldots, s$ gives

$$\Pr[E_I] \le 2^{-\tau(s-1)}.$$

If some position in I is never appended, then E_I is false, so the same bound applies. There are $\binom{Q+s}{s}$ possible position sets, and a union bound over them gives the claim. □

Lemma 8 (Adaptive Candidate Preimage)**.** *Fix a projection length τ and a cap Q, and a target $t \in \{0,1\}^\tau$ chosen independently of $\mathsf{RO}_{\mathsf{flat}}$. Consider any interaction with a lazy random oracle $\mathsf{RO}_{\mathsf{flat}} : \{0,1\}^* \to \{0,1\}^\infty$ that builds a sequence of distinct* candidate *inputs $Y_1, \ldots, Y_m$ with $m \le Q+1$ under the same* predictability *and* unrevealed-value *premises as Lemma 7. Let $\mathsf{pre}_{\tau,t}$ be the event that some candidate's τ-bit projection equals t:*

$$\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = t \quad \text{for some } 1 \le j \le m.$$

Then

$$\Pr[\mathsf{pre}_{\tau,t}] \le (Q+1) \cdot 2^{-\tau}.$$

Proof. For a fixed position $j \in \{1, \ldots, Q+1\}$, let E_j be the event that candidate position j exists and $\left\lfloor \mathsf{RO}_{\mathsf{flat}}(Y_j) \right\rfloor_\tau = t$. Expose the interaction in chronological order and condition on any positive-probability prefix immediately before candidate position j is appended; the target t is a constant. By the predictability premise the prefix determines Y_j; by the unrevealed-value premise no bit of $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ has been revealed in it, so under lazy sampling the value of $\mathsf{RO}_{\mathsf{flat}}(Y_j)$ may be sampled after the prefix is fixed and is uniform and independent of the conditioning. Hence the τ-bit projection of Y_j equals t with probability $2^{-\tau}$. If position j is never appended, then E_j is false, so the same bound applies. A union bound over the $Q+1$ positions gives the claim. □

A.6 Adaptive Key Tests

Lemma 9 (Adaptive Key-Test Bound)**.** *Consider a TW128 random-oracle-model game with hidden $K \leftarrow\$ \{0,1\}^k$, direct adversary access to $\mathsf{RO}_{\mathsf{flat}}$, and an immediate abort event $\mathsf{bad}_{\mathsf{key}}$ that fires on a direct query (Y, ℓ) exactly when $\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$ for some counted output-point class $\mathcal{C}$ of Table 8. Suppose that, before the abort, the following invariant holds for every direct query position i: after conditioning on the full visible execution prefix and on the set S_i of distinct decoded candidate keys from earlier direct queries, the hidden key is uniform on $\{0,1\}^k \setminus S_i$. Then*

$$\Pr[\mathsf{bad}_{\mathsf{key}}] \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}) = q_{\mathsf{RO}}/2^k,$$

where q_{RO} is the execution-uniform count of direct $\mathsf{RO}_{\mathsf{flat}}$ queries.

Proof. If $q_{\mathsf{RO}} \geq 2^k$ the right-hand side is at least one, so the claim is trivial. Assume $q_{\mathsf{RO}} < 2^k$.

Order the direct $\mathsf{RO}_{\mathsf{flat}}$ queries. By the definition of $\mathsf{KeyedTWOut}$ and Lemma 6, each query input either decodes to $\perp$ or decodes to a unique candidate key and counted output-point class. Queries that decode to $\perp$, and repeated queries whose decoded key already lies in the current set S_i, do not change the set of possible first successful key tests. Removing them can only shorten the sequence, so it suffices to analyze the subsequence of distinct fresh decoded candidate keys $K'_1, \ldots, K'_m$, where $m \leq q_{\mathsf{RO}}$.

Let F_i be the event that the first i fresh decoded candidate keys all miss the hidden key. The invariant says that, conditioned on the visible prefix before the $(i+1)$-st fresh test and on F_i, the hidden key is uniform over the $2^k - i$ keys not yet tested. The next fresh candidate key K'_{i+1} is one of those remaining keys and is determined by the adversary's next direct query, so

$$\Pr[K = K'_{i+1} \mid F_i] = \frac{1}{2^k - i}, \qquad \Pr[F_{i+1} \mid F_i] = 1 - \frac{1}{2^k - i}.$$

Multiplying the miss probabilities gives

$$\Pr[F_m] = \prod_{i=0}^{m-1} \left(1 - \frac{1}{2^k - i}\right) = \prod_{i=0}^{m-1} \frac{2^k - i - 1}{2^k - i} = \frac{2^k - m}{2^k}.$$

Thus the probability that some fresh decoded candidate key equals the hidden key is $m/2^k \leq q_{\mathsf{RO}}/2^k$. This event is exactly the first $\mathsf{bad}_{\mathsf{key}}$ abort, because $\mathsf{KeyedTWOut}$ returns either $\perp$ or a unique candidate key. □

A.7 Leaf Tag Collision Bound

Lemma 10 (Leaf Tag Collision Bound)**.** *Consider any TW128 random-oracle-model game with a hidden key, an adversary $\mathcal{B}$ with direct access to $\mathsf{RO}_{\mathsf{flat}}$, and construction computations that may generate final leaf transcripts; $n_{\mathsf{leaf}}(\mathcal{B})$ denotes the execution-uniform count of distinct construction-generated final leaf transcripts of Section 4.3. Let $\mathsf{hit}_{\mathsf{leaf}}$ be the event that, before some construction-generated keyed final leaf transcript is first created, a direct adversary query to $\mathsf{RO}_{\mathsf{flat}}$ has already hit the bridged input of that hidden-key leaf transcript. Let $\mathsf{bad}_{\mathsf{leaf}}$ be the event that two distinct construction-generated keyed final leaf transcript prefixes $X \neq X'$ in leaf tag output-point classes receive the same τ_{leaf}-bit projection from $\mathsf{RF}(\cdot)$. Then the probability that $\mathsf{bad}_{\mathsf{leaf}}$ occurs before $\mathsf{hit}_{\mathsf{leaf}}$ is bounded by*

$$\Pr[\mathsf{bad}_{\mathsf{leaf}} \text{ before } \mathsf{hit}_{\mathsf{leaf}}] \leq \binom{n_{\mathsf{leaf}}(\mathcal{B})}{2} \cdot 2^{-\tau_{\mathsf{leaf}}} = \frac{n_{\mathsf{leaf}}(\mathcal{B})(n_{\mathsf{leaf}}(\mathcal{B}) - 1)}{2^{\tau_{\mathsf{leaf}}+1}}.$$

Proof. Construction-generated means generated by encryption computations or by INT-CTXT verification computations (whether they accept or reject). Repeated evaluations of the same final leaf transcript are not new collision opportunities, since the lazy-sampled $\mathsf{RO}_{\mathsf{flat}}$ value is reused.

Run the relevant game and stop the leaf-collision accounting if $\mathsf{hit}_{\mathsf{leaf}}$ occurs. The execution-uniform resource $n_{\mathsf{leaf}}(\mathcal{B})$ of Section 4.3 bounds the number of distinct final leaf transcripts. Order those transcripts by first creation. At the moment such a transcript is first created by a construction computation before the stop, no earlier direct $\mathsf{RO}_{\mathsf{flat}}$ query has hit the bridge image of that hidden-key transcript by definition of $\mathsf{hit}_{\mathsf{leaf}}$. Nor has an earlier construction computation queried the same $\mathsf{RO}_{\mathsf{flat}}$ input under a different well-formed transcript: Lemma 5 separates distinct final leaf transcript encodings, and the [BDPVA11, Theorem 3] bridge is injective. The τ_{leaf}-bit leaf tag projection of $\mathsf{RF}(X)$ for that transcript sampled at that moment is therefore uniform and independent of the entire

prior game view, including the prior $\mathsf{RO}_{\mathsf{flat}}$ table on other inputs, prior ciphertext and tag outputs, and the leaf tag projections assigned to all distinct prior final leaf transcripts.

For any fixed unordered pair of distinct final leaf transcripts created before this stop, the probability that their leaf tags are equal is $2^{-\tau_{\mathsf{leaf}}}$. There are at most $\binom{n_{\mathsf{leaf}}(\mathcal{B})}{2}$ such pairs, and a union bound gives the claim. By Lemma 5, conditioned on no leaf tag collision, equal root aggregation inputs identify the same leaf transcript sequence. □

A.8 Transcript Freshness

Lemma 11 (Transcript Freshness)**.** *For any fixed nonce-respecting encryption query in the TW128 random-oracle experiment, let* $\mathsf{hit}_{\mathsf{fresh}}$ *be the event that, before this query reaches some counted transcript point, a prior direct adversary query to* $\mathsf{RO}_{\mathsf{flat}}$ *has already hit the bridged input of that hidden-key point. On* $\neg\mathsf{hit}_{\mathsf{fresh}}$*, every transcript output of the query that contributes to a returned ciphertext block, to an internal leaf tag, or to the final root tag is fresh at the moment its* $\mathsf{RO}_{\mathsf{flat}}$ *value is sampled.*

Proof. Work on an execution outside $\mathsf{hit}_{\mathsf{fresh}}$ for the fixed query. This rules out prior adversary reads of the hidden-key construction inputs considered below. It remains to rule out repeats from previous construction-interface outputs and from the current encryption query.

First root keystream block. The first root keystream block, when present, is produced by the root duplex after the initialization call $p_{\mathsf{root}} \parallel K \parallel N$ and any AD absorption (elided when $A = \varepsilon$, in which case the σ_{init}call itself emits the first root keystream block). Since the privacy adversary never repeats a nonce, no previous encryption query can have used the same root initialization transcript. Within the current query, Lemma 5 separates initialization, AD, message, and aggregation calls by their suffixes and by the duplex-call encoding, so the first root keystream transcript is fresh. If the requested keystream length is zero, no ciphertext block is returned at that point and there is nothing to prove for that output.

First leaf keystream block. For each leaf chunk $j \geq 1$, the first leaf keystream block is produced after the initialization call $p_{\mathsf{leaf}} \parallel K \parallel N \parallel \langle j \rangle_8$. The prefix separates leaves from the root, the nonce separates this encryption query from all previous encryption queries, and $\langle j \rangle_8$ separates leaves within this query. Hence every first leaf keystream transcript is fresh.

Later message outputs. Argue inductively within each root or leaf message stream. Suppose the current message-output transcript is fresh. If its output is used as a keystream block, its $\mathsf{RO}_{\mathsf{flat}}$ value Z_i is, by freshness of the lazy sample, uniform conditioned on the full prior view; since the adaptive plaintext block M_i is a deterministic function of that prior view,

$$C_i = M_i \oplus Z_i$$

is uniform conditioned on the prior view. TW128 then absorbs C_i with the appropriate message-phase suffix ($\sigma^{-}_{\mathsf{msg}}$ or $\sigma^{+}_{\mathsf{msg}}$) before requesting the next keystream block. Once C_i is sampled, the next transcript is fixed; Lemma 5's injectivity and domain separation imply it has not appeared earlier unless the same duplex-call prefix has appeared earlier. Nonce uniqueness excludes this across encryption queries. By Corollary 3, the next message-keystream output point can coincide with a within-query earlier output point only if both belong to the same output-point class on the same duplex instance (root or leaf-j with matching j) and have the same duplex-call sequence; the current message-stream prefix is strictly longer than every earlier prefix in that stream, so this is excluded. Across

distinct instances within the same query (root vs leaf, or two leaves with distinct j), the prefixes already differ in the first call's σ by $p_{\mathsf{root}}/p_{\mathsf{leaf}}$ or $\langle j \rangle_8$ respectively. Each later message-output transcript that emits keystream is therefore fresh when sampled. The terminal message-output transcript of a leaf emits the leaf tag rather than a following keystream block; the same separation argument applies to that transcript, so each internal leaf tag is sampled at a fresh point. Empty inputs still induce a duplex call, but with zero keystream output they contribute no ciphertext block.

Final root tag transcript. The final root tag is sampled at the root tag output point selected by $\|C\|$: the $\mathcal{R}_{\mathsf{tag}}$ point after the root duplex absorbs the aggregation transcript built from the leaf tags and $\langle n \rangle_8$ when $n \geq 1$, and the $\mathcal{R}^{+}_{\mathsf{msg}}$ point closing the root message phase when $n = 0$ (Section 3.5). When $n \geq 1$, the leaf tags are themselves $\mathsf{RO}_{\mathsf{flat}}$ outputs on final leaf transcripts; condition on their sampled values, on the sampled ciphertext, and on the adversary-visible prior view and $\mathsf{RO}_{\mathsf{flat}}$ table state, so that the root aggregation transcript is fixed. When $n = 0$ there is no aggregation transcript; condition on the sampled ciphertext and the adversary-visible prior view alone. By Corollary 3, the selected root tag output-point class is separated from every other output-point class in the same query — including root initialization (suffix σ_{init}), AD-phase outputs, the remaining message-phase outputs, leaf-side output points (separated by p_{leaf}), and, in the $n \geq 1$ case, earlier $\mathcal{R}_{\mathsf{agg}}$-prefix output points (separated by the $\sigma^{-}_{\mathsf{agg}}$vs $\sigma^{+}_{\mathsf{agg}}$suffix). Even if two encryption queries realize the same aggregation transcript (or, for $n = 0$, the same root ciphertext), the full root transcript also contains the root initialization $p_{\mathsf{root}} \,\|\, K \,\|\, N$, and nonce uniqueness separates that initialization from all previous encryption queries. The final root tag transcript is therefore fresh, and its τ_{root}-bit $\mathsf{RO}_{\mathsf{flat}}$ output is uniform independently of the conditioned information. □

A.9 Privacy Accounting Lemmas

The random-oracle privacy proof of Theorem 5 uses Transcript Freshness and the Adaptive Key-Test Bound through the privacy-specific interfaces below. These interfaces track the hidden key through the set of key values still viable after the visible stopped-game prefix. The stopped privacy games use the event $\mathsf{bad}_{\mathsf{key}}$ from Theorem 5: a direct query (Y, ℓ) aborts when $\mathsf{KeyedTWOut}(Y) = (K, \mathcal{C})$ for the hidden or auxiliary key K and a counted output-point class $\mathcal{C}$. The stopped games inspect this predicate before returning the direct-oracle answer for the offending query; if it holds, no answer for that query is included in the adversary's view.

Lemma 12 (Privacy One-Query Replacement)**.** *Fix a nonce-respecting privacy adversary and an index $i \geq 0$. Let H_i be the stopped privacy hybrid that samples $K \leftarrow\!\$\ \{0,1\}^k$, answers the first i accepted encryption queries by Rand, answers all later accepted encryption queries by $\mathsf{Enc}^{\mathsf{RO}}_K$, answers direct $\mathsf{RO}_{\mathsf{flat}}$ queries by the lazy table, and aborts on $\mathsf{bad}_{\mathsf{key}}$. On executions with fewer than $i+1$ accepted encryption queries, the two hybrids coincide. Then H_i and H_{i+1} can be coupled so that the adversary's visible transcript and the abort indicator are identical. In particular, they have identical joint distributions of the adversary's view and the abort indicator.*

Proof. Use the same adversary coins, the same key K, and the same answers to all direct queries and encryption queries before the $(i+1)$-st accepted encryption query. If the adversary never makes that query, the coupling is immediate. Otherwise condition on any common positive-probability visible prefix that includes the submitted query but not its oracle answer, and on no earlier abort. Let the query be (N, A, M).

Viable-key prefix. Let S be the set of distinct keys K' for which some prior direct query input Y satisfies $\mathsf{KeyedTWOut}(Y) = (K', \mathcal{C})$ for a counted output-point class $\mathcal{C}$, and

let $\mathcal{V} = \{0,1\}^k \setminus S$. Before the $(i+1)$-st accepted encryption query in H_i, the accepted construction-interface queries were answered by Rand, so the visible encryption answers and the lazy direct-query answers are independent of the hidden key except through the immediate-abort rule, which removes exactly the keys in S; hence the hidden key is uniform on $\mathcal{V}$ under this conditioning. For every $K' \in \mathcal{V}$, running the online execution of $\mathsf{Enc}^{\mathsf{RO}}_{K'}$ on the pending query reaches no counted transcript point whose bridged input was previously queried directly: such a point Y would give $\mathsf{KeyedTWOut}(Y) = (K', \mathcal{C})$ by Lemma 6, placing K' in S. The bridged transcript inputs reached for two distinct keys in $\mathcal{V}$ are disjoint, since equal inputs would, by bridge injectivity and Lemma 5, recover the same first duplex call and hence the same fixed-width key field.

For the actual key in H_i, the no-hit condition is precisely the event $\neg\mathsf{hit}_{\mathsf{fresh}}$ of Lemma 11 for the pending computation: any prior direct query that hit a counted transcript point reached by the computation would, by Lemma 6, be decoded by $\mathsf{KeyedTWOut}$ to the hidden key K and would already have triggered $\mathsf{bad}_{\mathsf{key}}$. Applying Lemma 11 gives freshness for every construction output of the query that contributes to the returned ciphertext, to an internal leaf tag, or to the final root tag. Applying the same transcript freshness argument to each $K' \in \mathcal{V}$, and using the disjointness of key-indexed transcript inputs established above, enumerate all such outputs in the order taken by the encryption computation.

Thus, to describe the conditional law, introduce an auxiliary online family of uniform coins indexed by a viable key and by the output position reached in that key's encryption computation. These auxiliary coins are sampled independently of the hidden key. If the hidden key is K', the H_i computation reads the K'-row and installs only those row entries in the lazy table, at the transcript addresses actually reached by $\mathsf{Enc}^{\mathsf{RO}}_{K'}$. The coins for other viable keys are not lazy-table entries; they only witness that every possible hidden-key row induces the same distribution on the returned string. A later direct query to a nonactual viable-key address is therefore answered by the ordinary shared lazy table in both hybrids, not by a coin sampled during this replacement coupling.

Each plaintext block is fixed before the corresponding keystream coin is drawn, so XOR with that coin makes the ciphertext block uniform. Once a ciphertext block is fixed, the next message transcript is fixed and fresh at its sampling time. When $n \geq 1$, leaf tags are sampled from fresh coins as well; after these tags and the ciphertext are fixed, the aggregation transcript is fixed and the final root tag is sampled from another fresh coin at the $\mathcal{R}_{\mathsf{tag}}$ point. When $n = 0$ the aggregation phase is elided and the final root tag is instead the fresh coin at the closing $\mathcal{R}^{+}_{\mathsf{msg}}$ root message point. Hence the returned string (C, T) is jointly uniform of length $\|M\| + \tau_{\mathsf{root}}/8$ and independent of K conditioned on the prefix. In the coupling, the Rand answer in H_{i+1} is sampled as this same independent string.

It remains to compare the lazy-table state exposed after this query. The entries that may be written in H_i but not in H_{i+1} are construction-internal entries at hidden-key counted transcript inputs for nonce N. A later direct adversary query to any such input would be decoded by $\mathsf{KeyedTWOut}$ to K and would trigger $\mathsf{bad}_{\mathsf{key}}$ before returning an oracle answer, so the possibly different table entry is never exposed. Later real encryption queries use nonces different from N by nonce-respectingness, and their typed construction inputs are therefore disjoint from this query's inputs by Corollary 3. Thus the extra lazy-table entries are unobservable before the abort. Continuing the coupling with shared answers on all exposed direct-oracle inputs and all later real construction inputs gives identical visible transcripts and identical abort indicators. □

Lemma 13 (Privacy Stopped-Game Advantage Bound)**.** *Let* P_0 *and* P_1 *be the unstopped real and ideal privacy games of Theorem 5, with* P_1 *carrying the auxiliary key used only for the abort predicate, and let* $\mathsf{P}_0^{\mathrm{stop}}$ *and* $\mathsf{P}_1^{\mathrm{stop}}$ *be their immediate-abort variants. Assume the adversary* $\mathcal{B}$ *has an execution-uniform cap* q_e *on accepted encryption queries. Then for*

any event E determined by the adversary's final output in the unstopped games,

$$|\Pr[E \text{ in } \mathsf{P}_0] - \Pr[E \text{ in } \mathsf{P}_1]| \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})).$$

Proof. Stopped-game equality. Define hybrids $H_0, \ldots, H_{q_e}$ as in Lemma 12. Then $H_0 = \mathsf{P}_0^{\mathrm{stop}}$. Since every accepted encryption query is among the first q_e such queries, H_{q_e} is $\mathsf{P}_1^{\mathrm{stop}}$. Applying Lemma 12 for $i = 0, \ldots, q_e - 1$ gives the same joint distribution of the adversary's view and the abort indicator in adjacent hybrids, and telescoping gives the same joint distribution in the two stopped games. Write

$$p = \Pr[\mathsf{bad}_{\mathsf{key}} \text{ in } \mathsf{P}_1^{\mathrm{stop}}];$$

by the stopped-game equality the abort probability is also p in $\mathsf{P}_0^{\mathrm{stop}}$. For each side $b \in \{0,1\}$, the unstopped game P_b and its stopped variant $\mathsf{P}_b^{\mathrm{stop}}$ agree exactly until the first $\mathsf{bad}_{\mathsf{key}}$ direct query and have the same occurrence probability for that event. Hence, for the final-output event E,

$$\Pr[E \text{ in } \mathsf{P}_b] = \Pr[E \wedge \neg \mathsf{bad}_{\mathsf{key}} \text{ in } \mathsf{P}_b^{\mathrm{stop}}] + x_b$$

for some $0 \le x_b \le p$. The first terms are equal for $b = 0$ and $b = 1$ by stopped-game equality, while $x_0, x_1 \in [0, p]$, so

$$|\Pr[E \text{ in } \mathsf{P}_0] - \Pr[E \text{ in } \mathsf{P}_1]| \le p.$$

Abort bound. It remains to show $p \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$. Let $\mathsf{P}_1^{\mathrm{aux}}$ be the non-aborting version of the ideal privacy game: it samples the auxiliary key $K \leftarrow\!\!\$\ \{0,1\}^k$, answers construction queries by fresh random strings exactly as Rand does, answers direct $\mathsf{RO}_{\mathsf{flat}}$ queries by the ordinary lazy table, and never aborts. For each direct query input Y_i, Lemma 6 either rejects Y_i as outside the bridged well-formed TW128 transcript language or recovers a unique candidate key from the decoded keyed transcript prefix.

The visible-prefix uniformity invariant required by Lemma 9 holds in $\mathsf{P}_1^{\mathrm{aux}}$. Condition on any positive-probability visible execution prefix immediately before a direct query, on no earlier $\mathsf{bad}_{\mathsf{key}}$, and on the set S_i of distinct candidate keys decoded from prior direct-query inputs. Construction responses are fresh random strings, direct $\mathsf{RO}_{\mathsf{flat}}$ answers are lazy random-oracle values, and both are independent of the auxiliary key K; the no-earlier-abort condition removes exactly the keys in S_i. Thus K remains uniform on $\{0,1\}^k \setminus S_i$.

The first $\mathsf{bad}_{\mathsf{key}}$ event in $\mathsf{P}_1^{\mathrm{stop}}$ has the same probability as the first successful adaptive key test in $\mathsf{P}_1^{\mathrm{aux}}$, because the two games are identical up to that query. Applying Lemma 9 with $q_{\mathsf{RO}} = q_{\mathsf{RO}}(\mathcal{B})$ gives $p \le \mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$. Combining with the stopped-game bound proves the claim. □

A.10 INT-CTXT Accounting Lemmas

The INT-CTXT proof of Theorem 6 uses the generic key-test and leaf-collision accounting lemmas together with the INT-specific facts below. The *root tag output point* of a computation is the output point that emits its root tag: $\mathcal{R}_{\mathsf{tag}}$ (suffix σ^+_{agg}) when $n \ge 1$, and the closing root message point $\mathcal{R}^+_{\mathsf{msg}}$ (suffix σ^+_{msg}) when $n = 0$ and the aggregation phase is elided (Section 3.5). Because $n = \mathrm{leaves}(C)$ is fixed by $\|C\|$, all computations on a common ciphertext share the same root tag output point. A *final root transcript class* is the set of construction computations whose encryption or verification computation realizes the same bridged final root input at this root tag output point. Such a class is *encryption-first* if its first construction generation is by an encryption query, and *verification-first* if its first construction generation is by a non-replay verification query.

Lemma 14 (INT Visible-Prefix Key Uniformity)**.** *In the random-oracle INT-CTXT game stopped at* $\mathsf{bad}_{\mathsf{key}}$ *as in Theorem 6, order the direct* $\mathsf{RO}_{\mathsf{flat}}$ *queries. For a direct query position* i*, let* S_i *be the set of distinct candidate keys decoded by* KeyedTWOut *from earlier direct-query inputs. Condition on any positive-probability visible execution prefix immediately before query* i*, on no earlier* $\mathsf{bad}_{\mathsf{key}}$*, and on* S_i*. Then the hidden key* K *is uniform on* $\{0,1\}^k \setminus S_i$*. Consequently the uniformity invariant required by Lemma 9 holds in the stopped INT-CTXT game.*

Proof. The visible prefix contains the adversary's oracle queries and replies: direct $\mathsf{RO}_{\mathsf{flat}}$ outputs, encryption outputs and the resulting replay table, verification bits, and the adversary state determined by these values. Fix such a prefix and two keys $K_0, K_1 \notin S_i$.

Define a transcript-renaming map $\Phi_{0\to1}$ on typed construction prefixes by replacing the first-call key field K_0 by K_1 and leaving every other recovered field unchanged: the nonce, leaf index, phase sequence, AD blocks, ciphertext blocks, leaf tag bytes appearing in root aggregation, and the root/leaf instance identifier are preserved. Let $\Phi_{1\to0}$ be the inverse map. By Lemma 6 and Corollary 3, $\mathsf{flat} \circ \Phi_{0\to1} \circ \mathsf{flat}^{-1}$ is a bijection between the bridged construction-addresses whose decoded key is K_0 and those whose decoded key is K_1; it preserves output-point classes and equality relations among addresses.

This bijection is applied online, not to a pre-existing fixed address set. Whenever a construction computation under K_0 would first query a hidden-key address Y, the coupled computation under K_1 queries $\Phi_{0\to1}(Y)$ and receives the same lazy-table value; repeated queries are coupled consistently through the same map. If the queried address is a root aggregation address containing leaf tag bytes, those bytes are already coupled equal because the corresponding leaf tag lookups were either repeated or first sampled earlier under the same renaming rule. Thus adaptively generated message transcripts and root aggregation transcripts are renamed step by step with the same visible ciphertext, leaf tag, and root tag bytes.

Prior direct-query inputs are not renamed: they are adversary-chosen visible strings. None lies in either hidden-key construction-address set for K_0 or K_1, because any prior direct query to such an address would be decoded by KeyedTWOut to K_0 or K_1 and would put that key in S_i. Direct-query lazy-table values are therefore coupled identically and remain disjoint from the renamed hidden construction addresses.

Use the same adversary coins, the same direct-query lazy-table values, and the online renamed construction-internal lazy-table values. Encryption outputs have the same distribution under the coupling, because every construction lookup that determines ciphertext, leaf tags, or the final root tag is paired with a distinct renamed lookup carrying the same sampled value. Replay verification decisions depend only on the visible replay table. Non-replay verification computations are renamed in the same online way as encryption computations, and the final root tag comparison therefore gives the same accept/reject bit. Verification does not reveal plaintext on rejection or acceptance; it reveals only that bit.

Thus every visible prefix with hidden key $K_0 \notin S_i$ has the same conditional probability as the corresponding execution with hidden key $K_1 \notin S_i$. Since K is initially uniform and the condition "no earlier $\mathsf{bad}_{\mathsf{key}}$" removes exactly the keys in S_i, all keys in $\{0,1\}^k \setminus S_i$ remain equally likely. □

Lemma 15 (INT Verification-First Root Guessing)**.** *In the random-oracle INT-CTXT game stopped before* $\mathsf{bad}_{\mathsf{key}} \cup \mathsf{bad}_{\mathsf{leaf}}$*, let* $\mathsf{win}_{\mathsf{vf}}$ *be the event that some accepting non-replay verification submission belongs to a verification-first final root transcript class. Then*

$$\Pr[\mathsf{win}_{\mathsf{vf}}] \le q_{\mathsf{v}}(\mathcal{B})/2^{\tau_{\mathsf{root}}}.$$

Proof. Work in the game stopped at the first occurrence of $\mathsf{bad}_{\mathsf{key}}$ or $\mathsf{bad}_{\mathsf{leaf}}$. For accounting, define an all-reject continuation: encryption queries are answered normally, replay verification queries accept normally, and every non-replay verification submission performs

the same construction computation but returns reject to the adversary. Lazy sampling is deferred for each verification-first class's final root tag value: the continuation records the class address and the submitted tag, but while the class remains verification-first it does not expose that root tag value to the adversary. The execution-uniform count $q_{\mathsf{v}}(\mathcal{B})$ applies to this continuation.

Fix a verification-first final root transcript class $\mathcal{C}$ and follow the all-reject path until the first later encryption query generating the same class, if any. Let $G_{\mathcal{C}}$ be the set of distinct candidate tags submitted by non-replay verification queries in class $\mathcal{C}$ before that cutoff. Repeated tags only shrink this set. The class's correct root tag $U_{\mathcal{C}}$ is the τ_{root}-bit projection of $\mathsf{RO}_{\mathsf{flat}}$ at the class's bridged root tag input. Before the stop, no direct query has hit this hidden-key input: such a query would be decoded by $\mathsf{KeyedTWOut}$ to the hidden key and would trigger $\mathsf{bad}_{\mathsf{key}}$. Before the cutoff, no encryption query has revealed $U_{\mathcal{C}}$ as an encryption tag. (When the class's root tag point is $\mathcal{R}^{+}_{\mathsf{msg}}$, i.e. $n = 0$, an encryption query on a longer message that shares chunk 0 may touch this same address while closing its chunk-0 message phase, but it requests zero output there and continues into its aggregation phase, so it does not reveal $U_{\mathcal{C}}$; its own tag is emitted at a later $\mathcal{R}_{\mathsf{tag}}$ point.) The all-reject path and the set $G_{\mathcal{C}}$ are therefore functions only of the adversary's coins, visible oracle replies, and lazy-table values at inputs other than this class's root tag address. By lazy sampling, conditioned on those values and on the fixed path, $U_{\mathcal{C}}$ may be sampled after $G_{\mathcal{C}}$ is fixed and is uniform independently of $G_{\mathcal{C}}$.

After the cutoff, if it exists, class $\mathcal{C}$ contributes no accepting non-replay submission. Let the cutoff encryption query return (N_e, A_e, C_e, T_e). Any later verification submission in the same final root transcript class has, by the final-root determinacy part of Lemma 5 and the absence of $\mathsf{bad}_{\mathsf{leaf}}$, the same (N, A, C) as that encryption computation. If the submitted tag is T_e, the tuple is a recorded encryption tuple and the verification is a replay; if the submitted tag is not T_e, the final tag comparison fails.

Consider the event that the first accepting non-replay verification submission in the stopped game belongs to class $\mathcal{C}$. Up to that global first accept, the adversary's visible answers match the all-reject continuation: every earlier non-replay submission rejects, replay queries accept in both executions, and encryption answers are unchanged. The construction computations also agree at every address except that the all-reject continuation defers the final root tag value for class $\mathcal{C}$ until the comparison is needed. Therefore the first accepting non-replay submission can belong to class $\mathcal{C}$ only before the cutoff, and only if $U_{\mathcal{C}} \in G_{\mathcal{C}}$, which has probability at most $|G_{\mathcal{C}}|/2^{\tau_{\mathsf{root}}}$.

Each non-replay verification submission deterministically reconstructs one final root transcript at the root tag output point. If the class is verification-first and the submission occurs before that class's encryption cutoff, the submitted tag contributes to at most one set $G_{\mathcal{C}}$; otherwise it contributes to none. Thus $\sum_{\mathcal{C}} |G_{\mathcal{C}}| \le q_{\mathsf{v}}(\mathcal{B})$, and a union bound over the verification-first classes gives the claim. □

A.11 Multi-User Hybrid Lemma

The multi-user indistinguishability theorem (Theorem 2) telescopes a hybrid over the D users, replacing one real user by its ideal $(\mathsf{Rand}, \mathsf{Replay})$ interface at each step. The following lemma bounds one such step, reusing the privacy accounting of Lemma 12 and the INT-CTXT accounting of Theorem 6 within a shared random-oracle environment.

Lemma 16 (Mu-Ind Adjacent-Hybrid Bound)**.** *Let $\mathcal{B}$ be a flat-RO mu-ind adversary in the pre-sampled-key formulation of Section 4.4, with keys $K_1, \ldots, K_D$ sampled at the start of the experiment. Fix $d \in \{1, \ldots, D\}$. Let $H^{\star}_{d-1}$ and $H^{\star}_{d}$ be the adjacent stopped hybrids that output a fixed bit if any two sampled keys collide, and otherwise run $\mathcal{B}$ with one shared lazy $\mathsf{RO}_{\mathsf{flat}}$ table as follows: users $e < d$ are answered by $(\mathsf{Rand}, \mathsf{Replay})$, users $e > d$ are answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_e}, \mathsf{Ver}^{\mathsf{RO}}_{K_e})$, and user d is answered by $(\mathsf{Enc}^{\mathsf{RO}}_{K_d}, \mathsf{Ver}^{\mathsf{RO}}_{K_d})$ in $H^{\star}_{d-1}$ and by*

(Rand, Replay) *in* $H^\star_d$*. If* $n^{(d)}_{\mathsf{leaf}}$ *and* $q^{(d)}_{\mathsf{v}}$ *are the user-*d *resource caps, then*

$$\left|\Pr[\mathcal{B}^{H^\star_{d-1}} \Rightarrow 1] - \Pr[\mathcal{B}^{H^\star_d} \Rightarrow 1]\right| \le 2\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}(n^{(d)}_{\mathsf{leaf}} - 1)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Proof. The fixed-output branch on a key collision is identical in $H^\star_{d-1}$ and $H^\star_d$, so only the branch with pairwise distinct keys contributes. On that branch, the two hybrids differ only in user d: users $e < d$ are ideal in both hybrids, and users $e > d$ are real in both hybrids.

Let $\mathcal{E}_d$ denote the common surrounding environment. It runs $\mathcal{B}$, answers users $e < d$ by their (Rand, Replay) interfaces, answers users $e > d$ by $\mathsf{Enc}^{\mathsf{RO}}_{K_e}, \mathsf{Ver}^{\mathsf{RO}}_{K_e}$, forwards $\mathcal{B}$'s direct $\mathsf{RO}_{\mathsf{flat}}$ queries unchanged, and shares the same lazy $\mathsf{RO}_{\mathsf{flat}}$ table with user d. The Bellare–Namprempre insertion is pathwise in this environment. Insert the interface in which user d's encryption oracle is still real but user d's verification oracle is replaced by its replay table. The first hop is the user-d INT-CTXT event; the second hop is the user-d privacy replacement with the same environment and replay table. The replay table is determined by user d's encryption answers and adds no direct $\mathsf{RO}_{\mathsf{flat}}$ queries or construction-generated transcripts.

The privacy and INT-CTXT accounting used in these two hops is local to user d's keyed transcript language. In the privacy hop, the one-query replacement of Lemma 12 only requires that no direct query has read a user-d counted transcript before $\mathsf{bad}^{(d)}_{\mathsf{key}}$, the $\mathsf{bad}_{\mathsf{key}}$ event of Sections 6 and 7 instantiated at user d's key K_d; any extra lazy-table entries written by the real user-d encryption computation are unobservable before that abort. The accepted user-d encryption queries are nonce-respecting because the mu-ind interface enforces nonce uniqueness per user. In the INT-CTXT hop, the analysis of Theorem 6—its visible-prefix key uniformity (Lemma 14), its leaf-freshness and encryption-first-replay steps, and the verification-first root-guessing bound (Lemma 15)—refers only to user-d hidden-key transcript points, user-d final leaf transcripts, and user-d non-replay verification submissions.

It remains to justify that the same-RO environment does not enlarge the direct-query term. The real users $e > d$ may perform construction-internal $\mathsf{RO}_{\mathsf{flat}}$ lookups while simulating $\mathsf{Enc}^{\mathsf{RO}}_{K_e}$ and $\mathsf{Ver}^{\mathsf{RO}}_{K_e}$. Whenever such a lookup lies in a counted keyed transcript class, Lemma 6 decodes its candidate key as K_e; on the non-collision branch this is different from K_d. If the lookup is outside the counted keyed language, it is irrelevant to $\mathsf{bad}^{(d)}_{\mathsf{key}}$. Thus foreign-user construction lookups neither trigger user d's key-test event nor pre-read a user-d hidden transcript point. They also do not create user-d leaf tag collisions or user-d verification attempts, which are counted by $n^{(d)}_{\mathsf{leaf}}$ and $q^{(d)}_{\mathsf{v}}$.

The key-test contribution is taken in the original pre-sampled-key experiment, not in a renormalized experiment conditioned on the non-collision branch. Fix the foreign keys and run the visible-prefix coupling used in the user-d privacy and INT-CTXT proofs, with foreign-key transcript addresses left unchanged. Before the first user-d key hit, the visible prefix is invariant under renaming K_d among keys not already tested and not equal to any fixed foreign key. For each fresh candidate key tested by one of $\mathcal{B}$'s original direct $\mathsf{RO}_{\mathsf{flat}}$ queries, the event contributing on the stopped branch is contained in $\{K_d \text{ equals that candidate}\}$; if the candidate is a foreign key, it contributes nothing on the non-collision branch. Hence each fresh original direct test costs at most 2^{-k} in the unconditioned key experiment, and at most $q_{\mathsf{RO}}(\mathcal{B})$ such tests occur.

The proofs of Theorems 5 and 6 therefore apply to the two user-d hops with the same direct-query cap $q_{\mathsf{RO}}(\mathcal{B})$ and the same user-d construction resources $n^{(d)}_{\mathsf{leaf}}, q^{(d)}_{\mathsf{v}}$. The surrounding environment affects oracle scheduling and foreign construction-internal table entries, but not the counted user-d events. The privacy hop contributes $\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B}))$, and the INT-CTXT hop contributes

$$\mathsf{KeyGuess}_k(q_{\mathsf{RO}}(\mathcal{B})) + \frac{n^{(d)}_{\mathsf{leaf}}(n^{(d)}_{\mathsf{leaf}} - 1)}{2^{\tau_{\mathsf{leaf}}+1}} + \frac{q^{(d)}_{\mathsf{v}}}{2^{\tau_{\mathsf{root}}}}.$$

Combining the two hops gives the stated bound. □

B The Flat-Sponge Lift

This appendix expands the sketch of Section 5.3 into the full argument: it supplies the flat-sponge lift machinery (Lemmas 18 and 19) invoked by the headline theorems of Section 5.1, and the public-permutation CMTDs-3 corollary (Corollary 5) consumed by Section C.

B.1 Simulator Convention

Throughout this appendix, $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ denotes the public-primitive simulator supplied by [BDPVA08, Theorem 2] — the indifferentiability theorem for the simple padded sponge over a random permutation — after applying the [BDPVA11, Theorem 3] bridge of Lemma 6 to TW128's native $\mathsf{pad10^*1}$ transcript inputs. The superscript records that both simulator directions share the same internal lazy $\mathsf{RO}_{\mathsf{flat}}$ table. This is the simulator referenced by the Sections 4.1 and 4.4 definitions of $\mathbf{Adv}^{\mathsf{priv}}_{\mathsf{TW128}}$, $\mathbf{Adv}^{\mathsf{ae}}_{\mathsf{TW128}}$, and $\mathbf{Adv}^{\mathsf{mu\text{-}ind}}_{\mathsf{TW128}}$: the ideal side of each AE-style experiment exposes the construction interface specified by the game (either Rand or $(\mathsf{Rand}, \mathsf{Replay})$, instantiated per user in mu-ind) together with $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ as the public-primitive interface. The $\mathsf{RO}_{\mathsf{flat}}$ table in this notation is internal to the ideal system and is not an extra interface exposed to the application adversary; the INT-CTXT and committing-security advantages remain the real public-permutation success probabilities of Section 4.1, with the simulator appearing only inside the reduction to the corresponding random-oracle-model adversary.

Lemma 17 (Simulator RO-Call Filter)**.** *For the simulator $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$, every inverse primitive-interface query and every forward primitive-interface query violating one of the following conditions induces no simulator $\mathsf{RO}_{\mathsf{flat}}$ call. The symbols s, p, $\mathcal{R}$, $\mathcal{O}$, and $\mathcal{C}$ are the local internal notation of the BDPV simple-padded-sponge simulator, not TW128 resource notation:*

- **(C1) New edge.** *The simulator's internal graph has no outgoing edge from the input node s.*
- **(C2) Unsaturated.** $\mathcal{R} \cup \mathcal{O} \neq \mathcal{C}$.
- **(C3) Rooted input.** *The input node is rooted.*
- **(C4) Paddable path.** *The constructed path p decomposes uniquely as $p = p' \parallel 0^{rj}$ with p' not ending in 0^r, and the prefix p' is a valid padded sponge input: equivalently, $p' = x \parallel \mathsf{pad10^*}[r](|x|)$ for an unpadded BDPV08 random-oracle input x.*

When all four conditions hold, the forward primitive-interface query induces one simulator $\mathsf{RO}_{\mathsf{flat}}$ call. Consequently, each primitive-interface query induces at most one simulator-side $\mathsf{RO}_{\mathsf{flat}}$ call.

Proof. This is the branch structure of the [BDPVA08] simple-padded permutation simulator. The simulator invokes the random oracle only on the forward-query branch where the input node is rooted, unsaturated, paddable to an unpadded random-oracle input x, and not already extended by an outgoing edge in the simulator graph. Forward queries violating any of these conditions and all inverse queries return outputs sampled or read from the simulator's internal graph without invoking the random oracle. Under the [BDPVA11, Theorem 3] bridge fixed in Section B.1, that simulator random-oracle invocation is the simulator-side $\mathsf{RO}_{\mathsf{flat}}$ call used throughout this appendix. □

Lemma 18 (Flat-Sponge Lift Bound)**.** *Let* G *be any of the single-stage TW128 games of Sections 4.1 and 4.4: privacy, INT-CTXT, all-in-one AE, mu-ind, CMTDs-3, tag-CMTDs-4, CDY, or CMTs-ℓ for* $\ell \in \{1, 3, 4\}$*. Let* $\mathcal{A}$ *be a public-permutation* G *adversary with construction-interface cost at most* N *and at most* N_{prim} *direct primitive-interface queries. In the [BDPVA08] single-stage replacement, the combined construction-and-primitive transcript has length at most*

$$N_{\mathsf{tot}} = N + N_{\mathsf{prim}}.$$

Consequently replacing the real public permutation and TW128 construction by the simulator-world primitive interface $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ *and the corresponding flat-RO construction interface costs at most* $\mathsf{Lift}_c(N_{\mathsf{tot}})$*.*

Proof. The [BDPVA08] replacement is applied before the derived-adversary construction, so its transcript contains two kinds of public-primitive activity. First, each construction-interface computation contributes the TW128 duplex calls it performs. By Lemmas 4 and 6, each such root or leaf output point is a typed restriction of the bridged flat-sponge construction, and by the definition of N in Section 4.3 these calls are counted at the same per-block granularity as the underlying KECCAK-p[1600, 12] calls. This includes verification computations whether they accept or reject, the per-user sum of construction calls in mu-ind, and the deterministic challenger decryption or encryption checks actually performed in the committing and discoverability games.

Second, each direct adversary query to (π, π^{-1}) contributes one primitive-interface query and is counted in N_{prim}. These two counts are disjoint in the public-permutation experiment: N counts construction-internal use of the primitive, while N_{prim} counts only the adversary's public-primitive interface queries. The execution-uniform caps ensure that every adaptive transcript seen by the single-stage replacement has length at most $N + N_{\mathsf{prim}} = N_{\mathsf{tot}}$. If $N_{\mathsf{tot}} \geq 2^c$ then $\mathsf{Lift}_c(N_{\mathsf{tot}}) \geq 1$ and the claimed bound holds trivially, so assume $N_{\mathsf{tot}} < 2^c$, satisfying the hypothesis of [BDPVA08, Theorem 2]. That theorem gives distinguishing or success-probability cost at most the exact random-permutation differentiating probability $f_P(N_{\mathsf{tot}})$ of [BDPVA08, Lemma 5], which Section 5.2 bounds by $\mathsf{Lift}_c(N_{\mathsf{tot}})$. □

Lemma 19 (Derived-Adversary Execution Contract)**.** *Let* G *be any of the single-stage TW128 games of Sections 4.1 and 4.4: privacy, INT-CTXT, all-in-one AE, mu-ind, CMTDs-3, tag-CMTDs-4, CDY, or CMTs-ℓ for* $\ell \in \{1, 3, 4\}$*. Let* $\mathcal{A}$ *be a simulator-world* G *adversary whose public-primitive interface is answered by* $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$*, with at most* N_{prim} *direct primitive-interface queries. Define* $\mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ *to run* $\mathcal{A}$ *internally, answer each primitive-interface query by running the same simulator against direct finite-output queries to its own* $\mathsf{RO}_{\mathsf{flat}}$ *oracle, and forward construction-interface queries (or, in the committing and discoverability games, the final structured output) unchanged to the corresponding random-oracle experiment. Then the simulator-world execution of* $\mathcal{A}$ *and the random-oracle execution of* $\mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ *have identical joint distributions, and*

$$q_{\mathsf{RO}}(\mathcal{B}_{\mathrm{derived}}) \leq N_{\mathsf{prim}}.$$

In this coupling, simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ *calls and construction-internal* RF *lookups use the same lazy table. Therefore, if a simulator-induced query is the bridged input* $\mathsf{flat}(X)$ *of a well-formed TW128 transcript prefix* X *and a forwarded construction computation later reaches the same prefix* X*, both interfaces read the same* $\mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X))$ *value, projected to the requested output length. Moreover,* $\mathcal{B}_{\mathrm{derived}}$ *inherits the construction-side caps of* $\mathcal{A}$*:*

$$q_{\mathsf{v}}(\mathcal{B}_{\mathrm{derived}}) = q_{\mathsf{v}}(\mathcal{A}) \quad \textit{(INT-CTXT, AE, mu-ind)},$$
$$n_{\mathsf{leaf}}(\mathcal{B}_{\mathrm{derived}}) \leq n_{\mathsf{leaf}}(\mathcal{A}) \quad \textit{(all games above)}.$$

In the mu-ind game these resource inheritances hold per user, and the New*-query transcript is unchanged.*

Proof. Couple the two executions by using the same lazy $\mathsf{RO}_{\mathsf{flat}}$ table for the simulator and for all construction-internal $\mathsf{RF}(\cdot)$ lookups. In the simulator-world execution, $\mathcal{A}$ sees construction answers from the selected game interface and public-primitive answers from $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$. In the derived random-oracle execution, $\mathcal{B}_{\mathrm{derived}}$ runs the same $\mathcal{A}$, forwards construction-interface queries (or the final committing- or discoverability-game output) unchanged, and answers primitive-interface queries by invoking the same simulator code with the same lazy $\mathsf{RO}_{\mathsf{flat}}$ table. The construction interface or committing challenger therefore receives the same forwarded inputs, the simulator receives the same primitive queries in the same order, and the lazy table supplies the same values. The full views of $\mathcal{A}$ in the simulator-world execution and of $\mathcal{B}_{\mathrm{derived}}$ in the random-oracle execution are identically distributed.

The simulator's random-oracle calls may be on arbitrary [BDPVA08] H-input strings, not only bridged TW128 transcript inputs. These calls are the direct $\mathsf{RO}_{\mathsf{flat}}$ queries of $\mathcal{B}_{\mathrm{derived}}$ and are counted in $q_{\mathsf{RO}}(\mathcal{B}_{\mathrm{derived}})$. Construction-internal $\mathsf{RF}(\cdot)$ lookups made by $\mathsf{Enc}_K^{\mathsf{RO}}$, $\mathsf{Dec}_K^{\mathsf{RO}}$, $\mathsf{Ver}_K^{\mathsf{RO}}$, or a committing or discoverability challenger are forwarded to the same $\mathsf{RO}_{\mathsf{flat}}$ table but are not counted in q_{RO}, exactly as in Section 4.3.

By Lemma 17, each primitive-interface query induces at most one direct $\mathsf{RO}_{\mathsf{flat}}$ query by $\mathcal{B}_{\mathrm{derived}}$, and inverse queries or non-RO-touching forward queries induce none. Since $\mathcal{A}$ makes at most N_{prim} direct primitive-interface queries, this gives $q_{\mathsf{RO}}(\mathcal{B}_{\mathrm{derived}}) \le N_{\mathsf{prim}}$.

Lemma 6's bridge places TW128 construction restrictions and simulator-induced $\mathsf{RO}_{\mathsf{flat}}$ calls in a single lazy table. If the simulator invokes $\mathsf{RO}_{\mathsf{flat}}$ on $\mathsf{flat}(X)$ for a well-formed TW128 transcript prefix X, and a forwarded construction computation later reaches X, the construction lookup $\mathsf{RF}(X) = \mathsf{RO}_{\mathsf{flat}}(\mathsf{flat}(X))$ reads the same table entry and returns the requested projection. Since TW128 uses $r = r_{\max}$, the [BDPVA11, Theorem 3] output post-processing does not alter TW128 output bits. Thus simulator-induced direct queries and construction-internal lookups at the same bridged input are transcript-consistent. In the AE games, a simulator call whose input is accepted by KeyedTWOut for the hidden key is still a direct $\mathsf{RO}_{\mathsf{flat}}$ query and is covered by the same $\mathsf{bad}_{\mathsf{key}}$ accounting used by the random-oracle privacy and INT-CTXT theorems. The derivation step itself does not prove a key-guessing statement; it only supplies the derived adversary and the query bound at which the random-oracle envelopes of Section 5.3 are instantiated.

Finally, the execution-uniform caps of Section 4.3 hold along every oracle-answer trace of $\mathcal{A}$, including traces generated by the simulator on the ideal public-primitive interface. The derived-adversary construction does not add construction-interface encryption or verification queries and forwards $\mathcal{A}$'s construction-interface queries unchanged along each trace. $\mathcal{B}_{\mathrm{derived}}$ therefore inherits the relevant construction resources: $q_{\mathsf{v}}(\mathcal{B}_{\mathrm{derived}}) = q_{\mathsf{v}}(\mathcal{A})$ in the INT-CTXT, AE, and mu-ind games, and $n_{\mathsf{leaf}}(\mathcal{B}_{\mathrm{derived}}) \le n_{\mathsf{leaf}}(\mathcal{A})$ for the distinct construction-generated final-leaf-transcript cap. In mu-ind, construction-interface forwarding preserves the New calls and these caps user by user. □

B.2 Lift Application and Public-Permutation Corollaries

The lift bound of Lemma 18 and the derived-adversary construction of Lemma 19 combine into a single template, stated once and then instantiated by the headline theorems of Section 5.1 and the public-permutation corollaries below.

Corollary 4 (Lift Application)**.** *Let* G *be any of the games of Lemma 18. For every public-permutation* G*-adversary* $\mathcal{A}$ *against TW128 with the resources of Section 4.3, the derived adversary* $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$ *of Lemma 19 is a random-oracle* G*-adversary with* $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ *and the construction-side caps of Lemma 19—in particular* $q_{\mathsf{v}}(\mathcal{B}) \le Q_{\mathsf{v}}$ *and* $n_{\mathsf{leaf}}(\mathcal{B}) \le N_{\mathsf{leaf}}$*, where these apply—such that*

$$\mathbf{Adv}^{\mathsf{G}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}).$$

If moreover $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{RO}}(\mathcal{B}) \le \varepsilon_{\mathsf{G}}(q_{\mathsf{RO}}(\mathcal{B}), n_{\mathsf{leaf}}(\mathcal{B}))$ *for some* ε_{G} *nondecreasing in each argument, then*

$$\mathbf{Adv}^{\mathsf{G}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \varepsilon_{\mathsf{G}}(N_{\mathsf{prim}}, N_{\mathsf{leaf}}).$$

Proof. By the flat-sponge lift bound (Lemma 18), replacing the real public permutation and TW128 construction by $(\mathcal{S}^{\mathsf{RO}_{\mathsf{flat}}}, \mathcal{S}^{-1,\mathsf{RO}_{\mathsf{flat}}})$ and the flat-RO construction interface costs at most $\mathsf{Lift}_c(N_{\mathsf{tot}})$, where $N_{\mathsf{tot}} = N + N_{\mathsf{prim}}$ counts both the construction calls of N—including any construction oracle exposed during $\mathcal{A}$'s run and, in the win-event games, the challenger's post-output decryption or encryption checks—and the N_{prim} direct primitive queries. The derived-adversary contract (Lemma 19) expresses the resulting simulator-world execution as the identically distributed random-oracle execution of $\mathcal{B} = \mathcal{B}_{\mathrm{derived}}(\mathcal{A})$, with $q_{\mathsf{RO}}(\mathcal{B}) \le N_{\mathsf{prim}}$ and the stated construction-side inheritances, giving the first bound. The second follows by the monotonicity of ε_{G}. When G is parameterized by a target fixed before $\mathcal{A}$ runs—the discoverability target T^*—the coupling holds for each fixed target and the target-independent bound holds for the maximum defining $\mathbf{Adv}^{\mathsf{G}}_{\mathsf{TW128}}(\mathcal{A})$. □

The public-permutation privacy and INT-CTXT advantages are not stated as separate corollaries: the all-in-one AE theorem (Theorem 1) applies the lift once to the all-in-one AE adversary and then splits the resulting random-oracle advantage into the privacy and INT-CTXT envelopes of Section 5.3, adding $\mathsf{Lift}_c(N_{\mathsf{tot}})$ a single time rather than the two copies that lifting privacy and INT-CTXT separately would incur.

Corollary 5 (Public-Permutation CMTDs-3)**.** *For every* $s \ge 2$ *and every* s*-way CMTDs-3 adversary* $\mathcal{A}$ *against TW128 with* N_{prim} *direct primitive queries and total cost at most* N_{tot} *(both as in Section 4.3),*

$$\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) = \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. The CMTDs-3 challenger exposes no construction interface during $\mathcal{A}$'s run; on executions that pass the early rejecting checks it performs s deterministic decryption checks on the candidate ciphertext and opening tuples. By Theorem 7, every random-oracle CMTDs-3 adversary $\mathcal{B}$ satisfies $\mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s)$, which is nondecreasing in $q_{\mathsf{RO}}(\mathcal{B})$ by monotonicity of $\binom{\cdot+s}{s}$. The Lift Application corollary (Corollary 4) then gives the claim. □

Corollary 6 (Public-Permutation Tag-CMTDs-4)**.** *For every* $s \ge 2$ *and every* s*-way tag-CMTDs-4 adversary* $\mathcal{A}$ *against TW128 with* N_{prim} *direct primitive queries, total cost at most* N_{tot}*, and distinct-final-leaf-transcript cap* N_{leaf} *(all as in Section 4.3),*

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(N_{\mathsf{prim}} + N_{\mathsf{leaf}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s).$$

Proof. The tag-CMTDs-4 challenger exposes no construction interface during $\mathcal{A}$'s run; on executions that pass the early rejecting checks it performs the s deterministic decryption checks on the submitted bodies and common tag. By Theorem 8, every random-oracle tag-CMTDs-4 adversary $\mathcal{B}$ satisfies

$$\mathbf{Adv}^{\mathsf{tag\text{-}CMTDs\text{-}4}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{LeafColl}_{\tau_{\mathsf{leaf}}}(q_{\mathsf{RO}}(\mathcal{B}) + n_{\mathsf{leaf}}(\mathcal{B})) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}), s),$$

which is nondecreasing in both resource arguments. The Lift Application corollary (Corollary 4) gives the claim. □

Corollary 7 (Public-Permutation CDY)**.** *For every context-discoverability adversary* $\mathcal{A}$ *against TW128 with* N_{prim} *direct primitive queries and total cost at most* N_{tot} *(both as in Section 4.3),*

$$\mathbf{Adv}^{\mathsf{CDY}}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootPre}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}) = \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{N_{\mathsf{prim}} + 1}{2^{\tau_{\mathsf{root}}}}.$$

Proof. The CDY challenger exposes no construction interface during $\mathcal{A}$'s run; on executions that pass the early rejecting checks it performs the single deterministic decryption check on $C \,\|\, T^*$ for the target T^* fixed in advance. By Theorem 9, every random-oracle CDY adversary $\mathcal{B}$ satisfies $\mathbf{Adv}^{\mathsf{CDY}}_{\mathsf{RO}}(\mathcal{B}) \le \mathsf{RootPre}_{\tau_{\mathsf{root}}}(q_{\mathsf{RO}}(\mathcal{B}))$, nondecreasing in $q_{\mathsf{RO}}(\mathcal{B})$. The Lift Application corollary (Corollary 4), whose target-parameterized case covers the fixed T^*, then gives the claim. □

C Committing-Notion Bounds

The headline committing results bound full-input CMTDs-4 (Corollary 1) and its tag-projected strengthening (Theorem 3). The remaining [BH22] committing notions—CMTDs-ℓ and the CMTs-ℓ source games for $\ell \in \{1, 3, 4\}$—all reduce to the same root-collision bound, so we record the two reduction patterns once rather than accounting for each notion separately.

Lemma 20 (Forwarding Reductions for the D-Notions)**.** *Fix $s \ge 2$ and $X \in \{\mathsf{CMTDs\text{-}1}, \mathsf{CMTDs\text{-}4}\}$. The identity reduction $\mathcal{A}^* = \mathcal{A}$—preceded, for $X = \mathsf{CMTDs\text{-}1}$, by the source-game syntax, length, and pairwise-WiC_1 prechecks—turns every s-way X-adversary against TW128 into an s-way CMTDs-3 adversary with*

$$\mathbf{Adv}^{X}_{\mathit{TW128}}(\mathcal{A}) \le \mathbf{Adv}^{\mathsf{CMTDs\text{-}3}}_{\mathit{TW128}}(\mathcal{A}^*), \quad N_{\mathsf{prim}}(\mathcal{A}^*) = N_{\mathsf{prim}}(\mathcal{A}), \quad N_{\mathsf{tot}}(\mathcal{A}^*) \le N_{\mathsf{tot}}(\mathcal{A}).$$

Proof. A CMTDs-4 winner has pairwise distinct full tuples (K_i, N_i, A_i, M_i); by determinism of Dec, equal triples $(K_i, N_i, A_i) = (K_j, N_j, A_j)$ would force $M_i = M_j$, so the triples are already pairwise distinct and the winner is a CMTDs-3 winner on the same output. For CMTDs-1, pairwise WiC_1-distinctness makes the keys K_i, hence the triples, pairwise distinct; the prechecks are source-game rejections, and the eager length precheck is success preserving because TW128 decryption returns $\perp$ or a message of length $\|C\|$, so no winning transcript is discarded. Either reduction makes no primitive-interface query and performs at most the source-game decryption checks, so neither cost increases. □

Corollary 8 (Committing-Notion Root-Collision Bound)**.** *For every $s \ge 2$, every*

$$X \in \{\mathsf{CMTDs\text{-}1}, \mathsf{CMTDs\text{-}3}, \mathsf{CMTDs\text{-}4}, \mathsf{CMTs\text{-}1}, \mathsf{CMTs\text{-}3}, \mathsf{CMTs\text{-}4}\},$$

and every s-way X-adversary $\mathcal{A}$ against TW128,

$$\mathbf{Adv}^{X}_{\mathit{TW128}}(\mathcal{A}) \;\le\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \mathsf{RootColl}_{\tau_{\mathsf{root}}}(N_{\mathsf{prim}}, s) \;=\; \mathsf{Lift}_c(N_{\mathsf{tot}}) + \frac{\binom{N_{\mathsf{prim}}+s}{s}}{2^{\tau_{\mathsf{root}}(s-1)}}.$$

Proof. The D-notions reduce to CMTDs-3 by Lemma 20 (the identity reduction for CMTDs-3 itself), whose public-permutation bound is Corollary 5; monotonicity of Lift_c absorbs $N_{\mathsf{tot}}(\mathcal{A}^*) \le N_{\mathsf{tot}}(\mathcal{A})$.

The E-notions are bounded directly by the final-root candidate-collision argument of Theorem 7, with the challenger's s deterministic *encryption* checks supplying the candidate final-root transcripts in place of decryption checks. On a winning execution all s encryptions are equal, hence of equal length and with one common root tag T; pairwise WiC_ℓ-distinctness forces pairwise distinct triples (for $\ell = 4$ via determinism, as above), so Corollary 2 makes the s final-root candidates distinct while their τ_{root}-bit projections all equal T. The win is contained in the s-way candidate-collision event of Lemma 7, and the flat-sponge lift (Corollary 4) transfers the bound to the public-permutation setting at N_{prim}; the encryption checks are construction-internal work counted in N, not in N_{prim}. □

D Vectorized Kernel Details

This appendix details the architecture-specific cores summarized in Section 9.1: the instruction selection, register layout, and remainder handling of the `amd64` and `arm64` kernels.

`amd64`. The `amd64` implementation provides AVX-512 and AVX2 cores and selects between them once at startup from the `AVX512VL` CPUID feature bit. The single-duplex permutation has two variants: a general-purpose-register core adapted from the standard library's KECCAK-p permutation (using the lane-complement trick so that χ requires no separate negation), and an AVX-512 variant; the root duplex uses whichever matches the selected core. The eight-way AVX-512 core packs the lane-major state into 25 ZMM registers, one per lane with all eight instances in the register's eight 64-bit slots, and performs the permutation with no cross-lane shuffles: rotations use `VPROLQ`, and `VPTERNLOGQ` fuses the θ D-term formation and the χ and-not into single three-input logic instructions. The eight-way AVX2 core, lacking 512-bit registers, runs the eight instances as two groups of four (four 64-bit lanes per YMM register), rotating by shift-or and computing χ with `VPANDN`. The fused chunk kernels combine absorption with the permutation. The AVX-512 steady-state kernel reads the eight chunks with contiguous loads, transposes each rate block into the lane-major layout in registers (`VPUNPCKLQDQ`/`VPUNPCKHQDQ` and `VSHUFI64X2`), absorbs on the lane-major state, and transposes each output block back for a contiguous store, leaving only the 7-byte partial rate lane to a masked gather; these sequential accesses are prefetcher-friendly, whereas per-lane gathers and scatters at the chunk stride β (`VPGATHERQQ`/`VPSCATTERQQ`) would stall on cold memory and cap the long-message rate. The register-resident kernel that drains a two-to-seven-chunk remainder instead uses that per-lane gather/scatter form under a mask, reading the active chunks in place: the transpose's shuffle work is constant regardless of the chunk count and would be spent partly on inactive lanes, whereas the gathers scale with the active element count. The AVX2 path drains the same remainders with a variant of its grouped-by-four kernels that aims each inactive instance's loads and stores at a dummy block inside the kernel frame with a zero advance stride, so inactive instances touch no memory beyond the remainder and the hot loop stays branch-free.

`arm64`. The `arm64` path uses NEON plus the Armv8.2 SHA-3 extension (`FEAT_SHA3`), available on Apple Silicon and on Neoverse V1/V2 and N2. The permutation is expressed with the extension's fused instructions: `EOR3` (three-input XOR) for the θ column parities, `RAX1` (rotate-by-one and XOR) for the θ D-terms, `XAR` (XOR then rotate) to fuse the θ accumulation with the ρ rotation, and `BCAX` (bit-clear and XOR) for χ. The single-duplex core places one instance per 64-bit `D` lane across the 25 vector registers; the eight-way core packs two instances per 128-bit register and processes the batch of eight as four such pairs, using `ZIP1` and duplicating moves to interleave and extract the two instances of each lane in the fused chunk kernel. The same pair machinery backs a 2-wide kernel that drains a leaf-chunk remainder two chunks at a time—one instance per 64-bit half of a 128-bit register—at roughly twice the single-duplex per-chunk rate. Both batched kernels store the permutation state back into the batch after the closing block—the $\times 8$ kernel for its first pair, the 2-wide kernel for both instances—which is the writeback that permits the root duplex to occupy lane 0.

E Test Vectors

The following test vectors cover the corner cases of TW128 encryption, as specified in § 3.

All byte strings are written in hexadecimal. For each vector we list the key K, nonce N, associated data A, message M, ciphertext C, and tag T; the sealed output of encryption is $C \,\|\, T$. The empty string is written ε. The leaf index is encoded as a little-endian 64-bit integer and the phase suffixes are appended least-significant-bit first.

To keep the longer vectors compact, any value exceeding 32 bytes is given by its length in bytes together with its SHA-256 digest rather than in full. Every message and associated-data byte is defined by $m_i = (17i + 31) \bmod 256$ for $i = 0, 1, \ldots,$ so each such input is determined by its length alone and can be reconstructed and checked against the stated digest; a ciphertext given by its length and digest can likewise be checked by encrypting the corresponding message and hashing the result.

Vector 1. Empty associated data and empty message: the associated-data and aggregation phases are both elided, and the root tag is emitted by the single closing message call with no leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	ε
C	ε
T	`da65bbda0b20b1e936f5326458166633d1e5a3f7aebc0b318d24b051a4efa697`

Vector 2. Multi-block associated data with an empty message: the associated data spans two rate blocks and no message bytes are processed.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	168 bytes, SHA-256 `4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
M	ε
C	ε
T	`2093878418f3fca96ad3a8d1748a9be4d2a8b518e917d988fd703dcb7bf72173`

Vector 3. Empty associated data and a single-block message: the root message phase only, no leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	`1f30415263748596a7b8c9daebfc0d1e2f405162738495a6b7c8d9eafb0c1d2e`
C	`676a1441e20daea0f1ee4571ee4543842c946d59e0d3df6f1dd2f92dc4f876aa`
T	`1873332d359ff2fe5197488224af7bf6ad821875fdbd964100de424aa77342be`

Vector 4. Empty associated data and a two-block message: the root keystream is chained across two rate blocks.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	168 bytes, SHA-256 `4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
C	168 bytes, SHA-256 `df3f0a98a91a69ed474edbc52e0dc1d8e54bfd6fc514966ce7b878d48be0bf9a`
T	`aab2413074a202cfadd5eb784663b1831eb330025972d163d03e9ed5a964b765`

Vector 5. A message of exactly one chunk: the root is full, there is still no leaf, and the aggregation phase is elided so the closing root message call emits the tag.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	8183 bytes, SHA-256 `6532a370a4ab6b5f4094dd8a6016512ab8e8416abe910bc1a0b532979224151d`
C	8183 bytes, SHA-256 `8d58f8aa3413ea0ba1b8cc3ed4e9a334521d8dafe9694a0661fb8251049907e9`
T	`c2201637f1116c1303b8982622756770d23acb85384f4658fcb63c37e4b629dd`

Vector 6. A message of one chunk and one further byte: one root chunk and one leaf, exercising leaf tag aggregation.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	8184 bytes, SHA-256 `c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7`
C	8184 bytes, SHA-256 `ef1b2f7607ade503ab497fc61e62cef2b44c6328fa06e499718eb7da5fdd04d3`
T	`c96bea48e07a8431fd19721ffbe8afe6a73047f96d953e3281f10126eeca06a1`

Vector 7. A multi-leaf message of two chunks and one further byte: the full tree path with two leaves.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	ε
M	16367 bytes, SHA-256 `17e31f3700d0602aab8a686024ecc05b8192e80a531483c67dd90d5a26557c5e`
C	16367 bytes, SHA-256 `6d83d8fbdc2a7c2ef590d29a36527a45019e50d1f8c8db037959081685a46908`
T	`cdc2e50ac5380419f06772a2f3af2146050bd0340b35f61b67f839351767804e`

Vector 8. Maximum associated data with a short message: the associated-data blocks dominate the cost.

K	`000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f`
N	`202122232425262728292a2b2c2d2e2f303132333435363738393a3b3c3d3e3f`
A	337 bytes, SHA-256 `04ca7b24930bc6b5fe8b95ff1eb0933907657c1283cc3a7e69c5f7f02220f281`
M	`1f30415263748596a7b8c9daebfc0d1e`
C	`0dd615a0a05ed82a78c3999b9e80cbed`
T	`22e4e6725f548ad0660041d66cf2f18dc45c69b447cdaae1f147d33a3b6aa1a9`

Vector 9. An all-ones key and nonce with a multi-chunk message.

K	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
N	`ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff`
A	168 bytes, SHA-256 `4c7d125a3488a7603e2f403f0010a6060b8321b60b0fd2471120ae627bbe759a`
M	8184 bytes, SHA-256 `c4c383bca0808e2ce44d481ddfe56efdcd7a147dcd5a14c898ef5423f2604cd7`
C	8184 bytes, SHA-256 `49818c320ab282296639798d840cbd60751e1623f449dcbf9423371d44145dfb`
T	`310c9fff20e9651f8ed47eed9bb182a086ed1af1b3f1799e35307a4139865fc7`